UNIVERSIDAD NACIONAL DE ASUNCIÓN
FACULTAD POLITÉCNICA
INGENIERÍA EN INFORMÁTICA



TRABAJO FINAL DE GRADO

Detección y clasificación de ataques en tráfico HTTP mediante enfoques binario, multiclase y multietiqueta

---

Autores: Andrés Moisés Román Medina
Juan Andrés González Arévalos

Tutor: Cristian R. Cappo A.

SAN LORENZO – PARAGUAY
2026



UNIVERSIDAD NACIONAL DE ASUNCIÓN
FACULTAD POLITÉCNICA
INGENIERÍA EN INFORMÁTICA

Detección y clasificación de ataques en tráfico HTTP mediante enfoques binario, multiclase y multietiqueta

Autores: Andrés Moisés Román Medina
Juan Andrés González Arévalos

Tutor: Cristian R. Cappo A.

Trabajo Final de Grado presentado a la Facultad Politécnica de la Universidad Nacional de Asunción como requisito para optar al título de Ingeniero en Informática.

SAN LORENZO – PARAGUAY
2026

Resumen

Detectar que una petición HTTP es maliciosa no siempre aporta suficiente información para el análisis de seguridad. Una salida binaria no identifica el tipo de ataque. Una salida multiclase representa cada petición con una sola clase, que puede corresponder a una familia o a una combinación predefinida, pero no produce una decisión separada para cada patrón. Este trabajo evalúa si las características de una petición HTTP permiten distinguir tráfico normal de tráfico malicioso, asignar una clase de ataque y, mediante una salida multietiqueta, identificar por separado varios patrones presentes en la misma petición. Para ello se construyó un flujo experimental con etapas de preparación de datos, extracción de características, entrenamiento y evaluación. Se estudiaron tres tareas: clasificación binaria supervisada, clasificación multiclase y clasificación multietiqueta.

Se emplearon cuatro conjuntos de datos: CSIC 2010, TorpEda 2012, SR-BH 2020 y DS-Augmented-v2. Cada petición se convirtió en una representación numérica e interpretable obtenida de su método, URI, versión HTTP, encabezados y cuerpo. Las características describen, entre otros aspectos, la longitud y estructura de esos campos, su composición de caracteres, formas de codificación y señales léxicas. Se evaluaron Random Forest para clasificación binaria, HistGradientBoosting para multiclase y XGBoost con un clasificador binario por etiqueta para multietiqueta.

Los resultados variaron según la tarea y el conjunto de datos. La clasificación multiclase alcanzó un $F_{1,\text{macro}}$ superior al 97 % en TorpEda y SR-BH 2020, pero obtuvo 84,49 % en DS-Augmented-v2, donde varias clases minoritarias fueron más difíciles. En multietiqueta se obtuvo 97,60 % de $F_{1,\text{micro}}$ y 96,92 % de coincidencia exacta, aunque el rendimiento no fue uniforme entre etiquetas. Además, se comprobó que SR-BH 2020 contiene peticiones repetidas. La evaluación multietiqueta principal mantuvo juntas las peticiones que compartían la huella definida sobre quince campos HTTP, de modo que cada grupo quedara en una sola partición.

En las tres tareas, los modelos se construyeron únicamente con características de una petición individual. Los errores se concentraron en clases o etiquetas minoritarias. Las mediciones de tiempo y recursos describen el costo del flujo en el entorno ejecutado; no evalúan una integración completa con un cortafuegos de aplicaciones web (WAF).

Palabras clave: tráfico HTTP, detección de anomalías, clasificación binaria, clasificación multiclase, clasificación multietiqueta, CAPEC, WAF.

Abstract

Detecting that an HTTP request is malicious does not always provide enough information for security analysis. A binary output does not identify the attack type. A multiclass output represents each request with a single class, which may correspond to one family or to a predefined combination, but it does not make a separate decision for each pattern. This work evaluates whether features extracted from an HTTP request can distinguish normal from malicious traffic, assign an attack class, and, through a multilabel output, identify multiple patterns separately within the same request. An experimental workflow was built with data preparation, feature extraction, training, and evaluation stages. Three tasks were studied: supervised binary classification, multiclass classification, and multilabel classification.

The study used four datasets: CSIC 2010, TorpEda 2012, SR-BH 2020, and DS-Augmented-v2. Each request was converted into an interpretable numerical representation computed from its method, URI, HTTP version, headers, and body. The features describe, among other aspects, field length and structure, character composition, encoding, and lexical signals. Random Forest was evaluated for binary classification, HistGradientBoosting for multiclass classification, and XGBoost One-vs-Rest for multilabel classification.

The results varied across tasks and datasets. Multiclass classification achieved more than 97 % macro-F_1 on TorpEda and SR-BH, but 84.49 % on DS-Augmented-v2, where several minority classes were more difficult. Multilabel classification achieved 97.60 % micro-F_1 and 96.92 % exact match, although performance was not uniform across labels. SR-BH was also found to contain repeated requests. The main multilabel evaluation kept requests sharing the fingerprint defined over fifteen HTTP fields in a single partition, so that each fingerprint group occurred in only one data split.

For all three tasks, the models used only features extracted from an individual request. Errors were concentrated in minority classes or labels. Time and resource measurements describe the workflow cost in the execution environment; they do not evaluate a full integration with a web application firewall (WAF).

Keywords: HTTP traffic, anomaly detection, binary classification, multiclass classification, multilabel classification, CAPEC, WAF.

Agradecimientos

Agradecemos a nuestras familias por su apoyo constante durante el desarrollo de este trabajo. Expresamos un agradecimiento especial a Cristian R. Cappo A., cuya orientación permitió delimitar el problema, revisar las decisiones experimentales y mejorar la interpretación de los resultados. Agradecemos también a la Facultad Politécnica de la Universidad Nacional de Asunción y a sus docentes por la formación recibida.

Índice general

Resumen **i**
Abstract **ii**
Agradecimientos **iii**
Glosario, siglas y notación general **x**
1 Introducción **1**
1.1 Contexto y motivación 1
1.2 Planteamiento del problema 3
1.3 Objetivos 4
1.3.1 Objetivo general 4
1.3.2 Objetivos específicos 4
1.4 Alcance 5
1.5 Organización del documento 5
2 Fundamentos teóricos **7**
2.1 Petición HTTP como unidad analizada 7
2.2 Arquitectura web, WAF y detección de anomalías 8
2.3 Ataques web y taxonomía CAPEC 10
2.4 Aprendizaje automático y formulaciones del problema 12
2.4.1 Clasificación binaria 12
2.4.2 Clasificación multiclase 13
2.4.3 Clasificación multietiqueta 13
2.5 Representación mediante características 14
2.6 Fundamento matemático de los modelos 14
2.6.1 Árboles de decisión e impureza 14
2.6.2 Random Forest para clasificación binaria 15
2.6.3 HistGradientBoosting para clasificación multiclase 16
2.6.4 XGBoost uno contra el resto para clasificación multietiqueta 17
2.7 Métricas de evaluación 18
2.7.1 Métricas binarias 18
2.7.2 Métricas multiclase 20
2.7.3 Métricas multietiqueta 20
2.8 Importancia por permutación 21
2.9 Modelos explorados antes de la referencia binaria 22
2.9.1 One-Class SVM 22
2.9.2 Isolation Forest 23
2.9.3 Regresión logística 23
3 Estado del arte y trabajos relacionados **24**
3.1 Criterio de comparación 24
3.2 Estrategia de búsqueda bibliográfica 24
3.3 Detección de anomalías en tráfico web 26

3.4 Representaciones aprendidas y evaluación en otros dominios 26
3.5 Clasificación supervisada e interpretabilidad 27
3.6 Clasificación multietiqueta de ataques web mediante CAPEC 27
3.7 Clasificación multiclase: WAMM y DS-Augmented-v2 28
3.8 Problemas abordados 29
4 Conjuntos de datos y control de calidad 30
4.1 Panorama, procedencia y función de los corpus 30
4.2 CSIC 2010 31
4.3 TorpEda 2012 32
4.4 SR-BH 2020 32
4.4.1 Origen y estructura 32
4.4.2 Conteos publicados y observados 32
4.5 DS-Augmented-v2 33
4.6 Formato común de las peticiones 33
4.7 Peticiones repetidas en SR-BH 2020 34
4.8 Procedencia y calidad de las etiquetas 36
5 Metodología de investigación 37
5.1 Diseño general y elección de los modelos 37
5.2 Control del diseño experimental 38
5.3 Qué analiza el modelo y qué formulaciones se estudian 38
5.4 Estrategia de representación 39
5.5 Protocolos de partición 39
5.5.1 Protocolo histórico por filas 39
5.5.2 Protocolo multietiqueta por grupos de huella 40
5.6 Entrenamiento, selección y ajuste de umbrales 41
5.7 Métricas utilizadas 42
5.8 Evaluación operativa 43
5.9 Análisis de importancia 43
5.10 Reproducibilidad e incertidumbre 43
5.11 Limitaciones que pueden afectar la validez 44
5.12 Ética, privacidad y uso responsable 44
6 Implementación del flujo experimental 46
6.1 Pasos del flujo 46
6.2 Software y ejecución de las corridas 46
6.2.1 Dependencias del software 47
6.2.2 Recursos de las ejecuciones principales 47
6.3 Carga y construcción del esquema común 48
6.4 Extracción de características 48
6.4.1 Núcleo textual y estructural 48
6.4.2 Método, protocolo y encabezados 49
6.4.3 Cuerpo y estructuras internas 49
6.4.4 Ejemplo trazable de petición a características 49
6.5 Cómo se construyen las etiquetas de salida 50

6.5.1 Salida binaria 50
6.5.2 Taxonomías multiclase 51
6.5.3 Etiquetas múltiples 51
6.6 Huella, agrupación y consolidación 51
6.7 Modelos implementados 52
6.7.1 Random Forest binario 52
6.7.2 HistGradientBoosting multiclase 52
6.7.3 XGBoost uno contra el resto 52
6.8 Medición operativa 53
7 Configuración e historial de experimentos 54
7.1 Nivel de evidencia de las corridas 54
7.2 Representaciones ejecutadas 55
7.3 Orden y propósito de las corridas 55
7.4 Experimentos binarios 56
7.4.1 One-Class SVM inicial y ajuste de hiperparámetros 56
7.4.2 Pruebas por endpoint y ampliación de características 57
7.4.3 Isolation Forest 57
7.4.4 Paso a la clasificación supervisada 58
7.4.5 Ablación en CSIC 58
7.4.6 Configuración común en tres corpus 58
7.5 Evolución multiclase 58
7.5.1 De la línea base a la expansión 58
7.5.2 Eliminación de características y agrupación de clases 59
7.5.3 Incorporación del tercer corpus 59
7.6 Evolución multietiqueta 59
7.6.1 Primera corrida 59
7.6.2 Segunda corrida 59
7.7 Comprobación de coincidencias de huella entre particiones 60
8 Resultados 61
8.1 Criterio de presentación 61
8.2 Resultados previos a la referencia binaria 61
8.3 Clasificación binaria supervisada 62
8.3.1 Ablación de características en CSIC 62
8.3.2 Misma configuración aplicada por corpus 63
8.4 Evolución multiclase 64
8.4.1 Resultados globales exploratorios por etapa 64
8.5 Rendimiento operativo multiclase 65
8.6 Clasificación multietiqueta 66
8.6.1 Línea base histórica L1 66
8.6.2 Resultados por etiqueta y errores de L1 67
8.6.3 Resultado principal L2 67
8.6.4 Operación, importancia y alcance 68
8.7 Peticiones HTTP repetidas entre particiones de SR-BH 2020 69

9 Discusión **71**
9.1 Información útil de una petición 71
9.2 Diferencias entre las tres formulaciones 71
9.3 Clases y etiquetas con mayor dificultad 72
9.4 Efecto de separar las peticiones según la huella definida 73
9.5 Efecto del uso previo del conjunto de prueba 73
9.6 Comparación con antecedentes 74
9.7 Rendimiento medido y límites para un WAF 74
9.8 Resultados en relación con los objetivos 75
9.9 Lecciones aprendidas 76
9.10 Amenazas a la validez y limitaciones 76
9.11 Qué explica la selección de Random Forest 78
10 Conclusiones y trabajo futuro **79**
10.1 Conclusiones 79
10.2 Contribuciones y responsabilidades 80
10.2.1 Punto de partida y contribuciones de los autores 80
10.3 Trabajo futuro prioritario 81
A Catálogos fixed30 y request90 **82**
B Materiales y guía de ejecución **85**
Referencias **88**

Índice de tablas

1.1 Etiquetado requerido, salida e implicación operativa de las tres tareas estudiadas 3
2.1 Ejemplos simplificados de patrones observables en peticiones HTTP 10
2.2 Ejemplos conceptuales de las salidas binaria, multiclase y multietiqueta 12
2.3 Matriz de confusión binaria y convención de la clase positiva 18
3.1 Consultas empleadas en la búsqueda bibliográfica 24
3.2 Comparación crítica de los principales trabajos relacionados 25
4.1 Conjuntos de datos empleados y función experimental 30
4.2 Procedencia e información de integridad disponible para los conjuntos de datos 31
4.3 Perfil de calidad y transformaciones documentadas por conjunto de datos 31
4.4 Mapeo de las fuentes al esquema común de petición 34
4.5 Repeticiones observadas en el conjunto completo de SR-BH 2020 35
6.1 Versiones fijadas en el archivo requirements.txt suministrado 47
6.2 Asignación y consumo registrado de las corridas principales 47
6.3 Ejemplo de cómo una petición HTTP produce características numéricas incluidas en las listas finales 50
7.1 Leyenda de los identificadores experimentales. 54
7.2 Representaciones utilizadas, con nombres descriptivos unificados. 55
7.3 Secuencia de experimentos y cambio evaluado en cada etapa. 56
8.1 Resultados históricos que motivaron la selección de Random Forest. 61
8.2 Ablación exploratoria de características en CSIC 2010. 62
8.3 Características de RF-10-Gini seleccionadas desde RF-57. 63
8.4 Resultados exploratorios de Random Forest por corpus. 63
8.5 Evolución exploratoria de resultados multiclase. 64
8.6 Mediciones operativas de M8 65
8.7 Resultados multietiqueta globales de L1 y L2. Las unidades de prueba difieren 66
8.8 Métricas por etiqueta de L1 67
8.9 Métricas por etiqueta de L2 sobre la prueba separada por huella 68
8.10 Unidades de prueba que coinciden con desarrollo bajo la regla de huella 70
9.1 Relación entre los objetivos específicos y la evidencia disponible 75
9.2 Amenazas a la validez, medidas aplicadas y riesgo pendiente 76
10.1 Declaración de contribuciones y responsabilidad de la versión final 81
A.1 Treinta variables fixed30 de L1 82
A.2 Sesenta variables adicionales de request90 en L2 82
B.1 Archivos de apoyo disponibles 85
B.2 Resúmenes SHA-256 de código y esquemas finales disponibles 85
B.3 Recursos de ejecución de los flujos finales 86

Índice de figuras

2.1 Ejemplo conceptual de los cinco componentes de una petición usados por el extractor. 8
2.2 Ubicación lógica de un WAF entre el cliente y la aplicación. El presente trabajo analiza únicamente la decisión sobre una petición entrante ya reconstruida; las ramas muestran acciones posibles y no implican que deban ejecutarse todas 9
2.3 Representación conceptual de la variante con CGI de CVE-2021-41773: la URI contiene el recorrido y el cuerpo contiene la orden enviada al intérprete 11
6.1 Secuencia implementada por el flujo experimental. En el protocolo multietiqueta agrupado, la construcción de la huella antecede a la división definida en la Sección 5.5. 46
6.2 Secuencia implementada para construir la huella, consolidar los grupos y registrar las particiones multietiqueta agrupadas. 52

Glosario, siglas y notación general

Esta tabla define los símbolos y términos usados en los capítulos teóricos, metodológicos y experimentales. Una **clase** es una salida exclusiva de un problema multiclase; una **etiqueta** es una decisión binaria que puede coexistir con otras en un problema multietiqueta.

Símbolo, sigla o término	**Definición empleada en este trabajo**
Protocolo, seguridad y conceptos	
RFC	Documento técnico publicado en la serie *Request for Comments*; las RFC citadas especifican la semántica y las versiones de HTTP.
HTTP	*Hypertext Transfer Protocol*; protocolo de aplicación cuya semántica es común a sus versiones.
URI / URL	Una URI (*Uniform Resource Identifier*) identifica un recurso; una URL es una URI que además indica cómo localizarlo.
UDP	*User Datagram Protocol*; protocolo de transporte sobre el que funciona QUIC.
QUIC	Protocolo de transporte sobre UDP utilizado por HTTP/3. En los documentos actuales, QUIC se usa como nombre propio.
TLS	*Transport Layer Security*; protocolo que protege una conexión durante el transporte.
WAF	*Web Application Firewall*; cortafuegos de aplicaciones web que inspecciona tráfico HTTP para proteger uno o más servidores (OWASP Community, s. f.).
OWASP / CRS	OWASP es una comunidad dedicada a la seguridad de aplicaciones. Su conjunto básico de reglas (*Core Rule Set*, CRS) reúne reglas genéricas para WAF.
CGI	*Common Gateway Interface*; interfaz mediante la cual un servidor web puede ejecutar un programa externo para generar una respuesta.
MITRE	The MITRE Corporation; organización que mantiene el catálogo CAPEC y publica otros recursos públicos sobre ciberseguridad.
CAPEC	*Common Attack Pattern Enumeration and Classification*; catálogo jerárquico de patrones de ataque mantenido por MITRE.
WAMM	Nombre propio que Osama et al. dan a su marco de clasificación multiclase de cargas web (Osama et al., 2026).
CVE	*Common Vulnerabilities and Exposures*; identificador de una vulnerabilidad de ciberseguridad divulgada públicamente y catalogada por el programa CVE (CVE Program, s. f.).
CR / LF / CRLF	Caracteres de control que indican retorno de carro, salto de línea o ambos. Pueden separar líneas y encabezados de un mensaje HTTP.

Símbolo, sigla o término	**Definición empleada en este trabajo**
CSV / TSV / JSON / XML	Formatos de datos. CSV y TSV guardan tablas separadas por comas o tabulaciones; JSON y XML representan información estructurada.
SQL / HTML / LDAP	SQL es el lenguaje de consulta usado por bases de datos relacionales; HTML estructura contenido web; LDAP permite acceder a servicios de directorio.
SQLi / XSS / SSRF / SSTI	Abreviaturas de inyección SQL, secuencias de comandos entre sitios (*Cross-Site Scripting*), falsificación de petición del lado del servidor e inyección de plantillas del lado del servidor, respectivamente.
LDAPi / SSI / XPath	Inyección LDAP, *Server-Side Includes* y lenguaje para seleccionar elementos de un documento XML, respectivamente.
DOI	Identificador digital persistente de una publicación o un conjunto de datos.
SHA-256 / BLAKE2b	Funciones que producen resúmenes de longitud fija para comprobar integridad o construir la huella definida en este trabajo; una coincidencia no sustituye la comparación semántica del contenido.
Resumen criptográfico (*hash*)	Código de longitud fija calculado a partir de un archivo o texto; permite identificar una copia y detectar cambios.
Proxy inverso	Componente que recibe una petición destinada a una aplicación, puede inspeccionarla y luego la reenvía cuando corresponde.
Clase	Categoría de salida que el modelo debe predecir. En la tarea multiclase de este trabajo, cada petición pertenece a una sola clase de un conjunto de K opciones.
Etiqueta	Indicador de presencia o ausencia de un patrón. En clasificación multietiqueta, una petición se representa mediante un vector de indicadores y puede tener varias etiquetas activas al mismo tiempo (Tsoumakas y Katakis, 2007).
Ablación	Experimento que retira un grupo de características y mantiene sin cambios las demás condiciones. Permite observar cuánto aportaba el grupo retirado.
Secuencia léxica (*token*)	Palabra, símbolo o secuencia de texto que el extractor busca y cuenta, por ejemplo un término SQL o un separador CRLF.
Átomo	Segmento interno de texto extraído para calcular ciertas características, por ejemplo una parte delimitada de una URI o de un cuerpo.
Contenido *multipart*	Cuerpo HTTP dividido en partes, cada una con sus propios encabezados y contenido.
Biblioteca de ejecución	Implementación que ejecuta un modelo, por ejemplo scikit-learn o XGBoost.
Taxonomía	Sistema de categorías usado para nombrar y ordenar los tipos de ataque.
FI	Importancia de características (*feature importance*).
TFG	Trabajo Final de Grado; la sigla aparece también en nombres históricos de rutas y repositorios.
Datos, tareas y predicciones	
Petición HTTP	Mensaje que un cliente envía a un servidor. Puede incluir método, objetivo o URI, versión, encabezados y cuerpo.
Carga útil	Contenido de datos que un estudio toma como entrada, por ejemplo una URL, sus parámetros o el cuerpo de una petición. Su alcance depende de la fuente citada y no equivale necesariamente a una petición HTTP completa.

Símbolo, sigla o término	**Definición empleada en este trabajo**
Notación $r = (m, u, v, h, b)$	Notación definida en este trabajo para reunir los campos disponibles de una petición: m es el método, u el objetivo o URI, v la versión, h los encabezados y b el cuerpo.
Conjunto de datos / corpus	Colección de peticiones y etiquetas usada para entrenar o evaluar los modelos. En la narración se evita *dataset*, salvo en nombres o títulos originales.
Detección *one-class*	Formulación que ajusta un modelo con una sola clase de referencia, como tráfico normal, y decide si una petición es compatible con ella.
Uno contra el resto	Estrategia multietiqueta que entrena un clasificador binario separado para cada etiqueta; también se conoce como *One-vs-Rest*.
top-n	Alias histórico de una taxonomía multiclase; n indica la cantidad total de clases, no una posición en un ranking.
`fixed30` / `request90`	Identificadores históricos de representaciones multietiqueta con 30 y 90 características, respectivamente.
Línea experimental	Conjunto de corridas relacionadas que estudian una misma tarea: binaria supervisada, multiclase o multietiqueta. Cada línea puede incluir varios corpus, representaciones, configuraciones y etapas de desarrollo.
Línea base	Flujo inicial que sirve como referencia para interpretar cambios posteriores dentro de una misma tarea.
Evento de captura	Ocasión en que se registró una petición. Dos eventos distintos pueden contener los mismos campos HTTP.
Fila o registro	Entrada individual de un archivo de datos. Puede contener una petición, su salida de referencia y, según la fuente, metadatos adicionales.
Huella de petición	Identificador obtenido mediante una función determinista aplicada a campos seleccionados de una petición. Dos registros con la misma huella se tratan como coincidentes bajo la regla definida, aunque ello no implica necesariamente igualdad byte a byte ni equivalencia semántica.
Partición	Parte de un conjunto de datos destinada a entrenamiento, validación, ajuste de umbrales o prueba.
Característica	Número calculado a partir de una petición, por ejemplo su longitud o la cantidad de parámetros. El conjunto ordenado de esos números forma el vector que recibe el modelo.
Partición final reservada	Conjunto que no se usa para elegir la configuración ni el umbral del modelo.
Entrenamiento / validación / ajuste de umbrales / prueba	Etapas con funciones distintas: en el entrenamiento el modelo aprende de los ejemplos; la validación compara configuraciones; el bloque de ajuste fija los umbrales; y la prueba mide el resultado final.
Semilla	Número que permite repetir una operación aleatoria, como una división de datos, bajo las mismas condiciones.
n	Cantidad de peticiones o instancias del conjunto considerado.
d	Cantidad de características numéricas por petición.
K	Cantidad de clases en una tarea multiclase.
L	Cantidad de etiquetas binarias en una tarea multietiqueta.
$x_i \in \mathbb{R}^d$	Vector de características de la petición i.
$X \in \mathbb{R}^{n \times d}$	Matriz de diseño; cada fila corresponde a una petición.

Símbolo, sigla o término	**Definición empleada en este trabajo**
y_i	Salida verdadera de la petición i: escalar en binaria o multiclase y vector $(y_{i1},\ldots,y_{iL})$ en multietiqueta. En binaria, 0 representa normal y 1 ataque.
$\widehat{y}_i$	Predicción del modelo para la petición i: escalar en binaria o multiclase y vector en multietiqueta.
$Y,\widehat{Y}\in\{0,1\}^{n\times L}$	Matrices verdadera y predicha de una tarea multietiqueta.
$y_{i\ell},\widehat{y}_{i\ell}$	Valor verdadero y predicho para la etiqueta ℓ de la petición i.
$p_\ell(x)$	Puntuación o probabilidad estimada para la etiqueta ℓ.
τ,τ_ℓ	Umbral de decisión global o específico de una etiqueta.
$\mathbb{I}[\cdot]$	Función indicadora: vale 1 si la condición es verdadera y 0 en caso contrario.
w_i / W_t	w_i es el peso de la petición i; $W_t=\sum_{i\in t} w_i$ es la suma de pesos dentro del nodo t, no una proporción.
p_{tc} / π_k	p_{tc} es la proporción ponderada de la clase c dentro del nodo t; π_k es la proporción ponderada global de la clase k.
ω_ℓ	Multiplicador aplicado a los ejemplos positivos de la etiqueta ℓ en la pérdida multietiqueta; no es una proporción.
$\delta_{\max}$	Límite opcional del valor de una hoja en la configuración de XGBoost; cero desactiva ese límite.
Métricas de evaluación	
TP,TN,FP,FN	Verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos.
TPR / sensibilidad	Tasa de verdaderos positivos: $TP/(TP+FN)$; también se denomina *recall*.
TNR / especificidad	Tasa de verdaderos negativos: $TN/(TN+FP)$.
FPR / FNR	Tasas de falsos positivos y falsos negativos: $FP/(FP+TN)$ y $FN/(FN+TP)$.
F_1	Media armónica de precisión y sensibilidad.
MCC	Coeficiente de correlación de Matthews.
ROC-AUC	Área bajo la curva de tasa de verdaderos positivos frente a tasa de falsos positivos.
AP	Precisión promedio; resume la curva precisión–sensibilidad como una suma de precisiones ponderadas por sus incrementos. Es la medida calculada por `average_precision_score`.
AUC-PR (`aucpr`)	Área bajo la curva precisión–sensibilidad usada internamente por XGBoost para la parada temprana multietiqueta. No se identifica numéricamente con AP porque emplea otra regla de integración.
n_k	Soporte o cantidad de instancias de la clase k.
HL / EM	Pérdida de Hamming y coincidencia exacta en multietiqueta.
Importancia y operación	
R	Cantidad de permutaciones usadas al estimar importancia.
$X^{(j,r)}$	Matriz X con la característica j permutada en la repetición r.
p50 / p95 / p99	Percentiles 50, 95 y 99 de la latencia por petición.
Tasa de procesamiento	Cantidad de peticiones procesadas por segundo.

Símbolo, sigla o término	**Definición empleada en este trabajo**
CPU / GPU	Unidad central de procesamiento y unidad de procesamiento gráfico, respectivamente. Una GPU solicitada al planificador no prueba por sí sola que el modelo la haya utilizado.
RAM	Memoria principal disponible para el proceso; en los lanzadores del clúster se expresa como recurso solicitado, no necesariamente como capacidad física total del nodo.
RSS / MaxRSS	Memoria física ocupada por un proceso y máximo registrado para un trabajo, respectivamente.
MiB / GiB	Unidades binarias de memoria equivalentes a 2^{20} y 2^{30} bytes.
IP	Protocolo de Internet; una dirección IP identifica un extremo de red dentro del alcance de ese protocolo.
SO	Sistema operativo.
N/D	Dato no disponible en la fuente consultada.
OpenMP	Interfaz de programación para paralelismo de memoria compartida; puede afectar cantidad de hilos, tiempos y repetibilidad numérica.
Slurm	Gestor de trabajos del clúster usado para solicitar recursos y ejecutar los programas.
Bash	Intérprete de órdenes utilizado por los lanzadores de las corridas.
BAcc	Exactitud balanceada, promedio de la sensibilidad por clase; en el caso binario equivale a la media entre sensibilidad y especificidad.

Capítulo 1

Introducción

1.1 Contexto y motivación

Las aplicaciones web reciben datos de sus clientes mediante peticiones HTTP. El método, la URI, los encabezados, los parámetros y el cuerpo describen la operación solicitada, pero esos mismos campos pueden transportar entradas maliciosas. Por ejemplo, un parámetro puede contener una expresión de inyección SQL, una ruta puede intentar salir del directorio permitido y un encabezado puede incorporar separadores de línea destinados a alterar una respuesta. Un cortafuegos de aplicaciones web (WAF, del inglés *Web Application Firewall*) inspecciona estas peticiones antes de que alcancen la aplicación. Los WAF basados en reglas o firmas permiten relacionar cada decisión con un patrón conocido, pero dependen de un catálogo que debe mantenerse y puede no cubrir variantes nuevas o modificadas (Kruegel y Vigna, 2003; OWASP Community, s. f.).

Detectar estas entradas antes de que la aplicación las interprete es un problema de ciberseguridad. Una petición maliciosa no reconocida puede alcanzar componentes que procesan parámetros, rutas, encabezados o cuerpos. Una petición legítima marcada por error puede aumentar la carga de revisión y, si la decisión se utiliza para bloquear, impedir una operación válida. Por ello, una detección útil debe reducir los ataques que pasan inadvertidos sin convertir el tráfico normal poco frecuente en una cantidad inmanejable de alertas (Ingham e Inoue, 2007; Kruegel y Vigna, 2003; OWASP Community, s. f.).

Un caso real muestra las posibles consecuencias. CVE-2021-41773 afectó a Apache HTTP Server 2.4.49 y fue explotada en sistemas reales. Una petición con una ruta manipulada podía exponer archivos y, en configuraciones que permitían ejecutar programas desde el servidor, alcanzar ejecución remota de código (Apache HTTP Server Project, 2021). La Sección 2.3 desarrolla las condiciones del caso y explica por qué su variante con CGI contiene indicios que pueden representarse mediante CAPEC-126 y CAPEC-88.

En este trabajo, cada ejemplo analizado corresponde a una *petición entrante*, antes de que el servidor genere una respuesta. Una petición indica una acción mediante un método HTTP, como `GET` o `POST`; identifica el recurso solicitado mediante una URI; declara una versión de HTTP; y puede incluir

encabezados, parámetros y un cuerpo. Las reglas que definen el significado y la estructura de estos elementos se encuentran en RFC 9110 y RFC 9112 (Fielding et al., 2022a,b).

La **detección de anomalías** ofrece una estrategia complementaria a las reglas: aprende una descripción del comportamiento habitual y señala las observaciones que se apartan de ella (Chandola et al., 2009; Kruegel y Vigna, 2003). Esta opción resulta especialmente pertinente cuando se dispone de un historial de tráfico normal, pero no de ejemplos representativos de todas las variantes de ataque. Cuando existen peticiones normales y maliciosas rotuladas, el aprendizaje supervisado permite responder preguntas más específicas.

Desde el punto de vista de la ciberseguridad, una salida binaria sólo indica si una petición debe tratarse como normal o como ataque. Puede servir para filtrar o priorizar, pero no informa qué patrón debe investigar el analista. Una salida multiclase produce una sola clase por petición. Esa clase puede representar una familia individual —por ejemplo, inyección SQL o recorrido de directorios— o una combinación exacta definida de antemano como clase compuesta (MITRE, s. f.-b; Osama et al., 2026).

La salida multiclase no genera una decisión separada para cada patrón: una combinación no prevista debe excluirse, agruparse o incorporarse como una clase nueva. La clasificación multietiqueta sí produce una decisión binaria por etiqueta y puede activar varias en la misma petición (Sureda Riera et al., 2022a; Tsoumakas y Katakis, 2007). Esa salida describe los patrones atribuidos a la petición; no demuestra que el ataque haya tenido éxito ni determina por sí sola la respuesta del operador. La detección *one-class* responde a otra necesidad: señalar comportamientos atípicos cuando el modelo se entrena exclusivamente con tráfico normal (Epp et al., 2018).

Estas formulaciones responden a necesidades de análisis diferentes: decidir qué petición revisar primero, distinguir la familia que motivó una alerta, y conservar varios patrones presentes en una misma petición. La Tabla 1.1 compara la información que requiere cada tarea, la salida que produce y su utilidad práctica.

Tabla 1.1: Etiquetado requerido, salida e implicación operativa de las tres tareas estudiadas.

Tarea	**Etiquetado requerido**	**Salida**	**Implicación operativa**
Clasificación binaria supervisada	Ejemplos rotulados de tráfico normal y de ataques.	Normal o ataque.	Permite filtrar o priorizar, pero no identifica la familia del ataque.
Clasificación multiclase	Una clase mutuamente excluyente por petición.	Una clase individual o compuesta predefinida entre K.	Resume la petición en una sola clase; no produce una decisión separada por etiqueta.
Clasificación multietiqueta	Presencia o ausencia de cada patrón en una misma petición.	Vector de L decisiones de tipo sí/no.	Conserva varios patrones atribuidos a la petición, pero requiere decidir un umbral para cada etiqueta.

Las características forman una representación tabular y el efecto de una puede depender de otras variables de la petición. Además, las clases tienen frecuencias diferentes, por lo que las métricas globales deben acompañarse de resultados por clase. El tamaño de los corpus también obliga a medir el tiempo y la memoria empleados durante el procesamiento.

Este trabajo compara la información producida por cada formulación: una decisión normal/ataque, una categoría predefinida o varias etiquetas CAPEC predichas por separado.

1.2 Planteamiento del problema

El problema de ciberseguridad no se limita a decidir si una petición es sospechosa. Una alerta aporta más contexto al análisis cuando también identifica la familia del ataque y, cuando corresponde, conserva varios patrones atribuidos a una misma petición. Una salida binaria concentra la detección, pero deja pendiente la identificación del ataque; una salida multiclase aporta una categoría individual o compuesta, pero no desglosa por separado las etiquetas que contiene.

A partir de esta necesidad aparecen dificultades metodológicas. El tráfico legítimo de una aplicación puede variar en método, URI, parámetros, encabezados y cuerpo, por lo que apartarse de un patrón frecuente no implica automáticamente un ataque. Además, las categorías maliciosas suelen estar desbalanceadas y una métrica global puede ocultar errores en las clases con pocos ejemplos (Ingham e Inoue, 2007; Kruegel y Vigna, 2003; Saito y Rehmsmeier, 2015). Las variantes que no coinciden con una regla conocida motivan el uso de detección de anomalías, pero esta estrategia también puede señalar como sospechosas peticiones legítimas poco frecuentes.

Ninguno de los conjuntos utilizados reúne por sí solo todas las salidas necesarias para estudiar las tres tareas. Por esa razón se emplean varios corpus con funciones distintas. CSIC 2010 aporta un escenario controlado con las etiquetas NORMAL y ANÓMALO (Information Security Institute, s. f.); TorpEda 2012 incorpora familias de ataques (Torrano-Giménez et al., 2012); SR-BH 2020 registra tráfico de un servidor señuelo (*honeypot*), expuesto para atraer y registrar actividad maliciosa, y conserva varias etiquetas CAPEC por petición (Sureda Riera et al., 2022a); y DS-Augmented-v2 amplía las categorías de cargas web mediante un proceso de refinamiento y aumento (Osama et al., 2026). Como sus etiquetas tienen distinto nivel de detalle y SR-BH 2020 contiene peticiones repetidas, cada corpus se prepara y evalúa por separado; no se los integra como muestras de una misma población.

Existe además un riesgo experimental: una misma petición puede aparecer en varias filas. Si una copia se usa durante el desarrollo y otra queda en la prueba, la métrica ya no mide exclusivamente el comportamiento ante peticiones sin coincidencias bajo la regla de identidad adoptada. El procedimiento para identificar y agrupar estas repeticiones se presenta en los Capítulos 4, 5 y 6.

Con estos antecedentes, la pregunta que guía el trabajo es hasta qué punto las características calculadas a partir de una sola petición HTTP permiten detectar tráfico malicioso, identificar una familia de ataque y, cuando las etiquetas lo admiten, conservar varios patrones simultáneos. Cada tarea se estudia con los datos y el protocolo disponibles; sus resultados se interpretan dentro de cada corpus.

1.3 Objetivos

1.3.1 Objetivo general

En los objetivos, *tipificar* significa identificar el tipo o la familia de un ataque.

Diseñar, documentar y evaluar un flujo de aprendizaje automático para detectar y tipificar ataques a partir de peticiones HTTP mediante formulaciones binaria, multiclase y multietiqueta, y medir el costo de ejecución en los entornos registrados, sin extrapolarlo a un WAF en producción. La evidencia se analiza dentro de cada corpus y protocolo, sin tratarlos como poblaciones intercambiables.

1.3.2 Objetivos específicos

1. Construir un esquema común para describir las peticiones HTTP disponibles en los corpus estudiados, dejando identificadas las diferencias de representación que permanecen entre las tres tareas.

2. Definir las tareas binaria supervisada, multiclase y multietiqueta de acuerdo con el nivel de detalle del etiquetado disponible en cada corpus.

3. Implementar un algoritmo de aprendizaje automático adecuado para cada tarea y analizar sus resultados dentro del corpus, la representación y el protocolo correspondientes.

4. Documentar las particiones y las decisiones de desarrollo, identificar coincidencias de huella y registrar el uso previo del conjunto de prueba, y aplicar una partición por grupos según una regla de identidad explícita y reproducible cuando los archivos disponibles lo permitan.
5. Evaluar el desempeño con métricas adecuadas a cada tipo de tarea.
6. Medir latencia, tasa de procesamiento, CPU, memoria y tamaño de modelo en los entornos registrados, y delimitar qué permiten inferir esas mediciones sobre un contexto tipo WAF.
7. Situar los resultados frente a antecedentes relevantes y examinar el aporte de los grupos de características, teniendo en cuenta las diferencias de datos y protocolos.

La implementación emplea Random Forest, HistGradientBoosting y XGBoost con estrategia uno contra el resto. Los Capítulos 2, 5 y 6 explican sus funciones de decisión, los criterios del estudio y su implementación. El Capítulo 7 identifica la configuración aplicada en cada corrida.

En estos objetivos, «representación» se refiere al conjunto de variables numéricas calculadas para cada petición. La «granularidad» es el nivel de detalle de las etiquetas disponibles. Existe contaminación cuando información reservada para validación o prueba influye indebidamente en el entrenamiento o en la selección del modelo. El costo operativo se evalúa aquí mediante recursos y tiempos medidos localmente.

1.4 Alcance

El trabajo diseña y evalúa modelos con datos almacenados, no con tráfico en vivo. No implementa un WAF completo, no intercepta conexiones de producción y no mide red, TLS, colas, concurrencia real ni el costo de un motor de reglas. La expresión «contexto tipo WAF» se limita aquí al tiempo y los recursos necesarios para calcular características y producir una predicción sobre una petición ya disponible. El flujo recibe peticiones ya reconstruidas en campos lógicos. No termina conexiones TLS ni decodifica directamente tramas HTTP/2 o HTTP/3; esas tareas corresponderían a un componente anterior del sistema.

Cada modelo se entrena y se evalúa dentro del corpus indicado. No se mide generalización temporal, no se entrena en un corpus para probar directamente en otro y no se estima la variación entre semillas.

Algunas corridas históricas se usaron para tomar decisiones de desarrollo y no deben interpretarse como confirmaciones independientes. Los capítulos metodológicos y de resultados identifican esos casos y explican qué conclusiones permite cada evaluación. El alcance de las conclusiones queda limitado a los corpus y particiones ejecutados.

1.5 Organización del documento

El Capítulo 2 presenta los conceptos y símbolos utilizados; el Capítulo 3 resume los trabajos anteriores; y el Capítulo 4 describe los datos y controles de calidad. El Capítulo 5 define el diseño de investigación y el Capítulo 6 explica cómo se implementó. El Capítulo 7 presenta las configuraciones y la secuencia

ejecutada; el Capítulo 8 informa sus resultados; el Capítulo 9 los analiza; y el Capítulo 10 sintetiza los hallazgos en relación con los objetivos, declara las contribuciones y responsabilidades de los autores y presenta el trabajo futuro.

Capítulo 2

Fundamentos teóricos

2.1 Petición HTTP como unidad analizada

Una aplicación web sigue un intercambio de solicitud y respuesta. El cliente envía una petición que identifica la operación y el recurso solicitado; el servidor o la aplicación procesa esa información y produce una respuesta. Los documentos normativos *Request for Comments* (RFC) 9110, 9112 y 9113 establecen, respectivamente, la semántica común de HTTP y su representación en HTTP/1.1 y HTTP/2. RFC 9114 especifica HTTP/3 sobre el protocolo de transporte QUIC (Bishop, 2022; Fielding et al., 2022a,b; Thomson y Benfield, 2022).

Aunque las representaciones transmitidas por la red son diferentes, después del análisis del protocolo una capa de aplicación puede exponer componentes lógicos equivalentes: método, objetivo o URI, versión o contexto del protocolo, encabezados y cuerpo. Para reunirlos se usa el símbolo r en la Ecuación (2.1):

$$r = (m, u, v, h, b), \tag{2.1}$$

donde m es el método HTTP, u el objetivo o URI, v la versión declarada o el contexto expuesto por la capa anterior, h el conjunto de encabezados y b el cuerpo opcional.

La Figura 2.1 muestra un ejemplo de cada componente. La expresión $r = (m, u, v, h, b)$ es una notación definida en este trabajo para indicar qué partes se analizan; no describe cómo se transmite la petición por la red ni constituye una codificación. En el ejemplo, el encabezado `Content-Type` indica el formato del cuerpo, y `application/x-www-form-urlencoded` representa pares de nombre y valor como `usuario=ana&clave=ejemplo`.

m – **método:** `POST`

u – **objetivo o URI:** `/login?origen=web`

v – **versión declarada:** `HTTP/1.1`

h – **encabezados:** `Host: ejemplo.test`; `Content-Type: application/x-www-form-urlencoded`

b – **cuerpo opcional:** `usuario=ana&clave=ejemplo`

Figura 2.1: Ejemplo conceptual de los cinco componentes de una petición usados por el extractor.

El extractor de características convierte cada petición en un vector numérico. Se representa con el símbolo ϕ en la Ecuación (2.2):

$$\phi : r_i \longrightarrow x_i \in \mathbb{R}^d, \qquad X = \begin{bmatrix} x_1^\top & \cdots & x_n^\top \end{bmatrix}^\top \in \mathbb{R}^{n \times d}. \tag{2.2}$$

2.2 Arquitectura web, WAF y detección de anomalías

En una arquitectura web básica, un cliente envía una petición HTTP a una aplicación y recibe una respuesta. Un WAF puede actuar como **proxy inverso**: recibe la petición destinada a la aplicación, la inspecciona y, si corresponde, la reenvía. Si la petición se admite, la aplicación la procesa; si se bloquea, no continúa por ese recorrido. La Figura 2.2 muestra esta ubicación lógica (OWASP Community, s. f.; OWASP WSTG, s. f.).

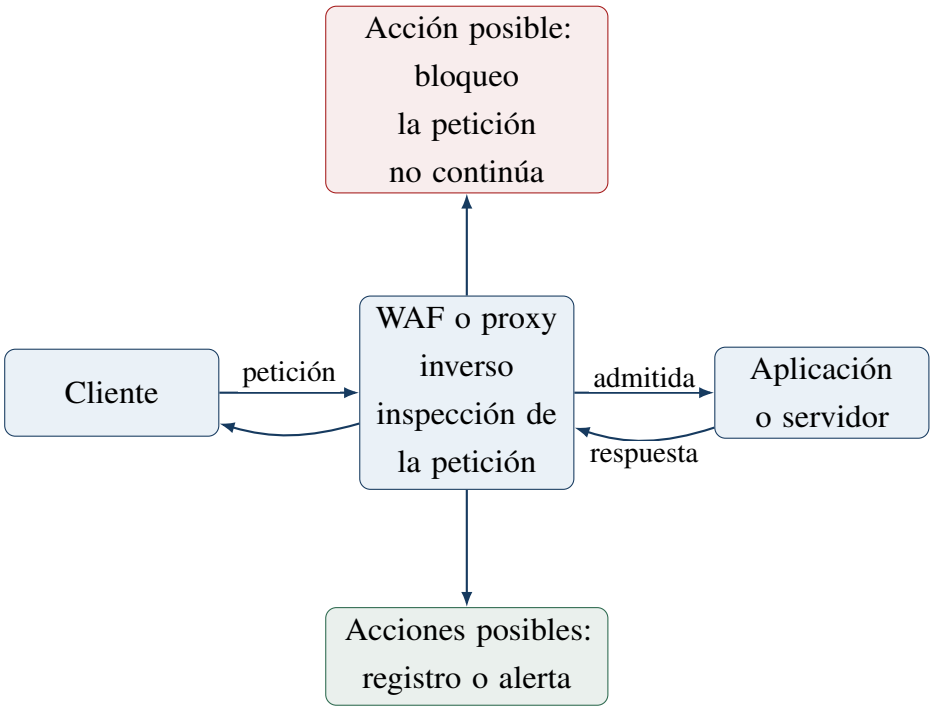


Figura 2.2: Ubicación lógica de un WAF entre el cliente y la aplicación. El presente trabajo analiza únicamente la decisión sobre una petición entrante ya reconstruida; las ramas muestran acciones posibles y no implican que deban ejecutarse todas.

Un WAF puede ejecutar acciones diferentes. **Registrar** significa conservar información del evento para revisión posterior; **alertar** significa comunicar la detección a un operador u otro sistema; y **bloquear** significa impedir o interrumpir la solicitud. Una política de sólo detección puede registrar y alertar sin bloquear. La salida de un modelo tampoco determina por sí sola cuál de estas acciones debe ejecutarse.

En un enfoque basado en reglas, cada petición se compara con firmas, expresiones o condiciones explícitas. La decisión puede relacionarse con las reglas activadas, pero la cobertura depende del catálogo disponible y de su mantenimiento. Algunos conjuntos de reglas acumulan las coincidencias en una puntuación y bloquean cuando ésta supera un umbral. En el conjunto de reglas OWASP Core Rule Set (CRS), este procedimiento se denomina **puntuación de anomalía** (*anomaly scoring*). Continúa siendo un mecanismo basado en reglas y no debe confundirse con la detección estadística de anomalías (OWASP CRS, s. f.).

La detección de anomalías aprende una descripción del comportamiento esperado y asigna una puntuación mayor a las observaciones que se apartan de ella. En tráfico web puede usar propiedades de la URI, los parámetros, los encabezados y el cuerpo. Este enfoque puede complementar las firmas cuando no existen reglas para todas las variantes relevantes (Chandola et al., 2009; Epp et al., 2018; Kruegel y Vigna, 2003). Las reglas aportan coincidencias explícitas con patrones conocidos y un detector aprendido puede priorizar solicitudes que se apartan de la referencia normal.

Un falso positivo es una petición legítima tratada como ataque; un falso negativo es una petición maliciosa tratada como normal. Al bajar el umbral se clasifican más peticiones como ataque; al subirlo, menos. Un umbral bajo puede recuperar ataques antes omitidos, pero también marcar más peticiones

legítimas. El valor elegido y las tasas de aciertos y errores que produce constituyen el **punto de operación** del detector. Su elección debe considerar la proporción esperada de ataques, el costo de cada error y la función de la salida —registro, alerta o bloqueo—; no existe un umbral universalmente óptimo (Fawcett, 2006; Saito y Rehmsmeier, 2015).

2.3 Ataques web y taxonomía CAPEC

Las familias de la Tabla 2.1 ilustran cómo un patrón puede aparecer dentro de los campos de (2.1). Son fragmentos simplificados: su presencia aislada no demuestra que una vulnerabilidad haya sido aprovechada ni que el ataque haya tenido éxito, porque la interpretación depende del contexto y de la aplicación (MITRE, s. f.-b; OWASP Top 10 Team, 2021).

Tabla 2.1: Ejemplos simplificados de patrones observables en peticiones HTTP.

Familia	Fragmento ilustrativo	Intención general
Inyección SQL (*SQL Injection*)	`id=1' OR '1'='1`	Alterar la lógica de una consulta construida sin validación adecuada.
Secuencias de comandos entre sitios (XSS)	`q=<script>alert(1)</script>`	Introducir contenido activo que un navegador podría interpretar.
Recorrido de directorios (*Path Traversal*)	`file=../../etc/passwd`	Salir del directorio previsto mediante segmentos de ruta.
División de respuesta HTTP (*HTTP Response Splitting*)	`name=a%0d%0aX-Test:1`	Inyectar los separadores CRLF —retorno de carro y salto de línea— para crear o alterar encabezados.

CAPEC (*Common Attack Pattern Enumeration and Classification*) es un catálogo público mantenido por The MITRE Corporation. Su unidad principal es el patrón de ataque: una descripción de un mecanismo recurrente utilizado para alcanzar un objetivo adversario. El catálogo ofrece nombres consistentes, relaciones, consecuencias, requisitos y mitigaciones. Por ello resulta útil para organizar y comunicar el tipo de comportamiento detectado, aunque no reemplaza la comprobación de una vulnerabilidad concreta (MITRE, s. f.-a).

CAPEC organiza el conocimiento en cuatro niveles. Una **categoría** agrupa patrones por un criterio común, pero no describe por sí sola una acción ejecutable. Un **metapatrón** (*Meta*) representa una metodología abstracta; un patrón **estándar** (*Standard*) describe una técnica más específica; y un patrón **detallado** (*Detailed*) incorpora detalles de ejecución asociados con tecnologías concretas. Un hijo suele refinar a su padre, aunque la jerarquía es principalmente organizativa y no siempre estricta (MITRE, s. f.-c). Por ejemplo, CAPEC-126, recorrido de directorios (*Path Traversal*), aparece como hijo de CAPEC-153, manipulación de datos de entrada (*Input Data Manipulation*); y CAPEC-88,

inyección de comandos del sistema operativo (*OS Command Injection*), se relaciona jerárquicamente con CAPEC-248, inyección de comandos (*Command Injection*) (MITRE, 2023b,c,d,e). El número identifica la entrada, pero no expresa su severidad ni su nivel jerárquico.

Una petición puede contener indicios compatibles con varios patrones. Distintos campos pueden transportar entradas diferentes; un patrón puede facilitar otro; o una taxonomía jerárquica puede describir el mismo comportamiento en más de un nivel. CAPEC registra, por ejemplo, que la división de respuestas HTTP (*HTTP Response Splitting*) puede preceder a la ejecución de código en el navegador mediante XSS (MITRE, 2023a). Esta coexistencia describe indicios compatibles, no confirma por sí sola cuál sería el efecto en una aplicación concreta.

El programa de vulnerabilidades y exposiciones comunes (CVE, del inglés *Common Vulnerabilities and Exposures*) asigna identificadores a vulnerabilidades de ciberseguridad divulgadas públicamente (CVE Program, s. f.). Los casos documentados permiten observar que una misma petición HTTP puede contener indicios compatibles con más de un patrón. En CVE-2016-1899, secuencias CRLF introducidas mediante un parámetro podían alterar encabezados y dar lugar a división de respuesta HTTP o XSS (NIST, 2016).

Caso real: CVE-2021-41773. Esta vulnerabilidad afectó a Apache HTTP Server 2.4.49 y se originó en un cambio defectuoso de la normalización de rutas. Una petición manipulada podía alcanzar archivos fuera de los directorios configurados mediante directivas de tipo `Alias`. Cuando los archivos externos no estaban protegidos por la restricción habitual `Require all denied`, la petición podía producir divulgación de archivos; si además CGI (*Common Gateway Interface*) estaba habilitado para las rutas afectadas, podía alcanzarse ejecución remota de código. Apache informó que la vulnerabilidad era objeto de explotación en sistemas reales (Apache HTTP Server Project, 2021).

La Figura 2.3 abstrae la variante con CGI mediante una petición sintética: la URI contiene un indicio de recorrido compatible con CAPEC-126 y el cuerpo contiene una orden compatible con CAPEC-88. La asociación del cuerpo con CAPEC-88 es una interpretación analítica del indicio observable en esa variante y no una asignación oficial de ese CAPEC al CVE. Una salida multietiqueta puede conservar ambas etiquetas por separado.

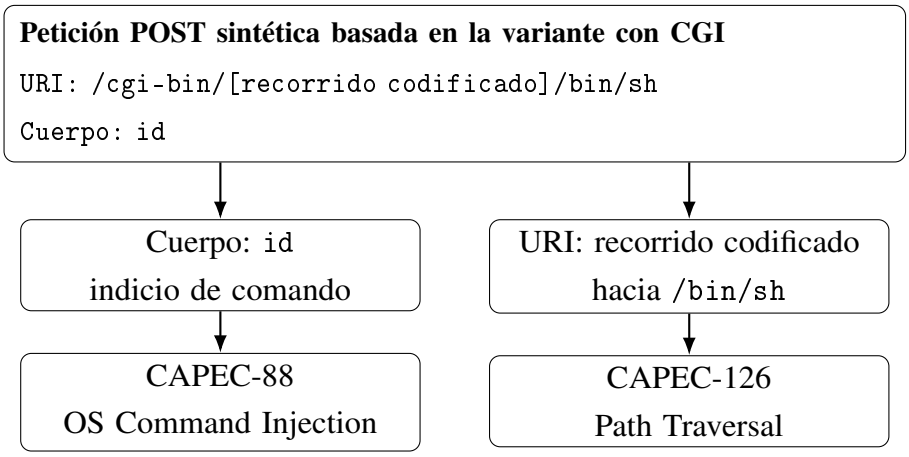


Figura 2.3: Representación conceptual de la variante con CGI de CVE-2021-41773: la URI contiene el recorrido y el cuerpo contiene la orden enviada al intérprete.

2.4 Aprendizaje automático y formulaciones del problema

El aprendizaje automático comprende métodos que estiman relaciones o estructuras a partir de datos en lugar de recibir todas las reglas de decisión escritas de forma explícita. Cada petición se representa mediante un vector $x_i \in \mathbb{R}^d$. En aprendizaje supervisado, los ejemplos incluyen una salida conocida y_i y el modelo aprende una relación entre entrada y salida. En aprendizaje no supervisado no se proporcionan salidas objetivo y se buscan estructuras, agrupaciones o desviaciones. El aprendizaje semisupervisado combina ejemplos etiquetados con otros sin etiquetar (Bishop, 2006).

La clasificación *one-class* constituye un escenario particular: el modelo aprende una descripción de una clase objetivo y decide si una observación nueva es compatible con ella. En detección de anomalías, la clase objetivo suele representar el comportamiento normal (Chandola et al., 2009; Tax y Duin, 2004).

Después de convertir una petición en x_i, las tareas supervisadas emplean $\mathscr{D} = \{(x_i, y_i)\}_{i=1}^{n}$. La forma de y_i y de la decisión depende de la tarea. La Tabla 2.2 compara las salidas mediante ejemplos conceptuales; no representa filas de los corpus.

Tabla 2.2: Ejemplos conceptuales de las salidas binaria, multiclase y multietiqueta.

Petición conceptual	**Binaria**	**Multiclase**	**Multietiqueta**
Operación legítima	0	NORMAL	Vector sin etiquetas de ataque.
Indicio de SQL Injection	1	SQL Injection	CAPEC-66 activa.
Indicios compatibles con CAPEC-88 y CAPEC-126	1	Una familia elegida o una clase compuesta predefinida	CAPEC-88 y CAPEC-126 activas.

2.4.1 Clasificación binaria

En la formulación binaria se adopta la convención de la Ecuación (2.3): 0 significa normal y 1 ataque o anomalía.

$$y_i \in \{0, 1\}. \tag{2.3}$$

Un clasificador binario supervisado aprende con observaciones de ambas categorías y estima la probabilidad de la clase positiva. La decisión se obtiene comparando esa probabilidad con un umbral (Bishop, 2006).

2.4.2 Clasificación multiclase

En multiclase, cada petición pertenece a una sola de K clases mutuamente excluyentes (Bishop, 2006). La variable objetivo y la regla general de predicción se definen en (2.4) y (2.5):

$$y_i \in \{1, \ldots, K\}, \tag{2.4}$$

$$\widehat{y}_i = \underset{k \in \{1,\ldots,K\}}{\arg\max}\ f_k(x_i), \tag{2.5}$$

donde $f_k(x_i)$ es la puntuación estimada para la clase k. Cuando una petición tiene varias etiquetas CAPEC, una reducción por familias individuales debe elegir o agrupar categorías y puede perder información sobre etiquetas simultáneas. También es posible tratar cada combinación exacta admitida como una clase compuesta; esa alternativa conserva la combinación, pero requiere una clase distinta para cada conjunto previsto. Una combinación no incorporada al catálogo de clases debe excluirse, agruparse o motivar una modificación de la taxonomía. La formulación multietiqueta evita esa transformación al representar cada CAPEC como un componente separado de la salida.

2.4.3 Clasificación multietiqueta

En multietiqueta, varias etiquetas pueden estar presentes en una misma petición (Tsoumakas y Katakis, 2007). La salida verdadera es el vector binario de (2.6):

$$y_i = (y_{i1}, \ldots, y_{iL}) \in \{0,1\}^L. \tag{2.6}$$

Cada componente indica la presencia o ausencia de la etiqueta ℓ. Una puntuación $p_\ell(x_i)$ se convierte en decisión mediante (2.7):

$$\widehat{y}_{i\ell} = \mathbb{I}[p_\ell(x_i) \geq \tau_\ell], \qquad \ell = 1, \ldots, L. \tag{2.7}$$

Como las etiquetas no son mutuamente excluyentes, la salida puede contener ninguna, una o varias. La formulación no exige independencia entre ellas; la descomposición en decisiones binarias separadas corresponde a estrategias de implementación como uno contra el resto.

2.5 Representación mediante características

Los algoritmos empleados reciben valores numéricos, por lo que el texto y la estructura de una petición deben transformarse en un vector. Las representaciones **generales** describen propiedades que no corresponden de antemano a un ataque concreto, como longitudes, composición de caracteres, distribución de bytes, entropía y conteos de secuencias. Las características **expertas** se diseñan a partir de conocimiento sobre mecanismos conocidos, por ejemplo términos SQL, delimitadores de etiquetas, separadores CRLF, operadores del intérprete de órdenes (*shell*) o segmentos `../`. Pueden facilitar la identificación de patrones concretos, pero introducen supuestos del diseñador sobre las señales relevantes (Kruegel y Vigna, 2003; Riverol et al., 2024; Wang y Stolfo, 2004).

La representación estudiada es tabular y combina ambos tipos: propiedades de longitud, estructura, caracteres, codificación y entropía junto con señales léxicas dirigidas a familias de ataques. La Ecuación (2.8) define,

$$H(Z) = -\sum_{c \in \mathscr{A}} p(c) \log_2 p(c). \tag{2.8}$$

Una entropía alta sólo expresa mayor diversidad de caracteres; no implica un ataque. De forma análoga, el conteo de una secuencia léxica sospechosa es una variable que el modelo combina con otras, no una regla de bloqueo autosuficiente.

2.6 Fundamento matemático de los modelos

Se eligieron modelos basados en árboles porque reciben directamente la representación tabular y pueden combinar condiciones sobre varias características. Random Forest se aplica a la salida binaria; HistGradientBoosting produce una clase multiclase; y XGBoost entrena un clasificador binario separado para cada etiqueta CAPEC. Las comparaciones posteriores se realizan entre los flujos completos definidos para cada tarea. Esta sección presenta la función general que optimiza cada método y la regla que convierte su salida en una decisión. Las decisiones específicas de partición, selección de modelos y ajuste de umbrales se documentan en el Capítulo 5; los hiperparámetros y detalles del código, en el Capítulo 6; y la configuración de cada corrida, en el Capítulo 7.

2.6.1 Árboles de decisión e impureza

Un árbol divide de forma recursiva el espacio de características. Sea w_i el peso efectivo del ejemplo i. En un nodo t, su masa total W_t y la proporción ponderada de la clase c son

$$W_t = \sum_{i \in t} w_i, \qquad p_{tc} = \frac{1}{W_t} \sum_{i \in t} w_i \mathbb{I}[y_i = c]. \tag{2.9}$$

El criterio de Gini usado por Random Forest se define como

$$G(t) = 1 - \sum_{c=1}^{K} p_{tc}^2. \tag{2.10}$$

Para una división candidata que produce los nodos izquierdo t_L y derecho t_R, la reducción de impureza es

$$\Delta G = G(t) - \frac{W_{t_L}}{W_t} G(t_L) - \frac{W_{t_R}}{W_t} G(t_R). \tag{2.11}$$

Cuando todos los pesos valen uno, $W_t = N_t$ y la expresión se reduce a la forma no ponderada. En Random Forest, el peso efectivo también refleja la multiplicidad con la que una observación aparece en una muestra obtenida por muestreo con reemplazo (*bootstrap*). El árbol selecciona, entre las características y puntos de corte candidatos, la división que maximiza (2.11). Las hojas estiman una distribución de clases a partir de los ejemplos que llegan a ellas. Un árbol profundo puede adaptarse demasiado a los datos de entrenamiento; los ensambles reducen este problema al combinar árboles diferentes (Breiman, 2001).

2.6.2 Random Forest para clasificación binaria

Random Forest entrena B árboles. Cada árbol usa una muestra *bootstrap*, obtenida por muestreo con reemplazo, y en cada nodo examina un subconjunto aleatorio de las d características. Estas dos fuentes de variación reducen la semejanza entre los árboles. Si $\widehat{p}_b(y = 1 \mid x)$ es la probabilidad estimada por el árbol b, la probabilidad del bosque es

$$\widehat{p}_{\mathrm{RF}}(y = 1 \mid x) = \frac{1}{B} \sum_{b=1}^{B} \widehat{p}_b(y = 1 \mid x), \qquad \widehat{y} = \mathbb{I}[\widehat{p}_{\mathrm{RF}}(y = 1 \mid x) \geq \tau]\,. \tag{2.12}$$

Cuando se aplican pesos balanceados, para K clases, n observaciones y n_c observaciones de la clase c, cada ejemplo de esa clase recibe

$$w_c = \frac{n}{K\, n_c}. \tag{2.13}$$

Así, las clases minoritarias intervienen con mayor peso en las proporciones p_{tc} y en las masas de (2.11) (Scikit-learn developers, 2026b). Al reducir el umbral τ se predicen más positivos; al aumentarlo se predicen menos. El conjunto empleado para elegirlo y la configuración experimental se describen en los capítulos metodológicos.

La importancia basada en Gini atribuye a cada división la contribución $(W_t/W_{\mathrm{raíz}})\Delta G$, la acumula por característica a través de los árboles y normaliza el resultado (Scikit-learn developers, 2026d). No es igual a la importancia por permutación de la Ecuación (2.49): la primera se calcula durante el ajuste y puede favorecer variables con más puntos de corte, mientras que la segunda mide el cambio de una métrica al alterar una columna.

2.6.3 HistGradientBoosting para clasificación multiclase

La potenciación por gradiente (*gradient boosting*) construye un modelo aditivo. Sea w_i el peso de la petición i y $y_{ik} = \mathbb{I}[y_i = k]$. La proporción ponderada inicial de la clase k es

$$\pi_k = \frac{\sum_{i=1}^{n} w_i y_{ik}}{\sum_{i=1}^{n} w_i}. \tag{2.14}$$

Aquí π_k es la proporción global de la clase; p_{tc} en (2.9) es, en cambio, una proporción dentro del nodo t, y W_t es la masa total de ese nodo. Si todos los pesos valen uno, $\pi_k = n_k/n$. Una elección equivalente de las puntuaciones internas iniciales es $F_k^{(0)} = \log \pi_k$. Estas puntuaciones sin normalizar también se denominan *logits*; sumarles una misma constante no cambia las probabilidades. En cada ronda s se ajusta un árbol para cada una de las K clases a fin de aproximar la dirección que reduce la pérdida del ensamble anterior (Friedman, 2001):

$$F_k^{(s)}(x) = F_k^{(s-1)}(x) + \eta f_{s,k}(x), \tag{2.15}$$

donde $f_{s,k}$ es el árbol nuevo de la clase k y η es la tasa de aprendizaje. La función *softmax* convierte las K puntuaciones en probabilidades que suman uno:

$$p_{ik} = \frac{\exp(F_k(x_i))}{\sum_{c=1}^{K} \exp(F_c(x_i))}, \qquad \widehat{y}_i = \arg\max_k \, p_{ik}. \tag{2.16}$$

La pérdida logarítmica multiclase ponderada es

$$\mathscr{L}_{\text{multi}} = -\sum_{i=1}^{n} w_i \sum_{k=1}^{K} y_{ik} \log p_{ik}. \tag{2.17}$$

El gradiente respecto de la puntuación interna, la componente diagonal de su curvatura y el gradiente negativo son, respectivamente,

$$g_{ik} = w_i(p_{ik} - y_{ik}), \qquad h_{ik} = w_i p_{ik}(1 - p_{ik}), \qquad -g_{ik} = w_i(y_{ik} - p_{ik}). \tag{2.18}$$

Los árboles aproximan esa dirección correctiva y la curvatura interviene al valorar cada división.

HistGradientBoosting no ordena todos los valores continuos en cada nodo. Primero asigna los valores de cada característica a un número finito de intervalos; después acumula gradientes y curvaturas por intervalo. Esta aproximación reduce el costo de buscar puntos de corte en matrices grandes (Scikit-learn developers, 2026c). La **regularización** penaliza modelos excesivamente complejos, y la **parada temprana** detiene la adición de árboles cuando la validación deja de mejorar. La ponderación de clases y los demás hiperparámetros empleados se documentan en los capítulos metodológicos y de implementación.

2.6.4 XGBoost uno contra el resto para clasificación multietiqueta

La estrategia uno contra el resto (*One-vs-Rest*) transforma el problema de L etiquetas en L problemas binarios (Tsoumakas y Katakis, 2007). Para la etiqueta ℓ, sea $F_\ell^{(0)}$ el margen inicial, es decir, la puntuación previa al primer árbol. Después de M_ℓ rondas, XGBoost actualiza (Chen y Guestrin, 2016)

$$F_\ell^{(s)}(x) = F_\ell^{(s-1)}(x) + \eta_\ell f_{\ell,s}(x), \qquad p_\ell(x) = \sigma(F_\ell^{(M_\ell)}(x)) = \frac{1}{1+\exp[-F_\ell^{(M_\ell)}(x)]}, \tag{2.19}$$

donde $s = 1, \ldots, M_\ell$, $f_{\ell,s}$ es el árbol añadido en la ronda s y η_ℓ es la tasa de aprendizaje. Se abrevia $p_{i\ell} = p_\ell(x_i)$. El margen inicial determina la probabilidad de partida antes de añadir árboles; su valor concreto corresponde a la configuración de XGBoost. La función objetivo combina pérdida logística y complejidad de los árboles:

$$\mathscr{L}_\ell = -\sum_{i=1}^{n} [\omega_\ell y_{i\ell} \log p_{i\ell} + (1 - y_{i\ell}) \log(1 - p_{i\ell})] + \sum_{s=1}^{M_\ell} \Omega(f_{\ell,s}), \tag{2.20}$$

$$\Omega(f) = \gamma T + \frac{\lambda}{2} \sum_{u=1}^{T} q_u^2 + \alpha \sum_{u=1}^{T} |q_u|, \tag{2.21}$$

El factor ω_ℓ aumenta la contribución de los positivos de la etiqueta ℓ; no es un peso común para todas las observaciones. Además, T es la cantidad de hojas y q_u el valor de la hoja u. El parámetro γ penaliza incorporar hojas, λ aplica regularización ℓ_2 a sus valores y α aplica regularización ℓ_1. Para simplificar la notación se omite el subíndice ℓ en α, λ, γ y los valores de hoja, aunque cada clasificador binario

puede recibir su propia configuración. Para la pérdida logística, el gradiente y la curvatura son

$$g_{i\ell} = a_{i\ell}(p_{i\ell} - y_{i\ell}), \qquad h_{i\ell} = a_{i\ell}p_{i\ell}(1 - p_{i\ell}), \qquad a_{i\ell} = 1 + (\omega_\ell - 1)y_{i\ell}. \tag{2.22}$$

Para una hoja u, sean $G_u = \sum_{i \in u} g_{i\ell}$ y $H_u = \sum_{i \in u} h_{i\ell}$. El operador de umbral suave y el valor regularizado son

$$S_\alpha(G) = \operatorname{sgn}(G)\max(|G| - \alpha, 0), \qquad q_u^\star = -\frac{S_\alpha(G_u)}{H_u + \lambda}. \tag{2.23}$$

XGBoost usa una aproximación de segundo orden basada en (2.22) para decidir si la reducción de la pérdida compensa la complejidad adicional (XGBoost developers, 2026). Los límites de actualización y los valores concretos de regularización examinados pertenecen a la implementación y se presentan en la Sección 6.7.

La salida final se calcula por etiqueta:

$$\widehat{y}_{i\ell} = \mathbb{I}[p_\ell(x_i) \geq \tau_\ell]. \tag{2.24}$$

La selección del modelo y de los umbrales se documenta en la metodología.

Los L clasificadores no modelan de forma explícita la dependencia entre etiquetas. La coexistencia se conserva porque cada decisión de (2.24) puede activarse al mismo tiempo. NORMAL se deriva cuando ninguna de las etiquetas CAPEC modeladas está activa y no constituye un clasificador adicional; es decir, se asigna cuando $\widehat{y}_{i1} = \cdots = \widehat{y}_{iL} = 0$.

2.7 Métricas de evaluación

Las métricas describen aspectos distintos del error y deben interpretarse de acuerdo con la tarea y la distribución de las clases. Ninguna medida conserva por sí sola toda la información de la matriz de confusión (Sokolova y Lapalme, 2009).

2.7.1 Métricas binarias

La Tabla 2.3 fija los cuatro conteos de una matriz de confusión cuando el ataque es la clase positiva.

Tabla 2.3: Matriz de confusión binaria y convención de la clase positiva.

	Predicción: ataque	**Predicción: normal**
Ataque real	TP	FN
Normal real	FP	TN

Las Ecuaciones (2.25), (2.26), (2.27), (2.28), (2.29), (2.30) y (2.31) definen las métricas derivadas. TPR coincide con la sensibilidad (*recall*) y TNR con la especificidad.

$$\text{Exactitud} = \frac{TP+TN}{TP+TN+FP+FN}, \tag{2.25}$$

$$\text{Precisión} = \frac{TP}{TP+FP}, \tag{2.26}$$

$$\text{TPR} = \text{Sensibilidad} = \frac{TP}{TP+FN}, \tag{2.27}$$

$$\text{TNR} = \text{Especificidad} = \frac{TN}{TN+FP}, \tag{2.28}$$

$$\text{FPR} = \frac{FP}{FP+TN} = 1-\text{TNR}, \tag{2.29}$$

$$\text{FNR} = \frac{FN}{FN+TP} = 1-\text{TPR}, \tag{2.30}$$

$$F_1 = 2\frac{\text{Precisión}\,\text{Sensibilidad}}{\text{Precisión}+\text{Sensibilidad}}. \tag{2.31}$$

La exactitud balanceada promedia la sensibilidad de ambas categorías y se define en (2.32):

$$\text{Exactitud balanceada} = \frac{\text{TPR}+\text{TNR}}{2}. \tag{2.32}$$

El MCC de (2.33) resume los cuatro conteos y toma valores entre -1 y 1; resulta informativo bajo desbalance (Chicco y Jurman, 2020).

$$\text{MCC} = \frac{TPTN-FPFN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}. \tag{2.33}$$

ROC-AUC resume TPR frente a FPR a través de umbrales (Fawcett, 2006). La curva precisión–sensibilidad, también denominada *precision–recall*, describe el compromiso entre ambas medidas. Los programas de evaluación emplean la precisión promedio (AP, del inglés *average precision*). Si (R_q, P_q) son los Q pares de sensibilidad y precisión producidos al recorrer los umbrales, ordenados de modo que la sensibilidad aumente, y $R_0 = 0$, entonces

$$\text{AP} = \sum_{q=1}^{Q} (R_q - R_{q-1}) P_q. \tag{2.34}$$

Esta suma pondera la precisión observada en cada incremento de sensibilidad; no interpola linealmente entre puntos y no equivale al área trapezoidal bajo la curva precisión–sensibilidad (Scikit-learn developers, 2026a). Ambas medidas requieren puntuaciones continuas, no sólo decisiones finales. En problemas desbalanceados, AP debe leerse junto con la proporción de la clase positiva (Saito y Rehmsmeier, 2015).

2.7.2 Métricas multiclase

Para evaluar cada clase k, se la trata temporalmente como positiva y se agrupan todas las demás como negativas. A partir de esta comparación de uno contra el resto se calculan $F_{1,k}$ y su soporte n_k. Las Ecuaciones (2.35) y (2.36) muestran dos agregaciones complementarias (Sokolova y Lapalme, 2009):

$$F_{1,\text{macro}} = \frac{1}{K}\sum_{k=1}^{K} F_{1,k}, \tag{2.35}$$

$$F_{1,\text{ponderado}} = \sum_{k=1}^{K} \frac{n_k}{n} F_{1,k}. \tag{2.36}$$

El promedio macro asigna igual peso a cada clase; el promedio ponderado concede mayor peso a las más frecuentes. La exactitud balanceada multiclase de (2.37) promedia la sensibilidad por clase:

$$\text{Exactitud balanceada}_K = \frac{1}{K}\sum_{k=1}^{K} \text{Sensibilidad}_k. \tag{2.37}$$

Por definición, esta cantidad es la sensibilidad macro. Cuando ambas columnas aparecen en una tabla de resultados, representan la misma operación y no dos métricas independientes.

El MCC multiclase se calcula directamente desde la matriz de confusión $C \in \mathbb{N}^{K\times K}$. Sean $s = \sum_{i,j} C_{ij}$, $c = \sum_k C_{kk}$, $t_k = \sum_j C_{kj}$ y $\widehat{n}_k = \sum_i C_{ik}$. Entonces

$$\text{MCC}_K = \frac{c\,s - \sum_{k=1}^{K} \widehat{n}_k t_k}{\sqrt{\left(s^2 - \sum_{k=1}^{K} \widehat{n}_k^2\right)\left(s^2 - \sum_{k=1}^{K} t_k^2\right)}}. \tag{2.38}$$

Para obtener AP macro, cada clase se trata como positiva frente a las restantes. Si AP_k es la precisión promedio de esa comparación,

$$\text{AP}_{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K} \text{AP}_k. \tag{2.39}$$

La medida necesita una puntuación por clase; no puede reconstruirse sólo a partir de la clase final.

2.7.3 Métricas multietiqueta

Las definiciones y agregaciones de esta subsección siguen la taxonomía de evaluación multietiqueta de Tsoumakas y Katakis (2007). Para cada etiqueta ℓ se calculan TP_ℓ, FP_ℓ y FN_ℓ. El promedio micro agrega primero las decisiones de todas las etiquetas según las Ecuaciones (2.40), (2.41) y (2.42):

$$P_{\text{micro}} = \frac{\sum_{\ell=1}^{L} TP_\ell}{\sum_{\ell=1}^{L} (TP_\ell + FP_\ell)}, \tag{2.40}$$

$$R_{\text{micro}} = \frac{\sum_{\ell=1}^{L} TP_\ell}{\sum_{\ell=1}^{L} (TP_\ell + FN_\ell)}, \tag{2.41}$$

$$F_{1,\text{micro}} = 2\frac{P_{\text{micro}} R_{\text{micro}}}{P_{\text{micro}} + R_{\text{micro}}}. \tag{2.42}$$

El promedio macro de (2.43) calcula primero F_1 por etiqueta y después les concede el mismo peso:

$$F_{1,\text{macro}} = \frac{1}{L}\sum_{\ell=1}^{L} F_{1,\ell}. \tag{2.43}$$

El promedio ponderado usa el soporte positivo $n_\ell = \sum_i y_{i\ell}$:

$$F_{1,\text{ponderado}} = \sum_{\ell=1}^{L} \frac{n_\ell}{\sum_{r=1}^{L} n_r} F_{1,\ell}. \tag{2.44}$$

Como una misma petición puede aportar soporte a varias etiquetas, los n_ℓ no forman una partición de las filas.

La coincidencia exacta de (2.45) sólo cuenta una petición cuando coincide todo su vector; la pérdida de Hamming, en (2.46), mide la fracción de decisiones etiqueta–petición erróneas (Tsoumakas y Katakis, 2007):

$$\text{EM} = \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}[y_i = \widehat{y}_i], \tag{2.45}$$

$$\text{HL} = \frac{1}{nL}\sum_{i=1}^{n}\sum_{\ell=1}^{L} \mathbb{I}[y_{i\ell} = \widehat{y}_{i\ell}]. \tag{2.46}$$

Si Y_i y $\widehat{Y}_i$ son los conjuntos de etiquetas verdaderas y predichas, la variante micro de Jaccard agrega las intersecciones y uniones de todas las peticiones antes de dividir:

$$J_{\text{micro}} = \frac{\sum_i |Y_i \cap \widehat{Y}_i|}{\sum_i |Y_i \cup \widehat{Y}_i|} = \frac{\sum_\ell TP_\ell}{\sum_\ell (TP_\ell + FP_\ell + FN_\ell)}. \tag{2.47}$$

La variante macro promedia la medida binaria por etiqueta (Tsoumakas y Katakis, 2007). Para AP macro multietiqueta se calcula una precisión promedio por etiqueta y luego

$$\text{AP}_{\text{macro}} = \frac{1}{L}\sum_{\ell=1}^{L} \text{AP}_\ell. \tag{2.48}$$

La variante micro aplana las L decisiones de las n peticiones y calcula una única AP sobre los nL pares etiqueta–petición.

2.8 Importancia por permutación

La importancia por permutación mide cuánto disminuye una métrica M al desordenar una característica y mantener las demás. Se supone que un valor mayor de M representa un resultado mejor. La estimación de la característica j se define en (2.49):

$$I_j = M(X, y) - \frac{1}{R}\sum_{r=1}^{R} M(X^{(j,r)}, y), \tag{2.49}$$

donde $X^{(j,r)}$ conserva la matriz X salvo por una permutación de la característica j en la repetición r.

Si la métrica disminuye, las predicciones dependían de información contenida en esa columna. Esto indica dependencia predictiva, no causalidad: la variable puede estar correlacionada con la etiqueta o con la forma en que se construyó el corpus. Dos variables relacionadas pueden repartirse la importancia y las clases con pocos ejemplos producen valores menos estables.

2.9 Modelos explorados antes de la referencia binaria

La selección de la referencia binaria estuvo precedida por modelos que responden a preguntas distintas. En una formulación one-class, el aprendizaje describe el tráfico de referencia y señala peticiones que se apartan de él. En clasificación supervisada, el entrenamiento utiliza ejemplos etiquetados de normales y ataques para separarlos. Ambos enfoques pueden producir NORMAL/ATAQUE, pero esa coincidencia de salida no iguala la información disponible durante el aprendizaje. Esta sección explica los modelos de la etapa exploratoria; sus configuraciones y resultados se presentan en los capítulos experimentales.

2.9.1 One-Class SVM

One-Class SVM (OCSVM) estima una región del espacio de características que contiene buena parte de los ejemplos de referencia. Su función de decisión combina la semejanza de una petición con un subconjunto de observaciones de entrenamiento, denominadas vectores de soporte. Una función de núcleo permite representar fronteras no lineales sin construir explícitamente todas las transformaciones de las variables. En detección de tráfico HTTP, las peticiones que quedan fuera de la región aprendida se consideran anómalas; esa condición debe contrastarse con las etiquetas para medir cuántas corresponden realmente a ataques (Schölkopf et al., 2001).

Con núcleo radial RBF, gamma controla la escala de la semejanza: valores mayores hacen que la influencia de cada observación sea más local. El parámetro nu regula el compromiso del ajuste; constituye un límite superior de la fracción de errores de entrenamiento y un límite inferior de la fracción de vectores de soporte. No representa directamente el porcentaje de ataques que se detectará en prueba. Por ello, modificar nu y gamma puede cambiar tanto la cobertura de ataques como la aceptación de tráfico legítimo (Scikit-learn developers, 2026e).

Las escalas de entrada también intervienen: una longitud con valores muy grandes puede dominar las distancias frente a una proporción entre cero y uno. El escalado debe estimarse con entrenamiento y aplicarse después sin reajustarlo con prueba. Además, aprender la función y elegir su umbral son operaciones distintas. La puntuación de OCSVM ordena las observaciones respecto de su frontera; no es una probabilidad de ataque. Si se modifica el punto de corte con etiquetas, debe registrarse qué partición se utilizó. Estas consideraciones permiten interpretar una recalibración sin atribuir al algoritmo todos los efectos del preprocesamiento o de la regla de decisión (Scikit-learn developers, 2026e, 2026f).

2.9.2 Isolation Forest

Isolation Forest construye árboles mediante elecciones aleatorias de una característica y de un punto de división dentro de su intervalo observado. Una petición que se separa del resto con pocas divisiones tiene un recorrido corto. Al reunir varios árboles, la longitud media de esos recorridos permite asignar una puntuación de anomalía. El método aprovecha que ciertas observaciones poco frecuentes resultan más fáciles de aislar. Sus árboles no aprenden divisiones usando etiquetas NORMAL/ATAQUE, por lo que su función difiere de la del bosque supervisado descrito en la sección 2.6.2 (Liu et al., 2008).

El algoritmo puede ajustarse sin etiquetas y también examinar nuevas observaciones después de entrenarse sólo con tráfico normal. Ese segundo uso corresponde a la exploración recogida en el informe metodológico del TFG. Un comportamiento legítimo infrecuente puede resultar fácil de aislar, mientras que un ataque parecido al tráfico de referencia puede recibir una puntuación poco anómala.

En la implementación de scikit-learn, contamination interviene en el umbral de las puntuaciones: un valor numérico expresa una proporción esperada de observaciones atípicas en entrenamiento, y la opción auto utiliza la regla propia de la implementación. No proporciona al modelo ejemplos etiquetados de ataque ni mide la prevalencia real en un conjunto futuro. Distinguir la puntuación del umbral evita confundir una mayor cantidad de alertas con una mejor separación entre clases (Scikit-learn developers, 2026g).

2.9.3 Regresión logística

La regresión logística aprende con ejemplos de ambas clases. Calcula una suma ponderada de las características y un término independiente; después aplica la función sigmoide para obtener un valor entre cero y uno. Un umbral transforma ese valor en una decisión. Aunque la sigmoide es no lineal, la frontera de decisión es lineal en las variables que recibe el modelo cuando se usa un umbral fijo. Su formulación permite estudiar si una combinación aditiva de las características disponibles separa suficientemente normales y ataques (Bishop, 2006, sección 4.3.2).

Cada variable aporta según su coeficiente. Si una señal sólo adquiere valor al combinarse con otra, esa interacción necesita estar representada en la entrada, por ejemplo mediante un producto de variables. Por tanto, una diferencia frente a un bosque puede orientar nuevas hipótesis sobre las interacciones, pero no demuestra por sí sola que toda separación lineal sea inadecuada.

La regularización limita la magnitud de los coeficientes para controlar el ajuste. En scikit-learn, C expresa la inversa de su intensidad: un valor menor impone mayor regularización. El tipo de penalización, el escalado y la convergencia del optimizador forman parte de la configuración que debe conservarse. Estos controles condicionan la interpretación de una comparación (Scikit-learn developers, 2026h).

Capítulo 3

Estado del arte y trabajos relacionados

3.1 Criterio de comparación

Los trabajos relacionados se comparan mediante cinco criterios: los datos que utilizan, el tipo de ejemplo analizado, la salida que predicen, las variables que reciben y la forma en que separan los datos para evaluar. La unidad de comparación es el **flujo experimental completo**: preparación de los datos, representación, modelo, selección de configuración, umbrales y protocolo de evaluación. Cuando dos trabajos informan la misma métrica, se comparan los valores obtenidos por sus flujos completos. Si también cambian los datos, las variables, las clases o la partición, esa comparación no identifica qué componente explica por sí solo la diferencia. La Tabla 3.2 resume los trabajos anteriores relacionados con las decisiones de esta tesis.

3.2 Estrategia de búsqueda bibliográfica

La revisión es narrativa y se orienta a fundamentar las decisiones del trabajo. Se realizaron búsquedas dirigidas en IEEE Xplore, ACM Digital Library, ScienceDirect y arXiv mediante las consultas de la Tabla 3.1. La búsqueda se complementó con las referencias de los trabajos directamente pertinentes.

Tabla 3.1: Consultas empleadas en la búsqueda bibliográfica.

Dominio restringido	**Consulta literal**
`ieeexplore.ieee.org`	`("HTTP request" OR "web attack") anomaly detection machine learning`
`dl.acm.org`	`HTTP request anomaly detection web attacks machine learning`
`sciencedirect.com`	`HTTP request anomaly detection CAPEC multi-label machine learning`
`arxiv.org`	`HTTP request anomaly detection web attack classification`

Se incluyeron estudios cuyo ejemplo de entrada fuera una petición, una URL o una carga útil HTTP y que abordaran detección de anomalías, clasificación de ataques, representación del contenido, incertidumbre o salida CAPEC. También se incluyeron las fuentes primarias de los modelos y conjuntos de datos utilizados. Se excluyeron estudios generales de anomalías de red o de Internet de las cosas que no evaluaran contenido HTTP a nivel de petición, documentos sin fuente rastreable y duplicados cuando se dispuso de una versión publicada. Con estos criterios se conformó el conjunto de antecedentes utilizado para contrastar las decisiones de representación, modelado y evaluación.

Alcance del capítulo. Las diferencias de datos, variables, salidas y particiones de los antecedentes se resumen en la Tabla 3.2. Las líneas base internas y la evolución de los experimentos se describen en los capítulos de configuración y resultados.

Tabla 3.2: Comparación crítica de los principales trabajos relacionados.

Trabajo	**Datos y tarea**	**Representación o enfoque**	**Relación con este trabajo e interpretación del contraste**
Kruegel y Vigna (2003)	Tráfico web; detección de anomalías	Longitudes, distribuciones de caracteres, estructura y modelos de normalidad	Incluye longitudes, distribución de caracteres y estructura de parámetros; esas familias de variables forman parte de la representación de esta tesis. No identifica familias de ataque ni usa los corpus evaluados aquí.
Wang y Stolfo (2004)	Cargas de red; anomalías	Modelado del contenido de la carga útil	Muestra que el contenido aporta información útil, pero analiza datos diferentes de una petición HTTP completa y aplica otro procedimiento de evaluación.
Ingham e Inoue (2007)	Tráfico HTTP; comparación de detectores	Técnicas de anomalías específicas para HTTP	Compara detectores que emplean representaciones diferentes; no estudia una salida multietiqueta.
Epp, Funk y Cappo (2018)	Peticiones HTTP; *one-class*	Características HTTP y máquina de vectores de soporte de una clase	Antecedente de detección HTTP mediante características específicas. Emplea un algoritmo, variables y particiones diferentes.
Riverol et al. (2024)	Ataques web; selección de variables	Conocimiento experto y características interpretables	Motiva agrupar variables por mecanismo. Una importancia alta indica asociación con la predicción, pero también puede reflejar cómo se creó el conjunto de datos.
Gniewkowski et al. (2023)	HTTP y URL maliciosas; anomalías	Representación aprendida Sec2vec	Ofrece una alternativa a las variables manuales. Sus resultados corresponden a otros datos y a otra división.
Nguyen et al. (2023)	Peticiones HTTP anómalas; clasificación	Procesamiento de lenguaje natural y aprendizaje profundo	Ejemplo del uso de BERT, arquitectura que aprende representaciones bidireccionales del texto. Se contrasta con este flujo como familia de representación, sin atribuir la diferencia sólo al modelo.
Luo et al. (2026)	CSIC 2010 y SR-BH 2020; detección binaria	WADBERT, con canales para URL y carga útil y atención sobre parámetros	Referencia externa de representación semántica y localización de parámetros. Las métricas describen el rendimiento de su flujo, con preparación y protocolo propios.

Trabajo	Datos y tarea	Representación o enfoque	Relación con este trabajo e interpretación del contraste
Li et al. (2023)	Ataques no vistos durante el entrenamiento y dominios distintos	Meta-aprendizaje y representación multidominio (RETSINA)	El meta-aprendizaje busca aprender cómo adaptar el modelo a nuevas tareas o dominios. El trabajo estudia si el modelo funciona en un dominio distinto del usado para entrenarlo. Aquí no se entrenó en un conjunto A para probar directamente en un conjunto B.
Zhou et al. (2026)	SR-BH y otro corpus web; detección de ataques	Ensamble de aprendizaje profundo por núcleos con estimación de incertidumbre	Muestra que una predicción puede acompañarse de incertidumbre del modelo y de los datos. Esa capacidad no forma parte de los flujos aquí evaluados.
Sureda Riera et al. (2022a)	SR-BH 2020; multietiqueta CAPEC	Codificación numérica y modelos de árboles LightGBM/CatBoost de una y dos fases	Proporciona el corpus usado en L1 y L2. Se comparan las métricas comunes, aunque difieren la preparación, la definición de NORMAL y la partición.
Osama et al. (2026)	DS-Augmented-v2; multiclase	Refinamiento y aumento de datos, variables manuales y representaciones textuales; varios modelos	
Yang et al. (2025)	CSIC y otros corpus web; detección web	Autoencoder y red neuronal multicapa	Se utiliza como referencia descriptiva de una representación aprendida; no emplea la misma preparación ni la misma arquitectura.

3.3 Detección de anomalías en tráfico web

Kruegel y Vigna (2003) demostraron que la longitud, la distribución de caracteres y la estructura de parámetros permiten asignar puntuaciones de anomalía a peticiones web. Wang y Stolfo (2004) modelaron cargas útiles y mostraron que el contenido contiene patrones que pueden aprenderse. Ingham e Inoue (2007) compararon varias técnicas para HTTP y mostraron que los resultados cambian según qué se considere normal y qué variables reciba el modelo.

Epp, Funk y Cappo aplicaron esta idea a un componente similar a un WAF. Utilizaron características específicas de HTTP y una máquina de vectores de soporte entrenada con una sola clase (Epp et al., 2018). Estos trabajos justifican entrenar con tráfico normal cuando no se dispone de ejemplos suficientes de todos los ataques. También muestran el límite principal: una puntuación que indica que una petición es poco habitual no identifica por sí sola el patrón de ataque.

3.4 Representaciones aprendidas y evaluación en otros dominios

Algunos trabajos recientes permiten que el propio modelo aprenda cómo convertir el texto en números, en vez de definir manualmente todas las variables. Sec2vec aprende una codificación para tráfico HTTP y URL maliciosas (Gniewkowski et al., 2023); Nguyen et al. (2023) aplican aprendizaje profundo y procesamiento de lenguaje natural a la detección de peticiones HTTP anómalas; y el método LLM-AE-MP, que combina un modelo de lenguaje de gran tamaño, un autoencoder y un perceptrón multicapa,

obtiene una versión numérica comprimida de la entrada y luego aplica un clasificador sobre varios conjuntos de datos (Yang et al., 2025). Estos enfoques pueden aprender patrones que una lista manual de variables no incluye. Sin embargo, resulta más difícil relacionar cada número interno del modelo con una propiedad concreta de HTTP. También pueden requerir otros recursos, datos y procedimientos de evaluación.

WADBERT separa el procesamiento de URL y carga útil y agrega representaciones a nivel de parámetro, con el propósito adicional de localizar qué parámetro contribuye a la detección (Luo et al., 2026). La propuesta de Zhou et al. incorpora incertidumbre asociada con los parámetros del modelo y con la distribución de los datos antes de combinar varios predictores (Zhou et al., 2026). Ambos trabajos delimitan dos capacidades que este estudio no evalúa: representación semántica aprendida del texto e incertidumbre predictiva explícita. Las probabilidades de XGBoost y las importancias por permutación aquí informadas no sustituyen ninguna de esas capacidades.

RETSINA estudia otro problema: entrena con datos web de ciertos dominios y evalúa el modelo en dominios diferentes, incluso con pocos datos disponibles (Li et al., 2023). Esto permite saber si el modelo conserva su rendimiento fuera del origen usado para entrenarlo. En este trabajo, la evaluación binaria multicorpus y las corridas multiclase entrenan un modelo nuevo dentro de cada conjunto de datos. Por tanto, no evalúan el caso «entrenar en A y probar en B». Estos trabajos sólo se comparan en cuanto a sus objetivos y métodos. Sus resultados sirven como referencia de un problema de generalización distinto del evaluado aquí.

3.5 Clasificación supervisada e interpretabilidad

La clasificación supervisada requiere etiquetas de ataque y puede producir una salida más específica. Random Forest y los métodos de *gradient boosting* combinan árboles de decisión. De esta forma pueden aprender casos en los que el efecto de una variable depende de otras, como longitudes, conteos, entropías y patrones de codificación (Breiman, 2001; Chen y Guestrin, 2016; Friedman, 2001). Riverol et al. (2024) estudian el aporte del conocimiento experto a la selección de características para ataques web. En este trabajo, una característica es interpretable cuando puede relacionarse con una propiedad concreta de la petición, como su longitud o la cantidad de caracteres especiales. Sin embargo, encontrar un fragmento de texto sospechoso o una característica con importancia alta no demuestra por sí solo la causa de la predicción.

3.6 Clasificación multietiqueta de ataques web mediante CAPEC

Sureda Riera et al. (2022a) publicaron SR-BH 2020, obtenido durante doce días de exposición de un *honeypot* WordPress, es decir, una instalación preparada para atraer y registrar actividad maliciosa. Cada petición puede conservar varias etiquetas CAPEC. Los autores evalúan LightGBM y CatBoost en procesos de una o dos etapas y miden si el modelo recupera simultáneamente cada CAPEC atribuido a

una petición. Una salida binaria no conserva esas etiquetas y una salida multiclase sólo conserva una combinación cuando ésta fue definida previamente como una clase compuesta.

Para este trabajo, el antecedente cumple dos funciones: aporta el corpus multietiqueta y fundamenta la necesidad de evaluar si un modelo conserva varios patrones CAPEC atribuidos a una misma petición. La propuesta mantiene esa salida, pero emplea características explícitas y XGBoost uno contra el resto. No se presenta como una reproducción del artículo original porque cambian la representación, las particiones y las peticiones evaluadas. La construcción detallada del protocolo multietiqueta agrupado se describe en los Capítulos 5 y 6.

3.7 Clasificación multiclase: WAMM y DS-Augmented-v2

Osama et al. denominan **WAMM** a su marco para distinguir mediante clasificación multiclase tipos de contenido malicioso como SQLi, XSS, SSTI y SSRF. A diferencia de una salida binaria, asigna una familia a cada carga o registro y permite analizar qué tipos de ataque se confunden o presentan menor rendimiento por clase. Para construir este escenario, depura y amplía datos derivados de SR-BH, reasigna las etiquetas en categorías y evalúa varios modelos (Osama et al., 2026). En este documento se conserva WAMM como el nombre propio del marco.

Los autores relacionan sus categorías con la cobertura de OWASP para una tecnología específica. Esto no significa que cada etiqueta corresponda directamente a una de las diez categorías de riesgo de OWASP. Clases como SQLi, XSS o SSTI describen tipos de contenido malicioso, mientras que OWASP Top 10 agrupa riesgos más amplios (OWASP Top 10 Team, 2021).

Para XGBoost, WAMM informa 99,59 % de exactitud y 86,07 % de $F_{1,\text{macro}}$ en DS-Augmented-v2.

Sureda Riera et al. y WAMM representan niveles diferentes de información para el análisis de seguridad. WAMM asigna una clase predefinida a cada carga, mientras que Sureda Riera et al. permiten conservar decisiones separadas para varios CAPEC de una misma petición (Osama et al., 2026; Sureda Riera et al., 2022a). En este trabajo ambas tareas se estudian por separado porque responden preguntas distintas.

3.8 Problemas abordados

La revisión muestra una diferencia práctica entre las formulaciones: los enfoques binarios y *one-class* detectan sospecha sin tipificarla; la clasificación multiclase devuelve una categoría predefinida, individual o compuesta; y la clasificación multietiqueta decide cada CAPEC por separado y puede activar varios, aunque requiere anotaciones y métricas específicas. Además, los trabajos anteriores suelen estudiar una tarea o un conjunto de datos por vez. De estas diferencias surgen los problemas metodológicos abordados en los capítulos siguientes: adaptar la salida al etiquetado disponible, tratar el desbalance y las repeticiones, y evaluar cada tarea con métricas apropiadas. Este trabajo estudia tres formulaciones:

1. clasificación binaria supervisada entre normal y ataque;
2. clasificación multiclase, con una categoría por petición; y
3. clasificación multietiqueta, con varios CAPEC posibles por petición.

En las tres se documentan las características y las métricas. El costo local se conserva con mayor detalle para las evaluaciones finales multiclase y multietiqueta agrupada. Para Harvard también se analiza si una misma huella de quince campos aparece en más de una partición. Para cada corrida se identifica la tarea, el corpus, el protocolo y su carácter exploratorio o final. En Harvard también se comprueba si una misma huella de quince campos aparece en más de una partición.

Capítulo 4

Conjuntos de datos y control de calidad

4.1 Panorama, procedencia y función de los corpus

Los conjuntos de datos cumplen funciones diferentes. No se supone que todos representen el mismo tipo de tráfico HTTP. Para cada corpus se indica su origen, qué representa cada registro, qué etiquetas contiene, cuántos registros se usaron y para qué experimento se empleó. La Tabla 4.1 resume estas diferencias. Las secciones siguientes describen primero los datos y luego las decisiones metodológicas aplicadas.

Tabla 4.1: Conjuntos de datos empleados y función experimental.

Corpus	**Origen**	**Etiquetado utilizado**	**Función en el estudio**
CSIC 2010 (Information Security Institute, s. f.)	Peticiones de comercio electrónico generadas en un entorno controlado	Normal o anómalo	Clasificación binaria y ablación.
TorpEda 2012 (Torrano-Giménez et al., 2012)	Colección de peticiones normales y ataques web categorizados	Diez clases operativas: NORMAL, ANOMALOUS y ocho familias	Evaluación multiclase con fuerte desbalance.
SR-BH 2020 (Sureda Riera et al., 2022a,b)	Servidor señuelo WordPress expuesto durante doce días	Normal y etiquetas CAPEC múltiples	Salidas binaria, multiclase y multietiqueta.
DS-Augmented-v2 (Osama et al., 2026)	Corpus refinado y aumentado por WAMM	NORMAL y ocho familias de cargas de ataque	Extensión multiclase sobre categorías adicionales.

Tabla 4.2: Procedencia e información de integridad disponible para los conjuntos de datos.

Corpus	Fuente e identificador	Versión utilizada	Información de integridad disponible
CSIC 2010	DOI y sitio del corpus citados en (Information Security Institute, s. f.)	Distribución CSIC 2010 empleada en B1 y B2	Fuente, nombres de los tres archivos y poblaciones procesadas documentados.
TorpEda 2012	Publicación original del corpus (Torrano-Giménez et al., 2012)	Ocho archivos XML de la especificación 2012	
SR-BH 2020	Harvard Dataverse; DOI (Sureda Riera et al., 2022b)	Versión 1.2	SHA-256 del CSV utilizado y ficha pública con licencia CC0 1.0, dedicación al dominio público.
DS-Augmented-v2	Conjunto descrito por WAMM (Osama et al., 2026)	Archivo `combined_data.csv` utilizado por M7–M8	

Tabla 4.3: Perfil de calidad y transformaciones documentadas por conjunto de datos.

Corpus	Población procesada	Preparación y controles aplicados
CSIC 2010	Peticiones normales y anómalas del corpus empleadas en B1 y B2.	División estratificada 80/20 con semilla 42 para clasificación binaria B2; los procedimientos se documentan en los Capítulos 5 y 6.
TorpEda 2012	74.133 registros: 59.306 de desarrollo y 14.827 de prueba.	Se conservan las diez clases nativas y se aplica una división estratificada 80/20.
SR-BH 2020	907.815 filas originales; 907.813 participan en el análisis de huellas; 542.990 grupos consolidados en L2.	Se cuantifican repeticiones bajo una regla de quince campos, se documentan las reducciones de etiquetas y L2 asigna cada grupo completo a una sola partición.
DS-Augmented-v2	704.665 registros: 563.732 de desarrollo y 140.933 de prueba.	Se evalúa una taxonomía de ocho clases que fusiona las dos variantes de inyección de comandos.

DS-Augmented-v2 se incorpora como extensión multiclase para examinar SSTI, SSRF y dos familias de inyección de comandos. Cada corpus conserva su propia población, su taxonomía y un modelo entrenado de forma independiente.

4.2 CSIC 2010

CSIC 2010 registra peticiones HTTP dirigidas a una aplicación de comercio electrónico simulada. La distribución incluye tráfico normal destinado al entrenamiento y tráfico de prueba con peticiones normales y anómalas. La unidad es una petición HTTP y la etiqueta disponible distingue normalidad de anomalía; el corpus no ofrece una taxonomía multietiqueta equivalente a CAPEC (Information Security Institute, s. f.).

En este documento se conserva «anómalo», que es la nomenclatura nativa de CSIC para la clase que no corresponde a tráfico normal; el término no introduce una tercera categoría.

En este trabajo se emplea como fuente controlada para la clasificación binaria supervisada, la ablación de grupos de características. Estas funciones se detallan en los Capítulos 5 y 7. Por su origen sintético y controlado, CSIC ofrece un escenario estable para comparar configuraciones; la evaluación con tráfico real de producción se mantiene como una etapa posterior.

4.3 TorpEda 2012

TorpEda 2012 es una especificación y colección de peticiones normales, anomalías y ataques web categorizados. Incluye inyección SQL (SQLi), secuencias de comandos entre sitios (XSS), inyección XPath, inyección de separadores CRLF (CRLFi), inclusiones del lado del servidor (SSI), inyección LDAP (LDAPi), ataques por cadena de formato (*Format String*) y desbordamiento de búfer (*Buffer Overflow*), además de NORMAL y una categoría anómala general (Torrano-Giménez et al., 2012). La unidad de observación vuelve a ser una petición, pero, a diferencia de CSIC, el nombre de la familia permite construir una salida multiclase.

La población procesada contiene 74.133 registros. Para las corridas multiclase, este estudio aplicó una división estratificada 80/20 con semilla 42: 59.306 registros de desarrollo y 14.827 de prueba. La cantidad de ejemplos por clase es muy desigual; SQLi domina las clases de ataque y algunas categorías tienen menos de cien ejemplos de prueba. Por esa razón, una métrica perfecta sobre ocho casos no posee la misma estabilidad que otra calculada sobre miles. Se entrena un modelo propio con TorpEda. No se entrena en TorpEda para luego probar el mismo modelo en otro corpus.

4.4 SR-BH 2020

4.4.1 Origen y estructura

SR-BH 2020 registra peticiones capturadas durante doce días por un *honeypot* WordPress expuesto a Internet. Cada fila representa un evento de captura. El archivo `data_capec_multilabel.csv` contiene 38 columnas que reúnen campos de petición, metadatos de conexión, datos de respuesta y etiquetas CAPEC. Una fila maliciosa puede activar varios CAPEC; esto constituye una manifestación empírica del caso general descrito en la Sección 2.3 y permite estudiar una salida multietiqueta además de reducciones binaria y multiclase (Sureda Riera et al., 2022a,b). En algunos informes históricos se lo denomina «Harvard», por el repositorio Harvard Dataverse del que se obtuvo; ambas formas designan el mismo corpus.

4.4.2 Conteos publicados y observados

La documentación informa 907.814 peticiones: 525.195 normales y 382.619 ataques. El conteo del conjunto completo produjo 907.815 filas: las mismas 525.195 normales y 382.620 con al menos una etiqueta de ataque. Existe una diferencia publicada de una fila. Los experimentos conservan los conteos observados y no modifican una etiqueta para forzar la coincidencia con el total publicado.

La línea base multietiqueta por filas conserva las 907.815 filas; excluye CAPEC-248 sólo como variable objetivo por tener un positivo, sin retirar su registro. La reducción multiclase top-14 también parte de 907.815 filas y excluye 5.404 filas correspondientes a 46 combinaciones exactas que quedan fuera de las clases admitidas, conservando 902.411.

Para el análisis de contenido repetido se retiran dos eventos vacíos y se estudian 907.813 eventos válidos. Estos forman 542.990 grupos según la huella definida en la Sección 4.7. La evaluación multietiqueta agrupada usa una población de 542.990 registros consolidados, uno por grupo. Por tanto, sus 108.598 casos de prueba son grupos consolidados y no peticiones originales independientes. Esta unidad de análisis no debe confundirse con las filas usadas por la línea base o por las corridas multiclase.

4.5 DS-Augmented-v2

DS-Augmented-v2 es la versión refinada y aumentada que WAMM deriva de fuentes de cargas web. Contiene NORMAL y ocho familias: inyección de código (*Code Injection*), inyección de comandos (*Command Injection*), inyección de comandos del sistema operativo (*OS Command Injection*), recorrido de directorios (*Path Traversal*), SQLi, SSRF, SSTI y XSS (Osama et al., 2026). Los autores describen estas categorías como alineadas con la cobertura de OWASP para su escenario; no constituyen una correspondencia directa con las diez categorías de riesgo de OWASP, distinción desarrollada en el Capítulo 3.

La versión procesada contiene 704.665 registros. Para las corridas multiclase, este estudio creó una división estratificada 80/20 con semilla 42: 563.732 registros de desarrollo y 140.933 de prueba. Esos conteos describen la división ejecutada y no una partición oficial del proveedor. El corpus añade SSRF y SSTI al conjunto de familias evaluadas. Las métricas publicadas por WAMM se comparan como resultados del flujo de referencia en el Capítulo 3.

4.6 Formato común de las peticiones

Los archivos de origen no usan las mismas columnas ni el mismo formato. Para poder procesarlos se define un **esquema común**, es decir, una estructura intermedia con los mismos campos para todos los corpus. Cada cargador reconstruye, hasta donde su fuente lo permite, los cinco componentes de la petición definidos en la Ecuación (2.1). Si un componente no existe, se deja vacío. No se deduce a partir de la respuesta del servidor. La Tabla 4.4 resume esta conversión.

Este proceso distingue cuatro elementos. Primero están los **campos disponibles de la petición HTTP de origen**, por ejemplo, una línea `GET /index.php?id=1 HTTP/1.1` con los encabezados y el cuerpo que aporte la fuente. Segundo está la **representación común** $r = (m, u, v, h, b)$: guarda siempre en ese orden el método, la URI solicitada, la versión HTTP, los encabezados y el cuerpo. Esta representación no codifica la petición ni calcula su huella; sólo organiza los campos disponibles y deja vacío un componente ausente. Tercero está el **vector de características**, que convierte esos campos en números como la longitud de la URI o la cantidad de parámetros; esos números son la entrada del modelo. Cuarto está la **huella de petición**, usada en el análisis de Harvard y en la evaluación multietiqueta agrupada. Antes de calcularla se aplican reglas limitadas de formato a los quince campos

seleccionados. La huella permite agrupar y dividir los datos; no es una característica de entrada del modelo.

Tabla 4.4: Mapeo de las fuentes al esquema común de petición.

Corpus	Forma de origen	Resultado en el esquema común
CSIC 2010	Mensajes HTTP normales y anómalos	Se reconstruyen método, URI, versión, encabezados y cuerpo disponibles; la salida es binaria.
TorpEda 2012	Colecciones de peticiones por familia	Se reconstruye la petición y se conserva el nombre de familia como clase.
SR-BH 2020	CSV estructurado de 38 columnas	Se mapean columnas explícitas de método, URI, protocolo, once encabezados y cuerpo; los CAPEC forman la salida multietiqueta.
DS-Augmented-v2	Registros de cargas y clase	El cargador propio reconstruye los componentes presentes y deja vacíos los no provistos; la clase nativa se conserva.

La marca de tiempo, las direcciones, los puertos y la respuesta permanecen, cuando existen, como metadatos del evento. No ingresan al vector predictivo porque no forman parte de la petición que un detector recibiría antes de decidir. El uso de un esquema común no significa que los corpus se hayan recolectado o etiquetado de la misma manera. Se comparte el extractor cuando es posible, pero se entrena un modelo independiente para cada corpus.

4.7 Peticiones repetidas en SR-BH 2020

El análisis de peticiones repetidas se limita a SR-BH 2020. No se realizó una comparación equivalente sobre CSIC, TorpEda o DS-Augmented-v2; por ello, sus conteos no se trasladan a esos corpus. En Harvard se distinguen dos conceptos:

Criterio de repetición.

- dos **registros completos son iguales** cuando coinciden los 38 campos del evento, incluidos los metadatos y la respuesta;
- dos registros tienen **contenido HTTP repetido según la regla del estudio** cuando producen la misma huella calculada sobre los quince campos seleccionados después de las reglas limitadas de formato, aunque cambien el instante, el puerto, la IP o la respuesta.

Una división por filas evita que un mismo registro aparezca en dos particiones, pero puede colocar en desarrollo y prueba eventos distintos con la misma huella. En ese caso, la prueba incluye contenido HTTP que ya estuvo representado durante el ajuste. Para evaluar el desempeño sobre huellas no observadas en desarrollo, todas las filas que comparten una huella se asignan a una única partición.

Para cuantificar ambos niveles en el conjunto completo de SR-BH 2020 se usaron:

- **registro completo igual:** coincidencia en las 38 columnas de SR-BH 2020;
- **contenido HTTP repetido según la regla del estudio:** coincidencia en método, URI, protocolo, once encabezados y cuerpo después de las reglas limitadas de formato.

El segundo criterio abarca *User-Agent*, *Referer*, *Host*, *Origin*, *Cookie*, *Content-Type*, *Accept*, *Accept-Language*, *Accept-Encoding*, DNT (*Do Not Track*) y *Connection*. Se excluyen la marca de tiempo, la IP, los puertos y la respuesta porque describen el evento o su resultado, no el contenido disponible al detector antes de responder. Para comparar siempre de la misma forma, el procedimiento convierte el método a mayúsculas, usa una escritura estable para la versión, pasa los nombres de encabezados a minúsculas, recorta espacios y tabulaciones al inicio y al final de sus valores y coloca los quince campos en un orden fijo. Después, BLAKE2b de 128 bits genera un código de 32 caracteres hexadecimales. Este código es la **huella de petición**. Una huella igual sólo expresa coincidencia bajo estas reglas; no demuestra igualdad byte a byte ni equivalencia semántica para cualquier aplicación.

La Tabla 4.5 presenta los conteos obtenidos al agrupar los quince valores normalizados mediante la huella definida. El programa utiliza esa misma huella para comprobar las intersecciones entre las particiones.

Tabla 4.5: Repeticiones observadas en el conjunto completo de SR-BH 2020.

Criterio de comparación	Grupos repetidos	Filas adicionales	Filas en grupos repetidos	Máximo de filas/grupo
38 columnas	1.415	2.746	4.161	10
15 campos HTTP tras reglas de formato	27.624	364.824	392.448	453
15 campos tras reglas, sin dos vacías	27.623	364.823	392.446	453

Las 364.823 apariciones adicionales equivalen al 40,187021 % de las 907.813 filas consideradas válidas. Sólo 2.746 serían filas completas adicionales en las 907.815 filas originales; después de retirar las dos vacías, ese conteo queda en 2.745. En la mayoría de esos grupos cambia algún metadato. Entre los 27.624 grupos formados mediante la regla de quince campos se reportan cambios de marca de tiempo en 26.594, de puerto de origen en 25.880 y de longitud de respuesta en 9.498.

El tratamiento depende del protocolo. Las corridas multiclase históricas y la línea base multietiqueta dividen por fila y no eliminan repeticiones; las 5.404 exclusiones corresponden al filtro top-14 previo. Se reconstruye el solapamiento de cuatro modos multiclase y de la línea base; la ejecución binaria sobre Harvard no se incluye en ese conteo. La evaluación agrupada conserva un registro consolidado por huella y une las etiquetas mediante la operación lógica OR: si una etiqueta aparece en al menos una fila, se conserva para todo el grupo. Existen 288 grupos con etiquetas variables y 284 que mezclan al menos un evento normal y otro de ataque. Los 288 grupos abarcan 2.331 filas; 2.322 pertenecen a los 284 grupos que mezclan normal y ataque. Los cuatro grupos restantes varían sólo entre etiquetas de ataque. La unión lógica puede asignar al registro consolidado una combinación que no estaba presente completa en ninguna fila individual. Por tanto, cambia la proporción de cada etiqueta, la unidad de

análisis y el objetivo del experimento. Se presenta como una decisión metodológica, no como una corrección del corpus.

4.8 Procedencia y calidad de las etiquetas

Las etiquetas de referencia son las suministradas por cada corpus. En CSIC se conserva la distinción normal/anómalo; en TorpEda, las familias nativas; en SR-BH, las columnas CAPEC publicadas, a partir de las cuales se construyen las salidas binaria y multiclase; y en DS-Augmented-v2, los nombres originales salvo las agrupaciones documentadas.

En SR-BH, las etiquetas se originaron a partir de ModSecurity 2.9.2 y OWASP Core Rule Set 3.3.0 en modo de detección, seguidos por el procesamiento manual y semiautomático descrito por sus autores (Sureda Riera et al., 2022a,b). Para los 288 grupos con etiquetas variables, el protocolo L2 aplica una unión lógica OR y conserva todas las CAPEC observadas dentro del grupo. Esta regla define la salida del registro consolidado sin modificar las anotaciones originales de sus filas. La ausencia de una adjudicación manual adicional para esos grupos se registra como limitación.

Capítulo 5

Metodología de investigación

5.1 Diseño general y elección de los modelos

La investigación es cuantitativa y experimental, con un análisis comparativo de alcance descriptivo. El flujo convierte cada petición HTTP en una fila de una matriz tabular, entrena modelos para distintos tipos de salida y evalúa la calidad de las predicciones, el desempeño por categoría y el costo de ejecución.

El diseño analiza la información aportada por las características, las diferencias entre formulaciones, las categorías con más errores y los límites que aparecen al separar peticiones repetidas y revisar el uso previo de la prueba. Se distinguen tres líneas principales: binaria supervisada, multiclase y multietiqueta. Una línea reúne las corridas de una misma tarea, aunque cambien el corpus, la representación o la configuración.

La rama binaria comenzó con detección de anomalías. One-Class SVM e Isolation Forest aprendieron principalmente del tráfico normal. Después se evaluaron regresión logística y Random Forest con ejemplos etiquetados de ambas clases. El paso a supervisado cambió la información disponible durante el aprendizaje; por ello, no fue solamente una sustitución de algoritmo.

La secuencia se documenta en el informe metodológico de Román Medina y González Arévalos (2026), que reúne reportes internos y capturas de las pruebas. La recalibración de OCSVM produjo una mejora, pero aislar un endpoint, ampliar las características y probar Isolation Forest no resolvió las dificultades observadas. Con 57 características, Random Forest alcanzó F1 cercano a 0,947 frente a 0,665 de la regresión logística. El historial y los resultados se desarrollan en las secciones 7.4 y 8.2.

La representación combina longitudes, conteos, entropía, codificación y señales léxicas cuyo efecto puede depender de otras variables. Los árboles aceptan esta entrada tabular y representan interacciones mediante divisiones sucesivas. Random Forest se mantuvo como referencia binaria y base de las ablaciones para estudiar las características con un algoritmo estable (Breiman, 2001). La elección se apoya en su adecuación y en los resultados observados; no establece que sea el mejor clasificador en términos generales.

HistGradientBoosting se emplea en multiclase porque su discretización por histogramas reduce el costo de buscar divisiones en matrices grandes (Scikit-learn developers, 2026c). XGBoost se combina con la estrategia uno contra el resto para entrenar una decisión por etiqueta. Esta descomposición permite asignar pesos positivos y umbrales específicos según la frecuencia y el comportamiento de cada CAPEC (Chen y Guestrin, 2016; Tsoumakas y Katakis, 2007).

La unidad de comparación es el flujo completo: datos, representación, taxonomía de salida, modelo, ponderación, umbrales y protocolo de evaluación. Cuando cambian varios componentes, las métricas describen esa evolución y no el aporte aislado de cada uno. Comparar algoritmos requeriría mantener constantes los demás componentes, aplicar presupuestos de ajuste comparables y repetir los entrenamientos.

5.2 Control del diseño experimental

Los identificadores B1-B2, M1-M8 y L1-L2 ordenan las etapas principales de cada línea. Las pruebas anteriores a B1 se presentan como antecedentes de la elección del modelo y conservan sus protocolos. El grado de control depende de cómo se separan los datos y de si la prueba interviene en decisiones anteriores. Se distinguen tres situaciones:

1. Resultado exploratorio: corridas históricas empleadas para desarrollar variables, taxonomías o umbrales.

2. Resultado final de desarrollo: una configuración concluida cuyo conjunto de prueba ya había sido observado, directa o indirectamente, durante el desarrollo.

3. Diseño con prueba separada por la huella definida: cada grupo que comparte la huella calculada sobre quince campos se asigna a una sola partición antes del entrenamiento, la selección y el ajuste de umbrales.

Cada corrida se interpreta según su protocolo. Los capítulos 7 y 8 presentan los conteos y las intersecciones comprobadas. Ninguno de estos niveles sustituye una evaluación temporal ni una prueba entre corpus.

5.3 Qué analiza el modelo y qué formulaciones se estudian

El modelo analiza el contenido de una petición individual. No analiza una sesión completa ni usa una dirección IP como unidad. A partir de cada petición se estudian tres formulaciones principales:

Binaria supervisada. Produce una salida normal o ataque y se entrena con ejemplos de ambas clases. Los modelos one-class fueron una exploración previa que empleó otra forma de aprendizaje.

Multiclase. Asigna una clase por petición, individual o compuesta. Cuando las etiquetas originales se solapan, la taxonomía debe indicar qué combinaciones admite o agrupa.

Multietiqueta. Doce decisiones CAPEC binarias pueden activarse simultáneamente. NORMAL se deriva de la ausencia de etiquetas de ataque.

Las taxonomías multiclase convierten las etiquetas disponibles en una sola clase por petición. En M5, CAPEC-153 y CAPEC-194 se agrupan como Data Manipulation, mientras que CAPEC-272, CAPEC-274 y la combinación 272+274 forman HTTP Protocol Manipulation. En M6, esas categorías se reúnen en Request Manipulation. Después de agrupar, una confusión entre dos CAPEC del mismo grupo deja de contarse como error. El aumento de F1 macro entre top-14, top-11 y top-10 refleja principalmente la consolidación de categorías y el menor detalle de la salida; no significa que las CAPEC agrupadas hayan pasado a distinguirse por separado.

5.4 Estrategia de representación

La representación numérica de cada petición es uno de los factores del estudio. Los extractores parten de listas básicas de longitudes, estructura, codificación, entropía, símbolos y palabras relevantes. Las etapas posteriores amplían o reducen esas listas para incorporar partes antes no utilizadas, comprobar el aporte de grupos de variables y retirar aquellas que no mostraron aporte en una etapa anterior.

Una comparación se interpreta como ablación cuando sólo cambia el conjunto de características y permanecen fijos el corpus, la taxonomía, el modelo y la partición. Si cambia la taxonomía, las métricas corresponden a objetivos distintos: agrupar clases elimina de la evaluación los errores que antes ocurrían entre ellas.

Todas las características se calculan a partir de una petición individual. No se usan ventanas temporales, secuencias de peticiones, direcciones IP ni información de una sesión completa. El capítulo 6 explica la extracción; el capítulo 7 enumera las versiones ejecutadas, y el apéndice A contiene las listas multietiqueta.

5.5 Protocolos de partición

5.5.1 Protocolo histórico por filas

Las ejecuciones multiclase históricas y la configuración binaria B2 utilizan una división estratificada 80/20 con semilla 42. La estratificación mantiene aproximadamente la proporción de cada clase; la semilla permite repetir las decisiones aleatorias que la utilizan. En B2, el umbral que maximiza F1 se elige sobre el mismo 20 % empleado para informar esa métrica. B1 también se conserva como ablación exploratoria en el registro experimental. Esta condición se distingue de las pruebas one-class anteriores, que no comparten necesariamente la misma partición ni el mismo ajuste.

L1 divide por filas en entrenamiento, validación y prueba mediante una etiqueta primaria.

En Harvard, la división por filas impide repetir una fila exacta entre particiones, pero no impide que dos eventos diferentes produzcan la misma huella. L2 aplica el control adicional: asigna cada grupo de huella a una única partición y comprueba que las seis intersecciones entre entrenamiento, validación, ajuste de umbrales y prueba sean nulas. El alcance de la auditoría por huellas se documenta en la Sección 7.7.

5.5.2 Protocolo multietiqueta por grupos de huella

El protocolo agrupado aplica las siguientes operaciones:

1. descarta las entradas con las que no puede reconstruirse una petición válida;
2. calcula la huella definida en la Sección 4.7;
3. agrupa los registros de una misma huella;
4. conserva un registro consolidado por grupo y une las etiquetas mediante OR;
5. divide los registros consolidados en entrenamiento, validación, ajuste de umbrales y prueba;
6. verifica las seis intersecciones posibles entre las cuatro particiones.

Las divisiones sucesivas usan semillas 42, 43 y 44 y estratificación por una etiqueta primaria del grupo, con proporciones efectivas 64/8/8/20. El protocolo exige que las seis intersecciones entre pares de particiones sean nulas antes del entrenamiento. La consolidación cambia la unidad de análisis y puede modificar la proporción de normales y ataques; por eso, este protocolo no se compara directamente con las corridas por filas como si sólo hubiera cambiado el modelo. Los tamaños obtenidos se presentan en el Capítulo 7 y las métricas en el Capítulo 8.

5.6 Entrenamiento, selección y ajuste de umbrales

Los hiperparámetros son decisiones de configuración que el modelo no aprende por sí solo, por ejemplo, la cantidad de árboles, la cantidad de hojas o el mínimo de ejemplos permitido en una hoja. El entrenamiento aprende los parámetros del modelo; la validación selecciona los hiperparámetros; el ajuste de umbrales elige el punto de corte que convierte una puntuación en una clase; y la prueba mide la configuración ya fijada. Elegir un umbral no calibra la puntuación como probabilidad. Cuando el conjunto de prueba intervino en la elección del umbral o de la configuración, la corrida se informa como exploratoria.

En la línea base multietiqueta por filas, cada umbral se obtiene con validación y después se aplica a un estimador que fue reentrenado con entrenamiento más validación.

En validación, la evaluación multietiqueta agrupada puntúa cada ensayo binario de la etiqueta ℓ mediante

$$Q_\ell = \frac{1}{2}\left(\mathrm{AP}_{\ell,\mathrm{val}} + F^{\star}_{1,\ell,\mathrm{val}}\right), \tag{5.1}$$

donde $\mathrm{AP}_{\ell,\mathrm{val}}$ se calcula con `average_precision_score` y $F^{\star}_{1,\ell,\mathrm{val}}$ es el mejor F_1 alcanzable al recorrer umbrales sobre esa validación. La parada temprana de XGBoost usa su AUC-PR interna (`aucpr`); la selección final usa AP en el promedio de (5.1). Ambas resumen el ordenamiento precisión–sensibilidad, pero aplican reglas de integración diferentes y no deben recibir el mismo nombre (Scikit-learn developers, 2026a; XGBoost developers, 2026).

Sea C la partición separada para ajustar umbrales, sea $\mathbf{y}_{C,\ell} = (y_{i\ell})_{i\in C}$ el vector verdadero de la etiqueta ℓ y sea $\mathscr{T}_\ell$ el conjunto de umbrales candidatos. Para cada τ se forma

$$\widehat{\mathbf{y}}_{C,\ell}(\tau) = (\mathbb{I}[p_\ell(x_i) \geq \tau])_{i\in C}.$$

El punto inicial se obtiene mediante

$$\tau_\ell^{(0)} = \underset{\tau\in\mathscr{T}_\ell}{\arg\max}\, F_1\left(\mathbf{y}_{C,\ell}, \widehat{\mathbf{y}}_{C,\ell}(\tau)\right). \tag{5.2}$$

Después, un ajuste conjunto modifica un umbral por vez y vuelve a evaluar el resultado global; mediante este recorrido por coordenadas intenta mantener $F_{1,\ell} \geq 0{,}85$, cuando es factible, y sensibilidad de NORMAL de al menos 0,99, con un máximo de seis pasadas. Por ello, los umbrales finales no son necesariamente los máximos independientes de F_1. El código denomina «calibración» a esa partición, pero su función es elegir umbrales, no transformar las puntuaciones para que representen probabilidades calibradas. Las salidas se interpretan como puntuaciones producidas por el objetivo logístico, no como probabilidades cuya calibración haya sido comprobada. La prueba se consulta sólo después de fijar modelos y umbrales.

Reducir τ_ℓ hace que más casos se predigan como positivos y, en general, aumenta tanto la sensibilidad como los falsos positivos; aumentarlo produce el efecto contrario. Este ajuste modifica la regla de decisión. La separación entre entrenamiento y prueba depende de las particiones utilizadas para elegir el umbral, no del valor del umbral en sí.

Los espacios concretos de búsqueda y los hiperparámetros ejecutados pertenecen a la implementación y se documentan en la Sección 6.7.

5.7 Métricas utilizadas

La métrica principal en multiclase es $F_{1,\text{macro}}$ porque asigna el mismo peso a clases frecuentes y raras. También se informan exactitud, exactitud balanceada, F_1 ponderado, MCC, AP macro y resultados por clase. En multietiqueta se priorizan $F_{1,\text{micro}}$, $F_{1,\text{macro}}$, coincidencia exacta, pérdida de Hamming, AP macro y métricas por etiqueta. No se fija un único valor mínimo para todas las métricas. Se informa cada valor y se identifican las categorías con peores resultados.

Para la lectura binaria derivada, ataque es la clase positiva. FPR mide la proporción de peticiones normales marcadas como ataque. FNR mide la proporción de ataques marcados como normales. Ambas medidas son necesarias para analizar un posible uso tipo WAF. Las métricas globales se acompañan con resultados por clase para mostrar las categorías débiles.

5.8 Evaluación operativa

La latencia mide cuánto tarda el flujo desde que recibe una petición disponible en memoria hasta que termina la extracción, la selección de variables y la predicción. Esta medición no incluye red, TLS ni una aplicación protegida. Se reportan los percentiles p50, p95 y p99. El modo **individual** procesa una petición por vez e incluye extracción de características y predicción. El modo por **lotes** procesa varias peticiones juntas e incluye la construcción de la matriz y la predicción. La **inferencia** es únicamente la predicción del modelo sobre vectores que ya fueron construidos. La tasa de procesamiento indica cuántas peticiones se completan por unidad de tiempo en cada modo.

También se registran el tamaño del modelo guardado, la memoria máxima usada por el proceso y el tiempo total de la corrida. La memoria se informa como RSS (*Resident Set Size*), que representa la memoria física ocupada por el proceso. MiB y GiB son unidades binarias: un MiB equivale a 2^{20} bytes y un GiB a 2^{30} bytes. Estas medidas describen el consumo de tiempo y memoria en el entorno probado. No establecen la viabilidad en producción porque no incluyen usuarios concurrentes, colas, un motor de reglas ni una carga sostenida.

5.9 Análisis de importancia

La importancia por permutación mide cuánto empeora una métrica al desordenar los valores de una característica. Se calcula para el resultado global y, en las corridas finales, para cada clase mediante comparaciones de uno contra el resto. Identifica las variables cuya permutación reduce más la métrica y aquellas cuya alteración produce un cambio cercano a cero. No mide cuántas veces el modelo utilizó una variable ni demuestra que ésta cause un ataque. Su definición matemática se presenta en (2.49).

5.10 Reproducibilidad e incertidumbre

Una corrida reproducible requiere código, lanzador, semilla, taxonomía, lista ordenada de características, configuración, índices o huellas por partición, predicciones y métricas. El Capítulo 6 describe cómo se producen esos elementos; la Tabla 7.3 identifica la configuración asociada con cada etapa, y los resultados se acompañan con conteos por clase o etiqueta para que pueda evaluarse su soporte.

Las métricas reportadas son estimaciones puntuales de las ejecuciones conservadas, generalmente realizadas con semilla 42. La semilla permite repetir las operaciones aleatorias que la utilizan, pero no equivale a disponer de ejecuciones independientes. Como no se realizaron repeticiones con varias semillas, las comparaciones se interpretan de forma descriptiva y no se presentan como diferencias estadísticamente significativas. Las versiones del entorno se documentan porque el paralelismo y las bibliotecas de ejecución pueden introducir pequeñas variaciones entre ejecuciones.

5.11 Limitaciones que pueden afectar la validez

Las principales amenazas consideradas son:

- **validez interna, o posibilidad de atribuir el resultado al método:** en las corridas binarias históricas, el umbral se seleccionó sobre la prueba; en parte de la secuencia multiclase, las características o taxonomías se definieron después de observar resultados sobre la misma población externa; y las divisiones por filas de Harvard no controlan coincidencias por huella. L2 controla este último aspecto mediante grupos e intersecciones nulas;
- **validez de la representación, llamada también validez de constructo:** una salida multiclase no conserva decisiones separadas por etiqueta y sólo representa las combinaciones previstas en su taxonomía; la definición de huella determina qué peticiones se consideran iguales y las características expertas pueden acercar la representación a señales empleadas por el proceso de anotación;
- **validez externa, o posibilidad de extender la conclusión a otros datos:** no se probó con tráfico de producción, con una división temporal ni entrenando en un corpus y probando en otro;
- **validez estadística, o certeza sobre la variación del resultado:** gran parte del historial usa una sola semilla, algunas clases tienen pocos casos y no existen intervalos de confianza;
- **comparabilidad:** entre corridas cambian al mismo tiempo la cantidad de ejemplos, las características, las clases y el ajuste de umbrales.

En SR-BH, las etiquetas y algunas características expertas consideran señales léxicas relacionadas con ataques web. Por ello, el rendimiento se interpreta como concordancia con la referencia del corpus. Ninguna columna de etiqueta se utiliza como predictor. En DS-Augmented-v2 y TorpEda se mantiene la división por filas ejecutada. El control verificado en L2 consiste en las seis intersecciones nulas entre sus grupos.

5.12 Ética, privacidad y uso responsable

El estudio utiliza conjuntos de datos secundarios creados por terceros; no recluta personas ni intercepta tráfico nuevo. Esto reduce la intervención directa, pero no elimina las responsabilidades sobre privacidad, licencia y uso. SR-BH puede contener direcciones, identificadores técnicos, cookies, agentes de usuario, referencias y otros encabezados capaces de revelar o vincular actividad. Las direcciones IP, puertos, marcas de tiempo y respuestas no se usan como características predictivas; aun así, algunos encabezados y cuerpos pueden contener información sensible.

El principio aplicado es minimizar los datos utilizados: el modelo recibe medidas derivadas de los campos necesarios para la investigación, y las peticiones ilustrativas de la Figura 2.3 y de la Tabla 6.3 son sintéticas. Los archivos originales deben mantenerse con acceso restringido, evitarse en ejemplos públicos y eliminarse o anonimizarse cuando ya no sean necesarios. Cuando la licencia de un corpus no está identificada, debe citarse la fuente y suspenderse su redistribución hasta verificar las condiciones aplicables.

Los modelos y las listas de señales poseen un posible uso dual. Pueden ayudar a detectar ataques, pero también revelar qué propiedades observa un detector. Su finalidad en este trabajo es defensiva, educativa y experimental. Los resultados no deben utilizarse para atacar sistemas, eludir controles ni atribuir intenciones a una persona.

Un falso positivo podría bloquear una petición legítima y un falso negativo permitir un ataque. Debido a los errores de las clases minoritarias, los modelos se evalúan como apoyo a la detección y no como mecanismos autónomos de bloqueo. Una aplicación posterior debería comenzar en modo de observación, combinar las predicciones con reglas y contexto, registrar las decisiones, permitir revisión humana y reversión, limitar la retención de datos y vigilar cambios en el tráfico. Estas medidas reducen el riesgo, pero no convierten las pruebas realizadas en una validación de producción.

Capítulo 6

Implementación del flujo experimental

6.1 Pasos del flujo

Los programas e informes describen una serie de pasos: localizar y leer los datos, normalizar sus campos, construir las etiquetas, extraer las características, dividir los datos, entrenar, evaluar y guardar los resultados. La Figura 6.1 muestra este recorrido. Las tres líneas experimentales comparten la secuencia general, aunque difieren en sus cargadores, etiquetas, particiones y modelos.

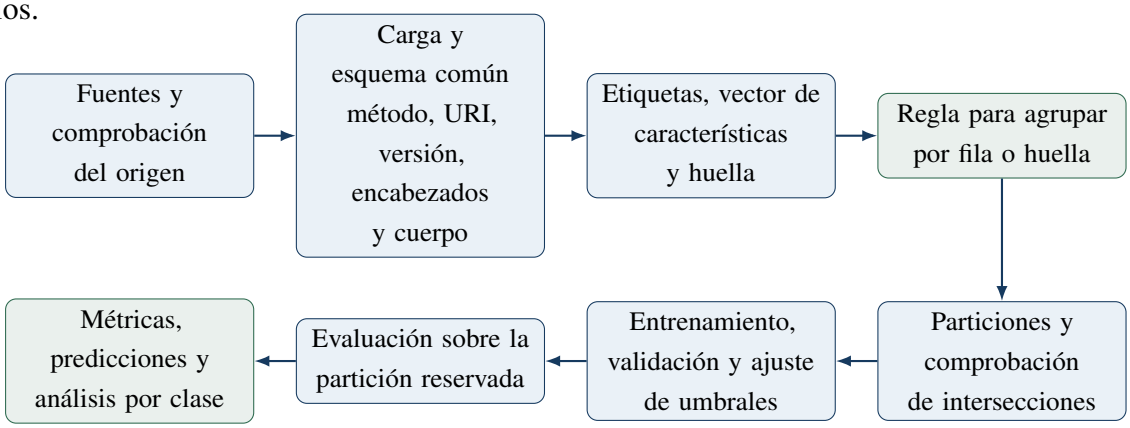


Figura 6.1: Secuencia implementada por el flujo experimental. En el protocolo multietiqueta agrupado, la construcción de la huella antecede a la división definida en la Sección 5.5.

6.2 Software y ejecución de las corridas

La implementación usa programas Python para cargar los datos, construir las características, entrenar los modelos, calcular las métricas y guardar los archivos de salida. Los lanzadores Bash fijan rutas, argumentos y recursos. Las ejecuciones de mayor tamaño se enviaron mediante Slurm; otras comprobaciones se realizaron localmente. Random Forest y HistGradientBoosting se implementan con scikit-learn; la línea multietiqueta usa XGBoost.

Los programas Python implementan la carga, la extracción, el entrenamiento y la evaluación. Los lanzadores Bash fijan los argumentos y recursos de cada ejecución, y Slurm registra la asignación solicitada. Según la corrida, las salidas contienen la configuración, las particiones, las predicciones, las métricas, las importancias y los tiempos de ejecución. La justificación de los modelos se encuentra en el Capítulo 5; su configuración, en el Capítulo 7.

6.2.1 Dependencias del software

Los lanzadores finales activan `.venv`, el directorio que aísla las bibliotecas de Python del proyecto. El archivo `requirements.txt` fija las versiones de la Tabla 6.1, mientras que `pyproject.toml` describe el paquete `waf-ml-starter` 0.1.0 y exige Python $\geq$ 3.10.

Tabla 6.1: Versiones fijadas en el archivo `requirements.txt` suministrado.

Función	Paquetes y versiones
Procesamiento y modelado	pandas 2.3.3; NumPy 2.4.0; scikit-learn 1.8.0; XGBoost 3.2.0; joblib 1.5.3
Visualización	Matplotlib 3.10.9
Lectura, progreso y recursos	PyYAML 6.0.3; tqdm 4.67.1; pyarrow 22.0.0; psutil 7.2.2
Servicio auxiliar	FastAPI 0.127.1; Uvicorn 0.40.0; python-multipart 0.0.21

Los registros de las corridas multietiqueta identifican además Python 3.11.7 y XGBoost 3.2.0. Estas versiones acompañan las configuraciones y resultados descritos en los capítulos experimentales.

6.2.2 Recursos de las ejecuciones principales

La Tabla 6.2 resume los recursos solicitados y el consumo registrado. Las directivas `#SBATCH` indican la asignación solicitada y MaxRSS informa el máximo de memoria observado por Slurm durante los pasos del trabajo.

Tabla 6.2: Asignación y consumo registrado de las corridas principales.

Ejecución	Trabajo y envío	Asignación y tiempo de pared	MaxRSS, biblioteca y límite
Multietiqueta agrupada	3315; `multietiquetaCorrida2.sh`	`c3`; 16 CPU y 60 GiB; 00:39:10	4,870 GiB; XGBoost en CPU.

El entorno de cómputo dispone de nodos Dell PowerEdge R740 con dos procesadores Intel Xeon Gold 6242, 125 GiB configurados en Slurm y una GPU Tesla V100 por nodo. La guía de ejecución, los resúmenes de integridad y los comandos principales se reúnen en el Apéndice B.

6.3 Carga y construcción del esquema común

Los cargadores admiten CSV, TSV y variantes comprimidas. Para Harvard reconocen alias habituales de método, URI y cuerpo; si falta alguno de esos campos requeridos, la carga finaliza con un error. Las columnas CAPEC se obtienen de la lista indicada en la configuración o se identifican mediante el formato esperado de sus nombres. El programa también registra las rutas de los archivos de entrada.

TorpEda, CSIC y DS-Augmented-v2 tienen cargadores propios para reconstruir la petición desde los campos disponibles. Todos producen la estructura $r = (m, u, v, h, b)$ definida en la Ecuación (2.1): método, URI, versión, encabezados y cuerpo. El Capítulo 4 muestra cómo se asignan los campos de cada fuente. Este paso sólo coloca los componentes disponibles en un orden común; no calcula una huella ni aplica las reglas limitadas de formato usadas para comparar peticiones repetidas. El extractor recibe siempre esta misma estructura. Los textos ausentes se representan con cadenas vacías y los números ausentes con ceros. Las columnas numéricas se revisan antes del entrenamiento.

En Harvard, el componente h se construye con once campos de encabezado. El vector de características se calcula desde el esquema común, mientras que la huella se calcula directamente desde los campos originales seleccionados. Las reglas exactas de esa segunda operación se documentan una sola vez en la Sección 6.6.

6.4 Extracción de características

6.4.1 Núcleo textual y estructural

El extractor separa la URI en camino y consulta y reconoce parámetros del cuerpo cuando éste puede interpretarse como una secuencia de pares. Calcula longitudes, profundidad, cantidad de parámetros, entropía, razones de caracteres, secuencias léxicas y conteos de símbolos. El código denomina **átomo** a cada segmento interno extraído durante ese análisis, por ejemplo una parte delimitada de la URI o del cuerpo.

En la implementación multietiqueta agrupada, los campos se concatenan sin agregar saltos CR/LF ficticios. Los extractores cuentan palabras, símbolos y secuencias HTTP definidas en los catálogos e informes, sobre el texto original o decodificado según la característica. La **codificación porcentual** representa un byte mediante el signo `%` y dos dígitos hexadecimales; las **entidades HTML** representan caracteres mediante secuencias como `<`. Se guardan indicadores separados para estas formas, la doble decodificación, los caracteres escapados y los bytes nulos.

6.4.2 Método, protocolo y encabezados

Los catálogos representan el método y la versión mediante indicadores numéricos. Para Harvard, los encabezados se reconstruyen desde once columnas y, cuando existe, desde una representación JSON. Sus valores se convierten en longitudes, entropía, cantidades o indicadores de coherencia. No se usan direcciones IP ni puertos explícitos como características, pero algunos encabezados pueden contener información sensible.

Este bloque incluye, por ejemplo, la cantidad de pares en *Cookie*, los productos declarados en *User-Agent*, las diferencias de host en *Origin*/*Referer*, la cantidad de elementos en *Accept* y la presencia de caracteres de control. Los campos de respuesta no intervienen.

6.4.3 Cuerpo y estructuras internas

Según los catálogos e informes, el tipo de contenido orienta el análisis de formularios, JSON, XML y contenido dividido en partes (*multipart*). Se describen indicadores de concordancia entre cuerpo y tipo declarado, además de cantidades de claves, etiquetas o partes cuando la estructura es reconocible. Los extractores ampliados también implementan conteos dirigidos a SSTI, SSRF e inyección. El Capítulo 7 indica en qué etapa se incorporó cada grupo.

6.4.4 Ejemplo trazable de petición a características

La Tabla 6.3 desarrolla un ejemplo que no pertenece a los corpus. Combina variables de las listas finales multiclase y `request90`. Muestra cómo una petición se convierte en números, pero no representa una predicción del modelo:

```
GET /index.php?id=1%20OR%201=1 HTTP/1.1
Host: ejemplo.local
User-Agent: curl/8.5
Accept: */*
Cookie: sid=abc
```

Tabla 6.3: Ejemplo de cómo una petición HTTP produce características numéricas incluidas en las listas finales.

Fragmento observado	**Variables afectadas**	**Interpretación**
`GET` y `HTTP/1.1`	`method_is_get`, `method_is_safe`, `http_version_is_1_1`	Indicadores de método y versión; no constituyen por sí solos evidencia de ataque.
`/index.php`	`uri_len`, `path_depth`, `full_request_len` `query_len`,	Longitudes y estructura del objetivo.
`id=...`	`n_query_params`, `max_param_name_len`, `max_param_value_len`	Estructura de parámetros de la consulta.
`%20`	`percent_count`, `uri_encoded_ratio`, `decode_once_length_delta_ratio`	Presencia y efecto de codificación porcentual.
`OR` y la igualdad, después de decodificar	`sql_token_count`, `uri_entropy`, `full_request_special_char_ratio`	Conteos de palabras y medidas estadísticas que el modelo combina; no demuestran por sí solos una inyección SQL.
Cuatro encabezados y una cookie	`request_header_present_count`, `request_header_total_len`, `cookie_pair_count`, `user_agent_entropy`	Resumen de estructura de encabezados. *Cookie* y *User-Agent* pueden permitir identificación indirecta, aunque aquí sólo se derivan medidas numéricas.
Cuerpo ausente	`body_len`, `n_body_params`, `content_type_body_mismatch`	Las variables de cuerpo quedan en cero salvo que el extractor detecte una inconsistencia.

Cada catálogo define un subconjunto y un orden fijo; no todas las corridas contienen todas las variables del ejemplo. El modelo combina varias características. Una sola palabra o secuencia no determina directamente la clase.

6.5 Cómo se construyen las etiquetas de salida

6.5.1 Salida binaria

Las etiquetas específicas se reducen a NORMAL o ATAQUE, y ATAQUE se considera la clase positiva. Esta reducción se utiliza como objetivo de Random Forest. Las predicciones multiclase y multietiqueta también pueden reducirse a NORMAL o ATAQUE para calcular métricas y una matriz de confusión binarias.

6.5.2 Taxonomías multiclase

El codificador de TorpEda conserva sus nombres nativos. Para Harvard se implementaron las taxonomías de nueve familias, top-14, top-11 y top-10. En top-14, cada combinación exacta admitida se representa como una única clase multiclase. En top-10, *Request Manipulation* agrupa CAPEC-153, CAPEC-194, CAPEC-272, CAPEC-274 y la combinación 272+274. La combinación CAPEC-88+CAPEC-126 permanece como una clase compuesta única, codificada en un solo valor de `label_multiclass`. En la formulación multietiqueta, en cambio, la misma fila activa por separado las columnas CAPEC-88 y CAPEC-126. Las combinaciones no admitidas por top-14 se excluyen antes de realizar la división.

Para DS-Augmented-v2 se implementó una tabla de ocho clases que fusiona Command Injection con OS Command Injection. El Capítulo 7 identifica la tabla usada en cada corrida y explica el efecto de cambiar el objetivo.

6.5.3 Etiquetas múltiples

Las columnas CAPEC se convierten en una matriz binaria con una columna por etiqueta modelada. NORMAL se deriva cuando ninguna de las etiquetas CAPEC modeladas está activa y no constituye un clasificador adicional, de acuerdo con (2.24). En la configuración fixed30, CAPEC-248 no se usa como variable objetivo porque posee un único positivo; la fila que la contiene se conserva para las demás etiquetas. El protocolo agrupado aplica después la consolidación descrita en la siguiente sección.

6.6 Huella, agrupación y consolidación

El protocolo multietiqueta agrupado usa los quince campos definidos en la Sección 4.7: método, URI, versión o protocolo, once encabezados y cuerpo. Convierte el método a mayúsculas, usa un formato estable para la versión, convierte los nombres de los encabezados a minúsculas y recorta espacios y tabulaciones al inicio y al final de los valores. Luego coloca los quince valores en un orden fijo, los separa indicando su longitud y calcula BLAKE2b-128 con una configuración propia del estudio. El resultado es una huella de 32 caracteres hexadecimales.

La huella y el vector de características tienen funciones diferentes. La huella permite agrupar como coincidentes las peticiones que producen el mismo código bajo la regla de quince campos y se calcula antes de dividir los datos. El vector numérico se calcula después y es la entrada del modelo. El protocolo conserva un registro consolidado por huella y aplica la unión lógica (OR) a las etiquetas del grupo. El programa agrupa los registros por huella y comprueba las seis intersecciones entre las cuatro particiones.

La Figura 6.2 resume esta secuencia de construcción y comprobación.

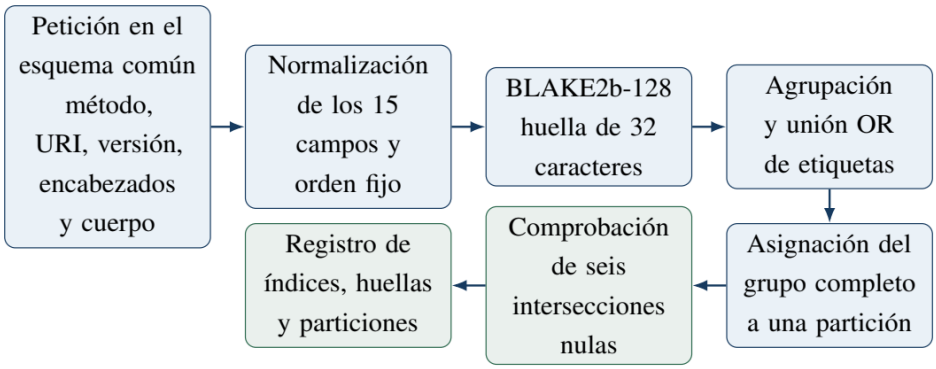


Figura 6.2: Secuencia implementada para construir la huella, consolidar los grupos y registrar las particiones multietiqueta agrupadas.

6.7 Modelos implementados

6.7.1 Random Forest binario

La implementación final de B2 declara 500 árboles, pesos de clase balanceados, 57 características, división estratificada 80/20 y semilla 42. Después del entrenamiento se prueban umbrales entre 0,05 y 0,95 con las probabilidades del mismo conjunto de prueba y se conserva el que produce el mayor F1. El programa crea un modelo independiente para cada corpus. La consecuencia de elegir el umbral sobre prueba se explica en la sección 5.6.

Los tres corpus usan la misma función y el mismo vector nominal de 57 variables. req_content_length toma el encabezado Content-Length cuando está disponible y utiliza la longitud del cuerpo como respaldo. El cargador de CSIC entrega los encabezados al extractor; los cargadores binarios de TorpEda y Harvard entregan método, URI y cuerpo, por lo que en estos últimos la variable se obtiene de la longitud del cuerpo.

6.7.2 HistGradientBoosting multiclase

La implementación multiclase usa HistGradientBoostingClassifier. M8 usa un máximo de 750 iteraciones, tasa de aprendizaje 0,035, 63 hojas, regularización ℓ_2 de 0,05, parada temprana y ejecución en CPU.

6.7.3 XGBoost uno contra el resto

La implementación multietiqueta entrena un XGBClassifier independiente por CAPEC. La línea base por filas usa treinta características, particiones de entrenamiento, validación y prueba por filas, ausencia de ponderación y umbrales por etiqueta elegidos para maximizar F1 en validación. Después vuelve a entrenar los estimadores sobre entrenamiento más validación y conserva los umbrales obtenidos con los modelos anteriores. La interpretación se presenta en la sección 5.6.

Durante ese reentrenamiento, la validación forma parte de los datos de ajuste y vuelve a pasarse a XGBoost como eval_set para la parada temprana. Por tanto, esa parada no usa una validación

independiente. La prueba por filas permanece fuera de estas operaciones, pero los estimadores finales no son los mismos con los que se eligieron los umbrales. L1 se reporta como línea base histórica.

La evaluación agrupada separa entrenamiento, validación, ajuste de umbrales y prueba. Para reanudar una ejecución, compara las firmas de los archivos de entrada y de la configuración y rechaza puntos de control incompatibles. Durante la selección se evalúan doce configuraciones para cada una de nueve etiquetas y veintiocho para cada una de CAPEC-153, CAPEC-272 y CAPEC-274: 192 configuraciones en total.

Cada ensayo usa binary:logistic, parada temprana según la AUC-PR interna (aucpr) de XGBoost y una búsqueda de scale_pos_weight por etiqueta. La elección sigue la ecuación (5.1). El modelo se elige con validación; los umbrales se inician y ajustan conjuntamente en el bloque siguiente, y la prueba se consulta al final. Esta evaluación no vuelve a entrenar después de fijar los umbrales.

Entre las opciones examinadas se encuentra max_delta_step, que limita el valor de una hoja después de calcular (2.23). Si δmax = 0, XGBoost desactiva el límite; si δmax > 0, aplica la ecuación (6.1).

$$q^{\star}_{u,\delta} = \begin{cases} q^{\star}_{u}, & \delta_{\max} = 0, \\ \mathrm{clip}(q^{\star}_{u}, -\delta_{\max}, \delta_{\max}), & \delta_{\max} > 0. \end{cases} \tag{6.1}$$

El operador clip(q, a, b) limita q al intervalo [a, b]: conserva el valor si ya está dentro y usa el extremo más cercano cuando lo supera. La búsqueda considera los valores 0, 1, 3 y 5 para δmax; la tasa de aprendizaje multiplica después la contribución del árbol. Esta opción pertenece a la configuración concreta y no a la formulación general multietiqueta (XGBoost developers, 2026).

6.8 Medición operativa

El modo individual procesa una petición e incluye la extracción de características y la predicción. El modo por lote cronometra la construcción de la matriz y la predicción conjunta. La inferencia mide sólo la predicción sobre vectores ya construidos. Cada programa registra además el tamaño del modelo, la memoria máxima y el tiempo total cuando están disponibles. Los valores observados se presentan en el capítulo 8 y sus límites se discuten en la sección 9.7.

Capítulo 7

Configuración e historial de experimentos

Una letra identifica la formulación y el número distingue etapas sucesivas; no expresa una jerarquía de calidad. Las pruebas previas de OCSVM, Isolation Forest y regresión logística explican la elección de la referencia binaria y se presentan en la sección 7.4.

Tabla 7.1: Leyenda de los identificadores experimentales.

Rango	Línea	Función general
B1-B2	Binaria supervisada	Ablación y aplicación multicorpus con Random Forest.
M1-M8	Multiclase	Evolución de características, taxonomías y corpus.
L1-L2	Multietiqueta	Línea base por filas y evaluación agrupada por huella.

7.1 Nivel de evidencia de las corridas

B1, B2 y M1-M8 se conservan como exploratorias porque la prueba intervino en el desarrollo de umbrales, variables o taxonomías. L1 es la línea base multietiqueta por filas. L2 agrupa las peticiones antes de dividir, reserva bloques distintos para validación, ajuste de umbrales y prueba, y comprueba que no haya huellas compartidas entre las cuatro particiones. Estas categorías siguen los criterios de la sección 5.2.

7.2 Representaciones ejecutadas

Tabla 7.2: Representaciones utilizadas, con nombres descriptivos unificados.

Nombre	Dim.	Formulación	Contenido o propósito
Base estructural (RF-25)	25	Binaria	Longitudes, estructura, codificación y conteos básicos.
Entropías y proporciones (RF-33)	33	Binaria	Añade cuatro entropías y cuatro proporciones de caracteres a RF-25.
Completa multicorpus (RF-57)	57	Binaria	Añade 24 variables de distribución de caracteres.
Selección por importancia (RF-10-Gini)	10	Binaria	Conserva las diez variables con mayor importancia Gini de RF-57.
Núcleo HTTP compartido (fixed30)	30	Multiclase y L1	Longitudes, razones, entropía, símbolos y secuencias léxicas.
Ampliada HTTP	84	Multiclase	Incorpora método, protocolo, parámetros, XPath/XML y rutas.
Seleccionada por importancia	58	Multiclase	Retiene variables con importancia positiva previa para alguna clase.
Ampliada con señales WAMM	72	M8	Añade características generales y dirigidas a SSTI, SSRF e inyección.
Extendida multietiqueta (request90)	90	L2	Extiende fixed30 con método y versión, encabezados, cuerpo, parámetros, codificación y decodificación.

Los identificadores de corrida y los nombres de las representaciones cumplen funciones distintas. En RF-n, n es la cantidad de variables, no de árboles. RF-10-Gini contiene diez variables seleccionadas por importancia; Harvard top-10 designa una taxonomía de diez clases. fixed30 y request90 se conservan como identificadores técnicos de las representaciones multietiqueta.

El informe de ablación emplea el nombre histórico RF-34, pero enumera 25 variables originales más ocho nuevas. Se adopta RF-33 para evitar esa inconsistencia. Las ocho añadidas son cuatro entropías y cuatro proporciones de caracteres. La representación completa incorpora otras 24: 25 + 8 + 24 = 57. Esta normalización de nombres no modifica las corridas OCSVM de 26 y 34 variables.

En request90, catorce columnas son constantes después de consolidar la población. Se conservaron para evaluar la representación ejecutada; una repetición futura debe compararla con un vector depurado. En algunos nombres de request90, normalización se refiere al efecto numérico de decodificar el texto y no a las reglas para calcular la huella.

7.3 Orden y propósito de las corridas

Las tres líneas principales avanzaron en paralelo. No forman un único experimento que haya probado todas las combinaciones de hiperparámetros. Los informes 01 y 02 describen una misma corrida multiclase; el segundo corrige al primero. En los alias multiclase top-14, top-11 y top-10, el número indica la cantidad total de clases.

Tabla 7.3: Secuencia de experimentos y cambio evaluado en cada etapa.

ID	Formulación	Configuración	Propósito o cambio
Previa	One-class y binaria	OCSVM, Isolation Forest, regresión logística y RF	Revisar ajuste, representación y tipo de aprendizaje antes de fijar RF.
B1	Binaria	CSIC; RF-25, RF-33, RF-57 y RF-10-Gini	Comparación exploratoria de grupos de características.
B2	Binaria	Tres corpus; RF-57 y 500 árboles	Repetir el flujo con un entrenamiento independiente por corpus.
M1	Multiclase	fixed30; TorpEda y Harvard	Núcleo compartido con diez clases TorpEda y nueve familias Harvard.
M2	Multiclase	fixed30; Harvard top-14	Conservar combinaciones CAPEC frecuentes como clases exactas.
M3	Multiclase	84 variables; top-14	Añadir método, encabezados, cuerpo, XPath/XML y rutas.
M4	Multiclase	58 variables; top-14	Conservar rendimiento después de eliminar variables.
M5	Multiclase	58 variables; Harvard top-11	Agrupar categorías CAPEC relacionadas.
M6	Multiclase	58 variables; Harvard top-10	Unificar cinco grupos como Request Manipulation.
M7	Multiclase	58 variables; tres corpus	Incorporar DS-Augmented-v2 con ocho clases, uniendo Command Injection y OS Command Injection.
M8	Multiclase	72 variables; tres corpus	Añadir catorce señales inspiradas en WAMM; DS conserva ocho clases.
L1	Multietiqueta	fixed30; Harvard; XGBoost uno contra el resto	Línea base de doce CAPEC por filas; reentrenamiento tras elegir umbrales.
L2	Multietiqueta	request90; Harvard; XGBoost uno contra el resto	Consolidación y partición por huella; validación y umbrales separados; sin reentrenamiento posterior.

7.4 Experimentos binarios

7.4.1 One-Class SVM inicial y ajuste de hiperparámetros

El primer flujo One-Class SVM se entrenó principalmente con tráfico normal de CSIC 2010. En la ejecución completa detectó 7.306 de 20.864 ataques, con F1 = 0,5187 y sensibilidad de 35,02 %. Produjo solo tres falsos positivos entre 13.200 peticiones normales. Este resultado mostró un comportamiento conservador: reconocía casi todas las normales, pero omitía aproximadamente el 64,98 % de los ataques (Román Medina y González Arévalos, 2026, p. 3).

El diagnóstico recogido en el informe identificó que el ajuste automático favorecía la aceptación de normales. Se revisaron nu y gamma, parámetros del entrenamiento de OCSVM. Con gamma = 0,1, los valores nu = 0,01, 0,05 y 0,10 produjeron F1 de 0,765, 0,787 y 0,794, respectivamente, y FPR de 0,007, 0,099 y 0,150. Se tomó nu = 0,05 como referencia de balance. La mejora confirmó la

importancia del ajuste, aunque aumentar la detección también elevó las falsas alarmas. Este ajuste de nu y gamma debe distinguirse del cambio de umbral aplicado a puntuaciones o probabilidades después del entrenamiento.

También se evaluó la variante automática mean_score, con F1 = 0,607, sensibilidad = 0,440 y FPR = 0,017. Su mejora fue parcial y no reemplazó la configuración de referencia.

7.4.2 Pruebas por endpoint y ampliación de características

Para estudiar si la heterogeneidad entre endpoints explicaba el bajo rendimiento, se entrenó y evaluó OCSVM únicamente sobre registro.jsp. Con 26 características se obtuvo F1 aproximado de 0,457, sensibilidad de 0,301 y FPR de 0,056. La prueba no produjo la mejora esperada. Debilita la explicación de que mezclar endpoints fuera la única causa, pero no descarta que una arquitectura especializada por grupos pudiera resultar útil bajo otro diseño (Román Medina y González Arévalos, 2026, pp. 3-4).

La ampliación de 26 a 34 características en registro.jsp mantuvo F1 cercano a 0,457. Al extender el OCSVM global a 57, F1 permaneció alrededor de 0,787. El informe recoge alta esparsidad de algunas variables y valores SHAP cercanos a cero en varias características nuevas. Estas observaciones se limitan a las configuraciones examinadas. La secuencia OCSVM de 26 y 34 variables no debe confundirse con RF-25 y RF-33: el alias histórico RF-34 de la ablación enumera 25 variables originales más ocho adicionales y se normaliza como RF-33 en este libro.

7.4.3 Isolation Forest

La exploración continuó con Isolation Forest y 25 características HTTP. Según la reconstrucción metodológica, el entrenamiento final utilizó las 57.600 muestras normales del conjunto de entrenamiento. La corrida obtuvo F1 de anomalía = 0,5588, sensibilidad de 55,91 % y FPR de 15,39 %. El algoritmo separó parcialmente ambos grupos, pero no resolvió conjuntamente la cobertura de ataques y las falsas alarmas. Este resultado motivó revisar la formulación supervisada, sin establecer una comparación controlada de superioridad entre Isolation Forest y OCSVM (Román Medina y González Arévalos, 2026, p. 4).

En la importancia por permutación de esa corrida destacaron body_encoded_ratio, uri_entropy, uri_non_alnum_ratio, uri_encoded_ratio, query_len y avg_param_value_len. Varias señales explícitas de ataque tuvieron aporte bajo o nulo; ese resultado reforzó la necesidad de revisar cómo se representaban las peticiones.

7.4.4 Paso a la clasificación supervisada

La pregunta siguiente fue si la representación de 57 características contenía información para distinguir explícitamente NORMAL de ATAQUE cuando el entrenamiento incluyera ambas clases. La regresión logística alcanzó F1 aproximado de 0,665, sensibilidad de 0,750 y FPR de 0,160. Random Forest obtuvo F1 aproximado de 0,947, sensibilidad de 0,919 y FPR de 0,007. La diferencia observada indicó que introducir supervisión no bastaba para obtener el mismo comportamiento con cualquier modelo (Román Medina y González Arévalos, 2026, p. 5).

7.4.5 Ablación en CSIC

B1 compara cuatro representaciones de Random Forest: RF-25, RF-33, RF-57 y RF-10-Gini. La última conserva diez variables de RF-57 según su importancia Gini. La corrida permite estudiar qué ocurre al ampliar la representación o reducirla a las variables mejor posicionadas. Se conserva como evidencia exploratoria debido al uso de la prueba durante el desarrollo.

El informe de ablación identifica las representaciones y sus métricas, pero no detalla por sí solo todos los parámetros de entrenamiento y partición. La configuración de 500 árboles, pesos balanceados, división 80/20 y semilla 42 está explícita en el informe de B2. Esos valores no se extrapolan a las pruebas anteriores sin su registro correspondiente (Equipo de tesis, 2026a, 2026b).

La comparación con OCSVM reajustado y con OCS-WAF se conserva como contexto histórico: esos resultados no fueron obtenidos con el mismo protocolo. Las cifras de B1 se presentan en la sección 8.3.1.

7.4.6 Configuración común en tres corpus

B2 utiliza RF-57, 500 árboles, balanceo de clases, partición 80/20 y semilla 42 en CSIC, TorpEda y Harvard. Cada corpus se divide, entrena y evalúa por separado. El umbral se elige sobre la prueba, por lo que los resultados son exploratorios.

Aunque el informe usa la expresión comparación entre corpus, no se entrena en uno para probar el mismo modelo en otro. Se repite el flujo y se vuelve a entrenar en cada corpus. La diferencia de req_content_length entre cargadores se explica en la sección 6.7.1.

7.5 Evolución multiclase

7.5.1 De la línea base a la expansión

M1 usa treinta variables y una reducción de Harvard a nueve familias. M2 cambia a catorce combinaciones exactas frecuentes y conserva 902.411 filas. Este mayor detalle produce categorías que el modelo confunde con frecuencia. M3 mantiene esas clases y amplía la representación a 84 variables para comprobar si el método, los parámetros y el protocolo ayudan a distinguirlas.

7.5.2 Eliminación de características y agrupación de clases

M4 conserva 58 variables cuya importancia por permutación fue positiva para al menos una clase en M3. Esa importancia se calculó con la prueba top-14 y M4 reutiliza los mismos índices. Por ello, la evaluación no es independiente de la selección de variables.

M4, M5 y M6 mantienen 58 características y modifican la taxonomía de Harvard. M5 une CAPEC-153 con CAPEC-194 como Data Manipulation y CAPEC-272 con CAPEC-274 y su combinación como HTTP Protocol Manipulation. M6 integra esos grupos en Request Manipulation. Al dejar de evaluar por separado categorías que se confundían entre sí, F1 macro aumenta de 88,81 % en M4 a 94,40 % en M5 y 97,10 % en M6. La mejora se obtiene a cambio de una salida menos detallada.

7.5.3 Incorporación del tercer corpus

M7 aplica 58 variables a TorpEda, Harvard top-10 y DS-Augmented-v2. En este último utiliza egipcios_col2_canonicalized y ocho clases totales porque agrupa Command Injection y OS Command Injection. canonicalized indica que los nombres originales se normalizaron antes de agruparlos. M8 añade catorce características dirigidas y llega a 72 variables, manteniendo esas ocho clases.

M1-M8 emplean divisiones por fila y la misma semilla externa. Algunas características y clases se diseñaron después de observar resultados de prueba.

7.6 Evolución multietiqueta

7.6.1 Primera corrida

L1 trabaja sobre 907.815 filas y treinta variables. Divide por fila en entrenamiento, validación y prueba. Entrena un XGBoost por CAPEC, selecciona umbrales y después vuelve a ajustar los modelos sobre entrenamiento más validación sin volver a seleccionar esos umbrales. Produce métricas globales, por etiqueta y binarias derivadas. Es la primera corrida que predice varios tipos de ataque al mismo tiempo, pero comparte las limitaciones del protocolo histórico respecto del contenido repetido.

7.6.2 Segunda corrida

L2 agrupa antes de partir y obtiene 542.990 registros consolidados, 90 características, validación y ajuste de umbrales separados, y seis intersecciones nulas. La división produce 347.513 grupos de entrenamiento, 43.439 de validación, 43.440 para ajustar umbrales y 108.598 de prueba. La población consolidada contiene 171.348 grupos normales y 371.642 con al menos una etiqueta de ataque.

Durante la selección se evalúan doce configuraciones para cada una de nueve etiquetas y veintiocho para cada una de CAPEC-153, CAPEC-272 y CAPEC-274, para un total de 192 configuraciones. L2 no vuelve a ajustar el modelo después de elegir los umbrales.

L2 es el resultado principal de la línea multietiqueta por su partición en grupos según la huella definida. L1 se conserva como línea base histórica por filas. Sus métricas no constituyen una mejora controlada respecto de L1 porque cambian la población, la unidad de análisis, las variables y el protocolo.

7.7 Comprobación de coincidencias de huella entre particiones

El análisis de peticiones repetidas reproduce filtros, etiquetas, orden de filas, semilla y funciones de partición. Después compara huellas entre desarrollo y prueba y, en una segunda verificación, las 38 columnas completas.

El análisis cubre cuatro taxonomías multiclase Harvard y L1/L2. No se reconstruyó la partición exacta de B2 sobre Harvard; por ello, no se atribuye a esa corrida un porcentaje de coincidencias bajo la regla de huella. Tampoco puede estimarse cuánto cambiaría cada métrica sin volver a entrenar con los grupos separados.

Capítulo 8

Resultados

8.1 Criterio de presentación

Una diferencia entre corridas no se atribuye a una causa específica cuando también cambiaron otros elementos del experimento. Los términos exploratorio, final de desarrollo y prueba separada por huella siguen la sección 5.2. Los porcentajes conservan la precisión de los informes originales; los decimales no reducen la incertidumbre de una única ejecución.

Los resultados son estimaciones puntuales. No se realizaron repeticiones independientes ni se calcularon intervalos de confianza. Los términos mayor, menor y mejor describen diferencias observadas. La robustez examinada se limita a la ablación B1, la evolución histórica, el análisis por clase y la comprobación de huellas de L2.

8.2 Resultados previos a la referencia binaria

La tabla 8.1 reúne las pruebas previas a B1 y la referencia Random Forest, según Román Medina y González Arévalos (2026, pp. 3-5). Las métricas son proporciones redondeadas. Describen etapas con protocolos distintos, no una comparación controlada entre algoritmos.

Tabla 8.1: Resultados históricos que motivaron la selección de Random Forest.

Modelo o variante	Formulación	Dim.	F1	Sensibilidad	FPR
OCSVM inicial	One-class	~26	0,518	0,350	~0,0002
OCSVM nu = 0,01	One-class	~26	0,765	0,622	0,007
OCSVM nu = 0,05	One-class	~26	0,787	0,689	0,099
OCSVM nu = 0,10	One-class	~26	0,794	0,720	0,150
OCSVM mean_score auto	One-class	N/D	0,607	0,440	0,017
OCSVM registro.jsp	One-class	26	0,457	0,301	0,056
OCSVM registro.jsp ampliado	One-class	34	0,457	0,301	0,056
OCSVM global ampliado	One-class	57	0,787	0,689	0,099
Isolation Forest	One-class	25	0,559	0,559	0,154
Regresión logística	Binaria supervisada	57	0,665	0,750	0,160
Random Forest	Binaria supervisada	57	0,947	0,919	0,007

Las tres variantes manuales de nu usan gamma = 0,1; nu = 0,05 fue la referencia de balance. La cantidad inicial de variables se conserva como aproximada. N/D indica que la síntesis no explicita ese dato para mean_score. Isolation Forest se entrenó finalmente con 57.600 normales. Cada etapa conserva su propio protocolo.

El ajuste mejoró el OCSVM inicial, pero aislar registro.jsp y ampliar las características no produjo las mejoras buscadas. Con 57 variables, Random Forest obtuvo F1 = 0,947, sensibilidad = 0,919 y FPR = 0,007. La regresión logística, con la misma cantidad de variables, obtuvo F1 = 0,665. El resultado motivó fijar Random Forest para estudiar la representación. Las diferencias de entrenamiento, representación y protocolo impiden atribuir toda la variación al algoritmo.

8.3 Clasificación binaria supervisada

8.3.1 Ablación de características en CSIC

La tabla 8.2 presenta las cuatro representaciones de Random Forest. RF-57 obtuvo el mayor F1 observado y RF-33 quedó 0,012 por debajo. La comparación se conserva como exploratoria: no permite aislar causalmente el aporte de cada bloque ni estimar la variación entre entrenamientos.

Tabla 8.2: Ablación exploratoria de características en CSIC 2010.

Configuración	F1	Sensibilidad	Precisión	FPR	BAcc	ROC-AUC
RF-57 completa	0,947	0,919	0,975	0,007	0,956	0,9971
RF-33 ampliada	0,935	0,905	0,966	0,010	0,948	0,9958
RF-10-Gini seleccionada	0,914	0,863	0,971	0,008	0,927	0,9856
RF-25 base	0,904	0,861	0,951	0,014	0,924	0,9816

Fuente: Equipo de tesis (2026a). El uso previo de la prueba limita el alcance de B1; la configuración y la trazabilidad se precisan en la sección 7.4.5.

La reducción de RF-57 a RF-10-Gini perdió 3,3 puntos porcentuales de F1. Este resultado aporta evidencia sobre el conjunto reducido y no demuestra el efecto individual de cada variable. La tabla 8.3 enumera las características seleccionadas.

Tabla 8.3: Características de RF-10-Gini seleccionadas desde RF-57.

Pos.	Característica	Importancia	Grupo
1	uri_entropy	0,141	Entropía
2	uri_pct_non_alnum_ratio	0,121	Estructural
3	uri_len	0,100	Estructural
4	path_depth	0,046	Estructural
5	body_encoded	0,045	Estructural
6	req_content_length	0,029	Estructural
7	encoded	0,028	Estructural
8	body_len	0,026	Estructural
9	body_char_dist_i0	0,025	Distribución
10	body_pct_alpha	0,023	Composición de caracteres

La selección contiene seis señales estructurales, una de entropía y tres de composición o distribución de caracteres. uri_entropy ocupa el primer lugar. La importancia Gini indica cuánto utilizó Random Forest cada variable en esa corrida; no demuestra causalidad ni asegura la misma utilidad fuera de CSIC.

8.3.2 Misma configuración aplicada por corpus

La tabla 8.4 reúne los resultados de B2, cuyo flujo se repite con entrenamiento independiente en TorpEda, Harvard y CSIC.

Tabla 8.4: Resultados exploratorios de Random Forest por corpus.

Corpus	Umbral	F1	FPR	FNR	MCC
TorpEda	Óptimo sobre prueba	0,9989	0,0012	0,0021	0,9900
SR-BH 2020	0,44 sobre prueba	0,9507	0,0342	0,0514	0,9149
CSIC 2010	Óptimo sobre prueba	0,9467	0,0073	N/D	N/D

Fuente: Equipo de tesis (2026b), informe multicorpus del 9 de junio de 2026. N/D: dato no disponible en la fuente.

Para CSIC se informan F1 y FPR, las métricas disponibles en ese informe. FNR y MCC requieren conteos adicionales y se mantienen como N/D.

En Harvard, el informe compara el umbral fijo 0,50 con 0,44. El segundo se eligió por producir el mayor F1 sobre prueba: eleva F1 de 0,9504 a 0,9507 y sensibilidad de 0,9429 a 0,9486, pero aumenta FPR de 0,0301 a 0,0342. La mejora de F1 es pequeña y no constituye una evaluación independiente porque la prueba se utilizó para elegir el umbral.

El orden de importancia Gini en Harvard difiere del obtenido en CSIC. encoded ocupa el primer lugar (0,264), seguido de uri_entropy (0,174), uri_len (0,164), uri_pct_non_alnum_ratio (0,131) y path_depth (0,081). Una misma configuración puede utilizar señales distintas en cada corpus.

TorpEda contiene 88,72 % de ataques y presenta el mayor F1 observado entre los tres flujos binarios. Esa proporción hace necesario leer el resultado junto con MCC = 0,9900 y FPR = 0,0012. Cada valor corresponde a un modelo entrenado y evaluado dentro de su propio corpus.

8.4 Evolución multiclase

8.4.1 Resultados globales exploratorios por etapa

La tabla 8.5 ordena transiciones de distinto tipo. En Harvard, M4-M6 mantienen 58 variables y cambian la agrupación de las clases; esa secuencia muestra el efecto de simplificar la taxonomía. En las demás transiciones cambian además la representación, el corpus o el número de clases.

Tabla 8.5: Evolución exploratoria de resultados multiclase.

Etapa	Corpus y tarea	Dim.	Exactitud (%)	BAcc (%)	F1 macro (%)	MCC
M1	TorpEda	30	99,22	93,75	94,23	0,987
M1	Harvard 9 familias	30	95,09	92,11	86,64	0,918
M2	Harvard top-14	30	94,88	87,69	80,81	0,914
M3	TorpEda	84	99,65	99,09	99,37	0,994
M3	Harvard top-14	84	96,32	94,36	88,83	0,938
M4	TorpEda	58	99,65	99,09	99,37	0,994
M4	Harvard top-14	58	96,30	94,34	88,81	0,937
M5	Harvard top-11	58	97,16	94,56	94,40	0,951
M6	Harvard top-10	58	97,50	96,33	97,10	0,957
M7	DS-Aug.-v2 (8 clases)	58	99,01	84,76	81,17	0,9788
M8	TorpEda	72	99,777	97,82	98,68	0,9963
M8	Harvard top-10	72	97,745	96,62	97,23	0,9610
M8	DS-Aug.-v2 (8 clases)	72	99,425	86,39	84,49	0,9876

En M8, las curvas del informe no determinan la métrica final. Sus errores finales para Harvard (2,284 %) y DS (0,808 %) no coinciden con los valores calculados sobre toda la prueba (2,2545 % y 0,5755 %). El informe no indica qué muestra se usó para esas curvas. Se toman como referencia las tablas globales; las curvas se consideran un análisis posterior al experimento y no una validación adicional.

8.5 Rendimiento operativo multiclase

La Tabla 8.6 conserva la medición histórica de M8. La medición por petición incluye la extracción de características y la predicción. El informe no indica cuántas filas se usaron en esta prueba de rendimiento. La predicción sobre vectores ya construidos fue mucho más rápida que el flujo completo.

Tabla 8.6 : Mediciones operativas de M8.

Corpus	p50 ms	p95 ms	p99 ms	En línea, pet./s	Por lotes, pet./s	Modelo MiB	RSS MiB	Tiempo
TorpEda	20,14	21,33	22,45	49,68	235,87	4,35	716	12 min 47 s
Harvard	25,19	25,95	26,51	39,63	1.860,98	15,24	5.347	1 h 32 min 48 s
DS-Augmented-v2	13,68	48,95	96,51	55,11	176,01	9,02	6.103	1 h 11 min 35 s

La diferencia entre el flujo completo por lotes y la inferencia sobre vectores ya construidos incluye el análisis del texto HTTP, su decodificación, la búsqueda de patrones y otros costos de preparación; esta medición no separa el tiempo de cada operación. La inferencia por lotes alcanza 87.007,01, 28.582,50 y 52.658,84 peticiones/s en TorpEda, Harvard y DS-Augmented-v2, respectivamente. Estas cifras no incluyen red, TLS, solicitudes concurrentes ni un motor de reglas.

8.6 Clasificación multietiqueta

8.6.1 Línea base histórica L1

L1 usa 907.815 filas, doce etiquetas y treinta variables. Divide las filas en 581.001 para entrenamiento, 145.251 para validación y 181.563 para prueba. No agrupa peticiones mediante huellas ni elimina repeticiones. Después de elegir los umbrales, vuelve a entrenar los estimadores con la unión de entrenamiento y validación, pero conserva los umbrales obtenidos con los modelos anteriores. L2 modifica este procedimiento: selecciona la configuración con validación, fija los umbrales en una partición separada para ese ajuste y consulta la prueba sólo al final, sin reentrenamiento posterior.

La Tabla 8.7 presenta en conjunto los resultados globales de ambos protocolos.

Tabla 8.7 : Resultados multietiqueta globales de L1 y L2. Las unidades de prueba difieren.

Métrica	**L1 fixed30**	**L2 request90**
Unidad de prueba	181.563 filas	108.598 grupos consolidados
Coincidencia exacta	96,531%	96,920%
Pérdida de Hamming	0,330%	0,276%
Precisión micro	97,462%	99,153%
Sensibilidad micro	93,288%	96,103%
F_1 micro	95,329%	97,604%
F_1 macro	83,498%	93,940%
F_1 ponderado	95,067%	97,498%
Jaccard micro	91,075%	95,321%
ROC-AUC macro	99,378%	99,890%
AP macro	84,169%	96,260%

L2 supera numéricamente a L1 en todas las métricas globales informadas; la pérdida de Hamming baja de 0,330 % a 0,276 %, el $F_{1,\text{micro}}$ sube de 95,329 % a 97,604 % y el $F_{1,\text{macro}}$ de 83,498 % a

93,940 %. Esta es una comparación de los flujos completos: L2 consolida por huella, usa 90 variables en lugar de 30, separa validación y ajuste de umbrales y no reentrena después.

La coincidencia exacta global incluye las unidades normales —filas en L1 y grupos consolidados en L2—, cuyo vector verdadero no contiene etiquetas de ataque. Por ello puede aumentar cuando el modelo reproduce correctamente el vector todo-cero y no debe interpretarse como una medida exclusiva de las solicitudes maliciosas. Para L1 se conservó el desglose exacto que sigue. Para L2, el porcentaje global redondeado y los 33.925 grupos normales correctamente reconocidos sitúan la coincidencia exacta de los grupos de ataque alrededor de 95,96 %, pero las predicciones o el conteo sin redondear son necesarios para informar una cifra exacta.

La coincidencia exacta de L1 no fue igual en ambos grupos. Alcanzó 99,37 % en las 105.039 filas normales y 92,63 % en las 76.524 filas de ataque. Al reducir la salida a normal o ataque, la matriz produjo $TN = 104.382$, $FP = 657$, $FN = 4.369$ y $TP = 72.155$. Estas cantidades corresponden a una exactitud de 97,23 %, un F_1 de ataque de 96,63 %, MCC de 0,9437, FPR de 0,625 % y FNR de 5,71 %.

8.6.2 Resultados por etiqueta y errores de L1

La Tabla 8.8 muestra que el rendimiento cambia según la etiqueta. CAPEC-153, CAPEC-272 y CAPEC-274 presentan los tres menores F_1. Por tanto, una métrica micro alta no significa que todas las etiquetas se clasifiquen con la misma calidad.

Tabla 8.8 : Métricas por etiqueta de L1.

CAPEC	Soporte	Umbral	Prec.	Sens.	F_1	AP
126	4.198	0,356465	98,01%	94,04%	95,99%	97,76%
153	448	0,232727	34,92%	63,84%	45,15%	37,90%
16	368	0,940505	99,46%	99,18%	99,32%	99,76%
194	11.230	0,564343	99,13%	92,58%	95,75%	98,02%
242	3.168	0,851414	100,00%	98,80%	99,40%	99,48%
272	1.833	0,237677	58,84%	33,22%	42,47%	48,28%
274	1.107	0,188182	61,76%	37,94%	47,01%	48,17%
310	545	0,430707	95,32%	78,53%	86,12%	84,51%
33	211	0,262424	95,83%	98,10%	96,96%	99,82%
34	3.941	0,772222	100,00%	100,00%	100,00%	100,00%
66	50.091	0,420808	99,03%	96,27%	97,63%	99,74%
88	1.509	0,440606	98,68%	93,84%	96,20%	96,60%

El análisis de las combinaciones de etiquetas permite contar errores concretos. Se predijeron como NORMAL 1.570 peticiones SQLi, 993 CAPEC-272, 662 CAPEC-194 y 647 CAPEC-274. En el sentido contrario, 188 peticiones normales se etiquetaron como SQLi, 139 como CAPEC-272 y 99 como CAPEC-153. La matriz no permite conocer por sí sola la causa de estos errores. Sí permite identificar qué pares de clases deben revisarse con ejemplos de peticiones.

8.6.3 Resultado principal L2

L2 evalúa 108.598 grupos consolidados en la partición de prueba. La huella resume los quince campos HTTP usados para agrupar peticiones, y las etiquetas de cada grupo se unen mediante una operación

OR. Ningún grupo de prueba comparte huella con los usados para entrenamiento, validación o ajuste de umbrales. Diez de las doce etiquetas superan 85 % de F_1. CAPEC-153 alcanza 80,86 % y CAPEC-272 obtiene el menor resultado, con 58,04 %.

La Tabla 8.9 detalla el rendimiento de L2 para cada etiqueta.

Tabla 8.9 : Métricas por etiqueta de L2 sobre la prueba separada por huella.

CAPEC	Soporte	Umbral	Prec.	Sens.	F_1	AP
126	3.794	0,687616	98,60%	98,58%	98,59%	99,88%
153	452	0,505643	70,20%	95,35%	80,86%	91,52%
16	375	0,003397	100,00%	99,73%	99,87%	99,76%
194	10.679	0,904424	99,37%	97,78%	98,57%	99,93%
242	3.151	0,987145	99,97%	99,30%	99,63%	99,96%
272	1.568	0,349251	72,02%	48,60%	58,04%	67,09%
274	994	0,009391	100,00%	100,00%	100,00%	100,00%
310	503	0,015705	97,85%	99,60%	98,72%	97,85%
33	215	0,700672	100,00%	93,49%	96,63%	99,81%
34	3.949	0,256638	100,00%	99,87%	99,94%	100,00%
66	49.118	0,973981	99,99%	96,39%	98,16%	99,98%
88	1.489	0,341596	99,18%	97,38%	98,27%	99,33%

Al reagrupar las etiquetas de L2 en sólo NORMAL y ataque, el resultado produjo $TN = 33.925$, $FP = 345$, $FN = 2.438$ y $TP = 71.890$. Estos valores corresponden a una exactitud de 97,437 %, una precisión de ataque de 99,522 %, una sensibilidad de ataque de 96,720 %, un F_1 de 98,101 % y un MCC de 0,94254. Durante el ajuste conjunto de umbrales se intentó mantener la sensibilidad de NORMAL en al menos 99 %. En la prueba se reconocieron correctamente 33.925 de 34.270 grupos normales, equivalentes a 98,9933 %. Hacían falta tres aciertos adicionales para alcanzar 99 %. Por ese motivo, se informa el objetivo como no alcanzado y no se redondea el resultado.

8.6.4 Operación, importancia y alcance

L1 registró p50 9,293 ms, p95 9,840 ms y p99 10,251 ms. Procesó 106,979 peticiones/s en línea, 3.344,960 peticiones/s con el flujo por lotes y 25.974,381 peticiones/s sólo en inferencia. El modelo ocupó 14,62 MiB. El uso máximo de memoria RSS fue 3.920,9 MiB y la extracción de variables demoró 2 min 4,9 s. La salida declara `GPU=False`.

En XGBoost, la **ganancia** es la reducción de la función objetivo atribuida a una división que usa una característica. La ganancia agregada suma ese aporte entre los doce clasificadores binarios. Las diez variables con mayor ganancia agregada fueron `sql_token_count`, `ldap_token_count`, `paren_count`, `ssi_token_count`, `quote_count`, `max_repeated_char_run`, `format_string_token_count`, `crlf_token_count`, `xpath_token_count` y `semicolon_count`. `invalid_percent_encoding_count` y `buffer_overflow_token_count` quedaron en cero.

L2 registró p50 7,125 ms, p95 7,627 ms, p99 8,083 ms y 138,929 casos/s en línea sobre una muestra de 10.000 grupos consolidados. El flujo por lotes alcanzó 1.832,262 casos/s. La inferencia sobre vectores ya construidos alcanzó 65.010,937 casos/s. El modelo ocupa 12,933 MiB y el uso máximo de memoria RSS fue 4.098,6 MiB (aproximadamente 4,00 GiB). El uso de CPU en línea fue 49,862 % de

los 32 núcleos lógicos visibles. Esto equivale a 15,956 núcleos y a aproximadamente 99,72 % de los 16 asignados. Catorce variables permanecen siempre en cero en la población procesada por L2; por tanto, no aportan variación a esta corrida, aunque se mantienen para respetar la representación evaluada.

La importancia por permutación global de L2 ubica primero `cookie_pair_count` (0,1146), `plus_encoding_count` (0,1085), `crlf_token_count` (0,0962), `n_body_params` (0,0911) y `max_param_name_len` (0,0861). La ganancia ponderada combina la ganancia de los doce modelos binarios, uno por cada etiqueta, de acuerdo con el soporte de cada salida; destacan `crlf_token_count`, `paren_count`, `sql_token_count`, `ldap_token_count` y `ssi_token_count`. Ambos órdenes indican qué variables están más asociadas con las predicciones o con el rendimiento en esta población. No demuestran que esas variables causen el resultado y no deben convertirse directamente en reglas.

Con las 90 características de request90, L2 obtuvo 97,604 % de $F_{1,\text{micro}}$ y 93,940 % de $F_{1,\text{macro}}$ en una prueba sin huellas compartidas con entrenamiento, validación o ajuste de umbrales, según la regla de quince campos.

8.7 Peticiones HTTP repetidas entre particiones de SR-BH 2020

La Tabla 8.10 resume la reconstrucción de las particiones. Una huella se calcula a partir de los quince campos HTTP normalizados de una petición. El panel (a) presenta el tamaño del conjunto de prueba y la cantidad total de unidades que coinciden con desarrollo bajo la regla de huella. El panel (b) separa este resultado entre tráfico normal y de ataque. En este segundo panel, cada porcentaje se calcula sobre el número de ejemplos de su grupo en prueba. Ninguna corrida reutiliza el mismo índice de fila. Lo que se mide es si dos eventos diferentes producen la misma huella después de aplicar las reglas limitadas de formato a esos campos. Las cinco corridas históricas cuentan filas originales; L2 cuenta grupos consolidados.

Tabla 8.10: Unidades de prueba que coinciden con desarrollo bajo la regla de huella.

(a) Cantidad total de coincidencias con desarrollo

Corrida	Unidades de prueba	Unidades coincidentes	Proporción total
Multiclase 9 familias	181.563	77.184	42,5109%
Multiclase top-14	180.483	76.959	42,6406%
Multiclase top-11	180.483	77.060	42,6965%
Multiclase top-10	180.483	77.127	42,7337%
Multietiqueta L1	181.563	76.966	42,3908%
Multietiqueta L2	108.598	0	0%

(b) Unidades coincidentes por grupo

Corrida	Normal coincidentes/prueba (%)	Ataque coincidentes/prueba (%)
Multiclase 9 familias	73.909/105.039 (70,36%)	3.275/76.524 (4,28%)
Multiclase top-14	73.959/105.039 (70,41%)	3.000/75.444 (3,98%)
Multiclase top-11	73.969/105.039 (70,42%)	3.091/75.444 (4,10%)
Multiclase top-10	73.957/105.039 (70,41%)	3.170/75.444 (4,20%)
Multietiqueta L1	73.915/105.039 (70,37%)	3.051/76.524 (3,99%)
Multietiqueta L2	0	0

La comparación de registros completos encuentra una coincidencia exacta en desarrollo para sólo 735 de las 181.563 filas de prueba multiclase base (0,4048 %) y para 719 de las 181.563 filas de L1 (0,3960 %). Por tanto, la mayor parte del 42 % corresponde a eventos distintos que producen la misma huella después de esas reglas. No corresponde a la reutilización de la misma fila completa.

Capítulo 9

Discusión

9.1 Información útil de una petición

La representación empleada contiene sólo datos de una petición HTTP: método, URI, protocolo, encabezados y cuerpo. En los corpus estudiados, esos campos permiten distinguir el tráfico normal de distintas categorías de ataque. Parte de esa señal también puede reflejar la forma en que cada corpus fue capturado, generado y etiquetado.

La ablación de CSIC registra $F_1 = 0{,}904$ con 25 variables y valores mayores al incorporar los ocho atributos de la representación ampliada. Como B2 selecciona el umbral sobre el mismo conjunto usado para informar el resultado, esos máximos se consideran exploratorios. En Harvard top-14, el $F_{1,\text{macro}}$ pasa de 80,81 % con 30 variables a 88,83 % con 84. M8 alcanza 98,68 % en TorpEda, 97,23 % en Harvard y 84,49 % en DS-Augmented-v2. Las corridas se ejecutaron una sola vez. Sus diferencias se describen como valores observados, sin estimar la variación entre semillas.

9.2 Diferencias entre las tres formulaciones

La clasificación binaria supervisada distingue peticiones normales y ataques, sin identificar la familia del ataque.

La clasificación multiclase asigna una categoría exclusiva. Cuando las etiquetas de origen se solapan, una reducción multiclase puede elegir una familia, agrupar varias o crear una clase compuesta para una

combinación exacta. La última opción conserva una coocurrencia conocida, pero exige incorporar de antemano una clase diferente para cada combinación admitida. La formulación multietiqueta mantiene, en cambio, una decisión separada por CAPEC y puede activar varias de ellas en una misma salida.

Esta diferencia aparece en los datos de Harvard. La reducción top-14 excluye 5.404 filas correspondientes a 46 combinaciones que no pertenecen a sus clases admitidas. El CVE se utiliza en la Sección 2.3 como ejemplo externo del tipo de información que una salida multietiqueta puede conservar, no como una vulnerabilidad evaluada de manera específica por los modelos.

En Harvard, M4, M5 y M6 emplean 58 características, la misma configuración principal de HistGradientBoosting y una prueba de 180.483 filas. Con top-14, M4 obtiene 88,81 % de $F_{1,\text{macro}}$. Al reunir CAPEC-153 con CAPEC-194 y CAPEC-272 con CAPEC-274 y su combinación, top-11 alcanza 94,40 %. Al integrar esos grupos en *Request Manipulation*, top-10 llega a 97,10 %. La agrupación explica el aumento: las predicciones entre categorías fusionadas dejan de contarse como errores. El costo es una salida menos detallada, porque el modelo ya no distingue por separado las CAPEC reunidas.

L1 y L2 conservan las mismas doce etiquetas CAPEC; entre estas corridas no se fusionan clases. L1 obtiene F_1 de 45,15 % para CAPEC-153 y 42,47 % para CAPEC-272; L2 alcanza 80,86 % y 58,04 %, respectivamente. Para CAPEC-272, con el umbral 0,349251, L2 logra 72,02 % de precisión y 48,60 % de sensibilidad. Su AP de 67,09 %, calculada sobre el orden de las puntuaciones y no sobre un único umbral, indica que la dificultad no se limita al punto de corte elegido. Ese umbral se fijó en la partición de calibración y se aplicó sin cambios a la prueba.

L2 supera a L1 en las métricas globales, pero la comparación corresponde a dos flujos completos: L2 consolida peticiones por huella, usa 90 variables en lugar de 30, separa la selección de configuración del ajuste de umbrales y no reentrena después. Por ello, los resultados no miden por separado el aporte de cada cambio. Esta comparación no se refiere a la agrupación multiclase del párrafo anterior.

9.3 Clases y etiquetas con mayor dificultad

Las métricas globales fueron altas en varias corridas, aunque algunas categorías mantuvieron errores importantes.

La reducción binaria tampoco reemplaza el análisis por clase. El sistema puede reconocer que existe un ataque y asignarle una familia incorrecta, lo que afecta la explicación y la prioridad de la respuesta.

El análisis conservado es cuantitativo: matrices de confusión, falsos positivos, falsos negativos y pares frecuentes de clases confundidas. Los límites de la revisión cualitativa se reúnen en la Tabla 9.2.

9.4 Efecto de separar las peticiones según la huella definida

En esta sección, dos registros se tratan como coincidentes bajo la regla adoptada cuando producen la misma huella a partir de los quince campos seleccionados y de las reglas limitadas de formato. Esta coincidencia operacional no implica identidad byte a byte ni equivalencia para cualquier aplicación.

Las repeticiones ya existen en la población de Harvard antes de particionar y no fueron creadas por sobremuestreo, técnica que aumenta la presencia de ejemplos minoritarios durante el entrenamiento. Las corridas históricas dividieron filas individuales, por lo que dos eventos diferentes con el mismo método, URI, encabezados y cuerpo podían quedar uno en desarrollo y otro en prueba.

La reconstrucción identifica esta situación en cuatro corridas multiclase y en L1. Cerca del 42 % de las filas de prueba produce la misma huella que alguna fila de desarrollo. La proporción llega a cerca del 70 % en el tráfico normal y al 4 % en los ataques. Sólo alrededor del 0,4 % coincide en las 38 columnas completas, lo que indica que la mayoría corresponde a filas diferentes que producen la misma huella.

El 42 % mide coincidencia bajo esa regla, no cuánto aumentaron las métricas ni cuántos ataques había visto el modelo. Ese efecto puede estimarse mediante una nueva ejecución que mantenga constantes los demás componentes y cambie únicamente la partición. El análisis se aplica a las corridas con huellas reconstruidas y no se extiende al flujo binario de Random Forest sobre Harvard, que siguió otro protocolo de registro.

L2 obtiene cero coincidencias de huellas entre sus particiones. Además, elimina repeticiones y combina mediante OR las etiquetas de cada grupo. Como ambas decisiones se aplican juntas, sus efectos no pueden separarse. Una evaluación posterior debería comparar primero una partición por grupos que conserve todos los eventos y luego otra que también consolide las repeticiones. Las seis intersecciones nulas corresponden a los seis pares posibles entre cuatro particiones y constituyen comprobaciones de integridad de una misma división; no son seis pruebas independientes de generalización.

9.5 Efecto del uso previo del conjunto de prueba

Las coincidencias bajo la regla de huella y el uso previo del conjunto de prueba son problemas distintos. El primero aparece cuando eventos con la misma huella se dividen entre desarrollo y prueba. El segundo ocurre cuando los resultados de prueba influyen en una decisión posterior.

B2 selecciona el umbral sobre el conjunto utilizado después para informar el resultado. En la secuencia multiclase, las importancias obtenidas sobre la prueba de M3 guían la reducción de 84

a 58 variables y M4 reutiliza esos índices.

L2 evita dentro de su protocolo tanto las huellas compartidas como la reutilización de la prueba final. Estas diferencias determinan el alcance asignado a cada corrida.

9.6 Comparación con antecedentes

Los resultados binarios de Random Forest en CSIC, TorpEda y Harvard coinciden con trabajos que encuentran información útil en la longitud, la composición, los parámetros y el contenido de las peticiones (Ingham e Inoue, 2007; Kruegel y Vigna, 2003; Wang y Stolfo, 2004). Cada modelo se entrena y evalúa dentro del mismo corpus, de acuerdo con el objetivo comparativo de esta línea.

La propuesta de Epp, Funk y Cappo (Epp et al., 2018) constituye un antecedente de detección de anomalías HTTP mediante One-Class SVM. Las diferencias respecto de otros trabajos se interpretan como contraste de flujos completos.

L2 alcanza 93,940 % de $F_{1,\text{macro}}$, 97,604 % de $F_{1,\text{micro}}$, 96,920 % de coincidencia exacta y 0,276 % de pérdida de Hamming, con particiones sin huellas compartidas bajo la regla definida. En las dos métricas comunes, L2 mejora los valores publicados para el flujo de dos fases con CatBoost de Sureda Riera et al.: 96,920 % frente a 88,445 % de coincidencia exacta, y 0,276 % frente a 1,703 % de pérdida de Hamming, donde menor es mejor (Sureda Riera et al., 2022a). L1 queda entre ambos con 96,531 % y 0,330 %. El artículo predice trece salidas, incluida NORMAL, mientras que L1 y L2 modelan doce CAPEC y derivan NORMAL; también cambian la unidad y las particiones. Por eso, la comparación describe los flujos completos y no el algoritmo aislado.

9.7 Rendimiento medido y límites para un WAF

L2 evalúa 10.000 grupos consolidados a 138,929 casos/s, con p50 de 7,125 ms y p99 de 8,083 ms. La inferencia sobre vectores ya construidos es más rápida que el flujo completo. La diferencia incluye extracción, decodificación, búsqueda de patrones y otros costos del procesamiento; la medición no separa el tiempo de cada operación.

Estas cifras caracterizan el entorno local de prueba y abarcan la extracción y la predicción. La red, TLS, los clientes concurrentes, las reglas adicionales, el registro y las decisiones de bloqueo

forman parte de una evaluación operativa posterior. Las pruebas fueron secuenciales o por lotes, sin carga sostenida; las implicaciones se reúnen en la Tabla 9.2.

9.8 Resultados en relación con los objetivos

La Tabla 9.1 resume la evidencia obtenida para cada objetivo específico.

Tabla 9.1: Relación entre los objetivos específicos y la evidencia disponible.

N.º	Objetivo resumido	Estado	Evidencia
1	Esquema común y diferencias de representación identificadas.	**Alcanzado**	Se definió el esquema $r = (m, u, v, h, b)$, se documentaron las listas de variables y se explicó cómo cada fuente aporta los campos disponibles.
2	Definir tareas según el nivel de detalle de las etiquetas.	**Alcanzado**	Se obtuvieron resultados binarios, multiclase y multietiqueta.
3	Implementar modelos y analizar cada resultado en su contexto.	**Alcanzado**	Se ejecutaron las tres familias y se analizaron los resultados como desempeño de flujos completos dentro de cada tarea.
4	Documentar particiones, identificar coincidencias de huella y registrar el uso previo de la prueba.	**Alcanzado**	L2 verifica seis intersecciones nulas de huellas.
5	Evaluar con métricas adecuadas a cada tarea.	**Alcanzado**	Métricas binarias, macro/micro, por clase y etiqueta, matrices de confusión y tasas de falsos positivos y falsos negativos.

N.º	Objetivo resumido	Estado	Evidencia
6	Medir el costo local y delimitar su alcance operativo.	**Alcanzado**	Se midieron latencia, tasa de procesamiento, CPU, memoria, tamaño y tiempo total, y se delimitó el alcance de esas mediciones.
7	Situar los resultados y examinar las características.	**Alcanzado**	Se presentan ablaciones, importancia por permutación o ganancia y contrastes de flujos con SR-BH.

El objetivo general se considera **alcanzado dentro del alcance definido**. Se implementaron y evaluaron las tres formulaciones, se documentaron sus diferencias, se analizaron los resultados en sus respectivos corpus y protocolos y se midió el costo de ejecución en los entornos registrados. Las conclusiones se circunscriben a esas condiciones experimentales.

9.9 Lecciones aprendidas

La revisión de las corridas históricas confirmó la importancia de separar los grupos cuyas coincidencias se desea excluir de la prueba. L2 aplicó una partición por huellas y mantuvo el análisis por etiqueta; así evitó que el resultado global ocultara las categorías débiles. Las mediciones operativas muestran una diferencia entre el flujo completo y la inferencia sobre vectores ya construidos, aunque no separan el costo de cada operación.

Estas experiencias muestran el valor de conservar, junto con cada modelo, la versión de los datos, las particiones, la configuración y las predicciones. Ese conjunto de materiales facilita la reproducción de una corrida y la revisión de sus decisiones.

9.10 Amenazas a la validez y limitaciones

La Tabla 9.2 concentra las medidas aplicadas y los límites que todavía condicionan la interpretación.

Tabla 9.2: Amenazas a la validez, medidas aplicadas y riesgo pendiente.

Amenaza	Medida aplicada	Riesgo pendiente
Contenido repetido y alcance de la huella	L2 agrupa por una huella de quince campos antes de dividir y comprueba seis intersecciones nulas.	La coincidencia no implica identidad byte a byte ni equivalencia semántica. Otra aplicación podría requerir otros campos. B1, B2 y M1–M8 no se repitieron con una prueba separada por huella.

Amenaza	Medida aplicada	Riesgo pendiente
Uso previo de la prueba	Las corridas se clasifican como exploratorias, finales de desarrollo o con prueba separada por huella, y se registra qué decisiones observaron la prueba.	Una repetición con una nueva partición reservada permitirá confirmar B1/B2 y M8.
Etiquetas ambiguas	Se conservan las etiquetas de origen, se documentan las reducciones y se cuantifican 288 grupos con etiquetas variables; 284 mezclan filas normales y de ataque. L2 conserva mediante OR todas las etiquetas observadas en cada grupo.	No hubo adjudicación manual. La variación puede deberse a contexto fuera de los quince campos, anotaciones incompatibles o ruido.
Incertidumbre estadística	Se informan soportes, métricas por clase o etiqueta y medidas adecuadas al desbalance.	Las repeticiones con varias semillas y los intervalos de confianza quedan como extensión para cuantificar la variación, especialmente en categorías con pocos ejemplos.
Generalización	Se emplean cuatro corpus de distinto origen y se ajusta un modelo independiente para cada uno.	Las conclusiones corresponden a esos corpus; una división temporal y la transferencia entre corpus permitirán estudiar la generalización futura.
Análisis de errores	Se presentan matrices de confusión, cantidades de falsos positivos y negativos y pares frecuentes de clases confundidas. La petición ilustrativa se identifica como sintética.	Una revisión cualitativa de muestras anonimizadas complementará el análisis cuantitativo.
Interpretación de las características	La importancia se calcula por corpus mediante permutación o ganancia y se informa junto con el contexto experimental.	La importancia muestra uso por parte del modelo, no causalidad. Puede reflejar la construcción del corpus y no debe convertirse directamente en una regla WAF.
Correspondencia con la anotación	Se excluyen de la entrada las columnas de etiqueta y se documenta el origen de la verdad de referencia.	En SR-BH, las etiquetas se apoyan en ModSecurity/Core Rule Set y revisión posterior, mientras que varias características contienen señales expertas cercanas a patrones de ataque. El rendimiento puede reflejar en parte concordancia con ese mecanismo.
Reproducción	Se documentan características, semillas, configuraciones finales, comandos, asignaciones, consumo y resúmenes SHA-256 de las entradas y programas principales.	Una versión posterior puede ampliar los manifiestos versionados a todas las etapas de desarrollo.
Uso operativo y ético	Se miden latencia, tasa de procesamiento y memoria y se delimitan los falsos bloqueos.	La evaluación en modo de observación con red, TLS, concurrencia y carga sostenida es el siguiente paso antes de considerar decisiones automáticas.

9.11 Qué explica la selección de Random Forest

La secuencia binaria aporta una razón experimental para la selección de Random Forest. El ajuste de hiperparámetros de OCSVM mostró que una configuración conservadora puede ocultar capacidad de detección; sin embargo, las pruebas posteriores no resolvieron el problema mediante el endpoint, la ampliación de variables o el cambio a Isolation Forest. El contraste supervisado permitió observar que las 57 características contenían señal discriminativa aprovechable por Random Forest. Esto respalda su uso como referencia para estudiar la representación dentro del alcance del trabajo, sin extender esa elección a todos los problemas de clasificación (Román Medina y González Arévalos, 2026, pp. 6-9).

La ausencia de mejora al añadir variables a OCSVM no implica que esas variables carezcan de utilidad general. Su contribución depende del modelo, de los datos disponibles durante el aprendizaje y del procedimiento de evaluación. Del mismo modo, la importancia Gini orienta la selección, pero no demuestra causalidad. El reentrenamiento con RF-10-Gini añade evidencia sobre la pérdida de rendimiento del conjunto reducido; no permite aislar el efecto de cada característica ni sustituye las repeticiones independientes.

La proximidad entre el F1 de Random Forest y el informado para OCS-WAF tampoco establece equivalencia experimental: difieren la arquitectura, la implementación y el protocolo. Comparar exhaustivamente RF con XGBoost, SVM binario u otros clasificadores requeriría una campaña propia con particiones, representaciones y presupuestos de ajuste comparables. En este trabajo se priorizó estudiar las características manteniendo una referencia estable. Las métricas binarias conservan su alcance exploratorio, especialmente cuando la prueba interviene en la selección del umbral; la secuencia histórica explica la decisión adoptada, pero no elimina esa limitación.

Capítulo 10

Conclusiones y trabajo futuro

10.1 Conclusiones

El trabajo implementó y evaluó un flujo para la clasificación binaria supervisada, multiclase y multietiqueta de peticiones HTTP. Se documentaron los datos, las representaciones, los modelos y los costos de ejecución. Los objetivos se consideran alcanzados dentro de los corpus y protocolos estudiados.

La exploración binaria permitió justificar la elección de Random Forest. OCSVM mejoró al ajustar sus hiperparámetros, pero las pruebas por endpoint, las ampliaciones de variables e Isolation Forest no resolvieron las dificultades observadas. Random Forest aprovechó la representación de 57 variables y se mantuvo como referencia para B1. Su F1 de 0,947 y las ablaciones respaldan esa decisión experimental, con el límite del uso previo de la prueba; no establecen superioridad general frente a otros algoritmos.

La revisión de las particiones históricas de Harvard encontró que entre 42,39 % y 42,73 % de las filas de prueba de cinco corridas producían la misma huella que alguna fila de desarrollo. L2 evitó esas coincidencias al asignar cada grupo completo a una sola partición. Este control no permite calcular cuánto cambiarían las métricas históricas sin repetir los entrenamientos.

M8 obtuvo F1 macro de 98,68 % en TorpEda, 97,23 % en Harvard y 84,49 % en DS-Augmented-v2. Los resultados globales de DS conviven con valores bajos para SSTI y XSS, por lo que la exactitud total no describe el comportamiento de todas las clases.

L2 alcanzó 97,604 % de F1 micro, 93,940 % de F1 macro y 96,920 % de coincidencia exacta global. Esta última incluye grupos normales con vector todo-cero y el rendimiento siguió siendo desigual entre etiquetas. En el entorno registrado, la mediana de latencia fue de 7,125 ms por grupo. La medición no incluye red, TLS, reglas ni solicitudes concurrentes.

Los campos de una petición individual aportaron información para distinguir tráfico normal de malicioso y asignar clases o etiquetas. La representación, la taxonomía de salida y la separación de los datos condicionaron tanto las métricas como su interpretación. Esos factores deben conservarse y controlarse al repetir el estudio.

10.2 Contribuciones y responsabilidades

Las contribuciones documentadas por este trabajo son:

• un esquema común de peticiones y salidas binarias, multiclase y multietiqueta, adaptado a cuatro fuentes con distinto nivel de detalle;

- conjuntos de características calculadas sólo con los campos de cada petición, incluida la lista completa y ordenada de las 90 variables multietiqueta;
- una revisión de las peticiones repetidas en SR-BH 2020 que distingue registros completos iguales de eventos que producen la misma huella calculada a partir de quince campos HTTP. El análisis cuantifica las coincidencias bajo la regla definida en las divisiones históricas y sustenta la partición por grupos de L2, cuyas seis intersecciones de huellas son nulas;
- una reconstrucción de la secuencia experimental que identifica las decisiones tomadas después de observar la prueba y permite distinguir resultados exploratorios, finales de desarrollo y evaluaciones con prueba reservada; y

• un análisis de las tres formulaciones con métricas generales, resultados por clase o etiqueta y mediciones operativas. Incluye una agrupación multiclase de diez categorías para SR-BH 2020 sin modificar las etiquetas CAPEC empleadas en la tarea multietiqueta.

10.2.1 Punto de partida y contribuciones de los autores

Los conjuntos de datos, las taxonomías CAPEC, los artículos citados, los algoritmos y las bibliotecas son material preexistente y se atribuyen a sus fuentes; no se presentan como creación de los autores. La contribución propia corresponde a la integración del flujo, las decisiones de representación y evaluación, la reconstrucción del historial, la auditoría de contenido repetido, el análisis de resultados y la redacción crítica descritos en la lista precedente.

Para esta versión se adopta una tabla de roles de contribución adaptada al TFG. Los autores se distribuyen las líneas experimentales principales y ambos comparten la conceptualización, la interpretación, la redacción y la aprobación final. La Tabla 10.1 deja explícita esta distribución.

Tabla 10.1: Declaración de contribuciones y responsabilidad de la versión final.

Rol adaptado	**Andrés Moisés Román Medina**	**Juan Andrés González Arévalos**
Conceptualización y metodología	Diseño conjunto del problema, alcance, protocolos y métricas	Diseño conjunto del problema, alcance, protocolos y métricas
Curación e integración de datos	CSIC y TorpEda; auditoría de huellas y control de calidad	SR-BH y DS-Augmented-v2; esquema común y control de etiquetas
Software y experimentación	Experimentación multiclase	Experimentación binaria supervisada y multietiqueta
Validación y reproducción	Particiones, métricas por clase y evaluación operativa	Configuraciones, trazabilidad de archivos y comprobaciones de reproducción
Análisis y visualización	Análisis estadístico descriptivo y visualización de resultados	Análisis de errores, comparación crítica y visualización de resultados
Redacción y revisión	Redacción, revisión crítica y aprobación final	Redacción, revisión crítica y aprobación final

Cristian R. Cappo A. realizó la supervisión académica, la revisión crítica y la orientación metodológica. Ambos autores asumen responsabilidad por las afirmaciones y los materiales incluidos en la versión final.

10.3 Trabajo futuro prioritario

1. **Repetir las evaluaciones finales con mayor control.** Repetir Harvard multiclase con partición mediante una regla de identidad reproducible, varias semillas y una línea de base compatible, manteniendo fijos el corpus, las variables y el criterio de selección.
2. **Revisar los conflictos de etiquetas de Harvard.** Examinar los 288 grupos variables, en particular los 284 que mezclan normal y ataque, y medir por separado el efecto de agrupar por huella y el de consolidar mediante OR.
3. **Analizar las categorías con menor rendimiento.** Revisar ejemplos anonimizados de las etiquetas CAPEC y las clases de DS más débiles para distinguir problemas de etiquetado, soporte o representación.
4. **Evaluar generalización temporal y externa.** Entrenar con un periodo y evaluar con otro y, cuando las taxonomías sean compatibles, repetir el análisis entre corpus sin reentrenamiento.
5. **Medir el flujo en modo de observación.** Integrar el modelo sin bloqueo, optimizar el extractor y medir concurrencia, carga sostenida, red, TLS y registro.

Cada repetición deberá conservar la versión de los datos, las variables, las particiones, la configuración, las dependencias, el hardware y las predicciones.

Apéndice A

Catálogos fixed30 y request90

La Tabla A.1 enumera las treinta variables fixed30. L2 reutiliza ese grupo y añade sesenta variables para formar `request90`.

Tabla A.1: Treinta variables fixed30 de L1.

1–15	16–30
`uri_len`	`angle_bracket_count`
`body_len`	`paren_count`
`query_len`	`semicolon_count`
`path_depth`	`percent_count`
`n_query_params`	`slash_backslash_count`
`max_param_value_len`	`max_repeated_char_run`
`uri_encoded_ratio`	`max_token_len`
`body_encoded_ratio`	`sql_token_count`
`invalid_percent_encoding_count`	`xss_token_count`
`double_encoding_count`	`crlf_token_count`
`uri_non_alnum_ratio`	`ldap_token_count`
`body_non_alnum_ratio`	`xpath_token_count`
`uri_entropy`	`ssi_token_count`
`body_entropy`	`format_string_token_count`
`quote_count`	`buffer_overflow_token_count`

`n_query_params` cuenta parámetros de consulta e `invalid_percent_encoding_count` secuencias % que no poseen dos dígitos hexadecimales válidos. La Tabla A.2 documenta las sesenta variables que request90 añade a su núcleo de treinta.

Tabla A.2: Sesenta variables adicionales de request90 en L2.

Nombre	Definición operativa
Método y versión HTTP (14)	
`method_is_get`	Indicador GET.
`method_is_post`	Indicador POST.
`method_is_head`	Indicador HEAD.
`method_is_put`	Indicador PUT.
`method_is_delete`	Indicador DELETE.

Nombre	Definición operativa
method_is_patch	Indicador PATCH.
method_is_options	Indicador OPTIONS.
method_is_trace	Indicador TRACE.
method_is_connect	Indicador CONNECT.
method_is_other_or_missing	Método distinto de los anteriores o ausente.
http_version_is_1_0	Indicador HTTP/1.0.
http_version_is_1_1	Indicador HTTP/1.1.
http_version_is_2_plus	Indicador de versión 2 o posterior declarada.
http_version_is_other_or_missing	Versión no reconocida o ausente.
Encabezados de petición (20)	
request_header_present_count	Cantidad de encabezados presentes.
request_header_total_len	Longitud acumulada de sus valores.
request_header_max_value_len	Máxima longitud de un valor.
request_header_control_char_count	Caracteres de control en valores de encabezado.
user_agent_len	Longitud de User-Agent.
user_agent_entropy	Entropía de User-Agent.
user_agent_product_count	Cantidad de productos/componentes reconocibles en User-Agent.
referer_present	Indicador de Referer presente.
referer_host_mismatch	Host de Referer distinto de Host cuando ambos pueden compararse.
host_is_ip_literal	Host expresado como dirección IP literal.
origin_present	Indicador de Origin presente.
origin_host_mismatch	Host de Origin distinto de Host.
cookie_len	Longitud del encabezado Cookie.
cookie_pair_count	Cantidad de pares nombre–valor de Cookie.
content_type_is_form	Tipo declarado como formulario URL-encoded.
content_type_is_json	Tipo declarado como JSON.
content_type_is_xml	Tipo declarado como XML.
content_type_is_multipart	Tipo declarado como multipart.
accept_item_count	Cantidad de tipos o rangos en Accept.
connection_nonstandard	Valor de Connection fuera del conjunto previsto por el extractor.
Cuerpo y parámetros (18)	
n_body_params	Cantidad de parámetros detectados en el cuerpo.
distinct_param_name_count	Cantidad de nombres de parámetros distintos.
duplicate_param_name_count	Cantidad de nombres distintos que aparecen más de una vez.
empty_param_name_count	Parámetros sin nombre.
empty_param_value_count	Parámetros sin valor.

Nombre	Definición operativa
max_param_name_len	Máxima longitud de nombre.
mean_param_value_len	Longitud media de valores.
query_body_shared_param_count	Nombres presentes tanto en consulta como en cuerpo.
abnormal_param_name_count	Nombres que incumplen el patrón conservador del extractor.
body_is_json	Forma del cuerpo compatible con JSON.
body_json_valid	Análisis sintáctico de JSON completado sin error.
json_max_depth	Profundidad máxima del objeto/arreglo JSON.
json_key_count	Cantidad de claves JSON.
body_is_xml	Forma del cuerpo compatible con XML.
xml_tag_count	Cantidad de etiquetas XML reconocidas.
body_is_multipart	Forma del cuerpo compatible con multipart.
multipart_part_count	Cantidad de partes multipart reconocidas.
content_type_body_mismatch	Discordancia entre tipo declarado y forma reconocida del cuerpo.
Codificación y normalización (8)	
plus_encoding_count	Signos más usados como codificación de espacio o presentes literalmente.
backslash_hex_escape_count	Escapes hexadecimales con contrabarra.
backslash_unicode_escape_count	Escapes Unicode con contrabarra.
html_entity_count	Entidades HTML numéricas o nominales reconocidas.
null_byte_evasion_count	Representaciones crudas o codificadas de byte nulo.
decode_replacement_char_count	Caracteres de reemplazo producidos al decodificar.
decode_once_length_delta_ratio	Cambio relativo de longitud tras una decodificación.
decode_twice_additional_delta_ratio	Cambio relativo adicional tras decodificar por segunda vez.

La cobertura de características de L2 registra como constantes: invalid_percent_encoding_count, buffer_overflow_token_count, method_is_delete, http_version_is_2_plus, http_version_is_other_or_missing, request_header_control_char_count, connection_nonstandard, body_is_json, body_json_valid, json_max_depth, json_key_count, multipart_part_count, backslash_unicode_escape_count y html_entity_count. Este resultado describe únicamente los datos usados después de agrupar las peticiones repetidas en esa corrida.

Apéndice B

Materiales y guía de ejecución

El repositorio de trabajo identificado para esta tesis es https://github.com/androceo123/TFG, en la rama implementacion. Git es el sistema de control de versiones utilizado; una **rama** mantiene una línea de desarrollo y un **commit** identifica una versión registrada del código. La guía se concentra en L2, el flujo final descrito en los capítulos de implementación y resultados.

El repositorio público http-anomaly-attack-classification organiza los materiales de apoyo por función. La Tabla B.1 indica dónde consultar código, evidencia, datos y guías de reproducción.

Tabla B.1: Materiales de apoyo y ubicaciones en el repositorio.

Material de apoyo	Ubicación
Código y lanzadores	`05_codigo/`
Evidencia de Arandu/Slurm	`02_auditoria/historial_slurm/`
Auditoría y resultados	`02_auditoria/ y 06_resultados_consolidados/`
Fuentes públicas de datasets	`03_datasets/README.md`
Entorno y reproducción	`07_reproducibilidad/ENTORNO_SLURM.md`

La Tabla B.2 permite comprobar dos archivos de la corrida final y un esquema ordenado de características.

Tabla B.2: Resúmenes SHA-256 de código y esquemas finales disponibles.

Archivo	SHA-256
`multietiquetaCorrida2.sh`	`ef2891d4cfc1e87626df220661c990b8fe6dab09ed3202ca3695e9f3e810da08`
`multietiquetaCorrida2.py`	`1120c44776ebd75ad65c98fa2c69a11a4ed086c9f6a6a81d69d2e085c0619ecb`
Esquema L2 `sr-bh-request90-v2`	`0bbc812611513de0952653b398f5533afa6933d32faf3cbfa8ac88ac12a8e51f`

L2 registra la configuración seleccionada, las particiones y los doce modelos binarios. En este caso, los materiales permiten reconstruir la secuencia funcional descrita en esta tesis.

Tabla B.3: Recursos de ejecución de los flujos finales.

ID	Lanzador identificado	Comando de envío documentado	Materiales disponibles
L2	`multietiquetaCorrida2.sh`	`sbatch --nodelist=c3 multietiquetaCorrida2.sh`	Programa, configuración, métricas y doce conjuntos de parámetros seleccionados por etiqueta.

Estructura y preparación. L2 parte de la raíz del proyecto waf-ml-starter/. El lanzador activa el entorno virtual .venv; las versiones se presentan en la Tabla 6.1 y las asignaciones en la Tabla 6.2. Antes de ejecutar se fijan las rutas de los datos, se comprueban los resúmenes disponibles y se mantienen juntos el programa, el lanzador, la lista ordenada de características, la taxonomía y la configuración.

Entradas y salidas. Los datos deben obtenerse desde las fuentes citadas en la Tabla 4.2. Las salidas esperadas incluyen, según la corrida, configuración, índices o huellas de partición, parámetros seleccionados, predicciones, métricas globales y por categoría, modelos, importancias y tiempos. Una reproducción debe comparar primero estas salidas estructurales y después las cifras; las pequeñas diferencias numéricas asociadas con biblioteca de ejecución o paralelismo se interpretan junto con las métricas y configuraciones registradas.

Uso responsable de los datos. Los corpus se obtienen desde las fuentes citadas y se emplean conforme a sus condiciones de acceso. Los campos potencialmente sensibles de SR-BH se mantienen bajo acceso restringido y los resultados se presentan de forma agregada.

Referencias

Apache HTTP Server Project. Path traversal and file disclosure vulnerability in Apache HTTP Server 2.4.49 (CVE-2021-41773). 2021. https://httpd.apache.org/security/vulnerabilities_24.html#CVE-2021-41773. Consultado el 27 de agosto de 2026.

Bishop, C. M. Pattern Recognition and Machine Learning. Information Science and Statistics, Springer, Nueva York, 2006. ISBN: 978-0-387-31073-2.

Bishop, M. (ed.). RFC 9114: HTTP/3. RFC Editor, junio de 2022. DOI: https://doi.org/10.17487/RFC9114. Consultado el 18 de agosto de 2026.

Breiman, L. Random Forests. Machine Learning, 45(1), 5–32, 2001. DOI: https://doi.org/10.1023/A:1010933404324.

Chandola, V., Banerjee, A. y Kumar, V. Anomaly Detection: A Survey. ACM Computing Surveys, 41(3), artículo 15, 2009. DOI: https://doi.org/10.1145/1541880.1541882.

Chen, T. y Guestrin, C. XGBoost: A Scalable Tree Boosting System. En Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ’16), San Francisco, California, Estados Unidos, 13–17 de agosto de 2016, ACM, pp. 785–794. DOI: https://doi.org/10.1145/2939672.2939785.

Chicco, D. y Jurman, G. The advantages of the Matthews correlation coefficient over F1 score and accuracy in binary classification evaluation. BMC Genomics, 21, artículo 6, 2020. DOI: https://doi.org/10.1186/s12864-019-6413-7.

CVE Program. Overview of the CVE Program. s. f. https://www.cve.org/about/overview. Consultado el 27 de agosto de 2026.

Epp, N., Funk, R. y Cappo, C. Anomaly-based Web Application Firewall using HTTP-specific features and One-Class SVM. Revista Eletrônica Argentina-Brasil de Tecnologias da Informação e da Comunicação, 2(1), número especial ERRC 2017, publicado en agosto de 2018. DOI: https://doi.org/10.5281/zenodo.1336812.

Equipo de tesis. Evaluación Cross-Dataset de un Pipeline Binario para Detección de Anomalías HTTP en WAF. Informe interno del TFG, Facultad Politécnica, Universidad Nacional de Asunción, 9 de junio de 2026b.

Equipo de tesis. Informe técnico de ablación de features: CSIC 2010 + RandomForest. Informe interno del TFG, Facultad Politécnica, Universidad Nacional de Asunción, 5 de junio de 2026a.

Fawcett, T. An Introduction to ROC Analysis. Pattern Recognition Letters, 27(8), 861–874, 2006. DOI: https://doi.org/10.1016/j.patrec.2005.10.010.

Fielding, R., Nottingham, M. y Reschke, J. (eds.). RFC 9110: HTTP Semantics. RFC Editor, junio de 2022. DOI: https://doi.org/10.17487/RFC9110. Consultado el 30 de julio de 2026.

Fielding, R., Nottingham, M. y Reschke, J. (eds.). RFC 9112: HTTP/1.1. RFC Editor, junio de 2022. DOI: https://doi.org/10.17487/RFC9112. Consultado el 30 de julio de 2026.

Friedman, J. H. Greedy Function Approximation: A Gradient Boosting Machine. The Annals of Statistics, 29(5), 1189–1232, 2001. DOI: https://doi.org/10.1214/aos/1013203451.

Gniewkowski, M., Maciejewski, H., Surmacz, T. y Walentynowicz, W. Sec2vec: Anomaly Detection in HTTP Traffic and Malicious URLs. Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing (SAC ’23), Tallinn, Estonia, 27–31 de marzo de 2023, ACM, pp. 1154–1162. DOI: https://doi.org/10.1145/3555776.3577663.

Information Security Institute (CSIC). HTTP Dataset CSIC 2010. Descripción del conjunto de datos, s. f. DOI: https://doi.org/10.23721/100/1478804. https://www.impactcybertrust.org/dataset_view?idDataset=940. Consultado el 30 de julio de 2026.

Ingham, K. L. y Inoue, H. Comparing Anomaly Detection Techniques for HTTP. En Proceedings of the 10th International Symposium on Recent Advances in Intrusion Detection (RAID 2007), Gold Coast, Australia, 5–7 de septiembre de 2007, Springer Berlin Heidelberg, LNCS 4637, pp. 42–62. DOI: https://doi.org/10.1007/978-3-540-74320-0_3.

Kruegel, C. y Vigna, G. Anomaly Detection of Web-based Attacks. En Proceedings of the 10th ACM Conference on Computer and Communications Security (CCS ’03), Washington, DC, Estados Unidos, 27–30 de octubre de 2003, ACM, pp. 251–261. DOI: https://doi.org/10.1145/948109.948144.

Li, P., Wang, Y., Li, Q., Liu, Z., Xu, K., Ren, J., Liu, Z. y Lin, R. Learning from Limited Heterogeneous Training Data: Meta-Learning for Unsupervised Zero-Day Web Attack Detection across Web Domains. Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS ’23), Copenhague, Dinamarca, 26–30 de noviembre de 2023, ACM, pp. 1020–1034. DOI: https://doi.org/10.1145/3576915.3623123.

Liu, F. T., Ting, K. M. y Zhou, Z.-H. Isolation Forest. En 2008 Eighth IEEE International Conference on Data Mining, Pisa, Italia, IEEE, pp. 413–422, 2008. DOI: https://doi.org/10.1109/ICDM.2008.17.

Luo, K., Xie, Y., Zhao, S. y Pan, J. WADBERT: Dual-channel Web Attack Detection Based on BERT Models. arXiv:2601.21893, 2026. https://arxiv.org/abs/2601.21893. Consultado el 1 de agosto de 2026.

National Institute of Standards and Technology. CVE-2016-1899 Detail. National Vulnerability Database, 2016. https://nvd.nist.gov/vuln/detail/CVE-2016-1899. Consultado el 27 de agosto de 2026.

Nguyen, M.-T.-A., Tong, V., Bannour Souihi, S. y Souihi, S. Deep Learning in NLP for Anomalous HTTP Requests Detection. Proceedings of the 19th International Conference on Network and Service Management (CNSM 2023), Niagara Falls, Ontario, Canadá, 30 de octubre–2 de noviembre de 2023, IEEE, pp. 1–8. DOI: https://doi.org/10.23919/CNSM59352.2023.10327888.

Osama, H., Elebiary, O., Qassim, Y., Amgad, M., Maghawry, A., Saafan, A. y Ghalwash, H. Enhanced Web Payload Classification Using WAMM: An AI-Based Framework for Dataset Refinement and Model Evaluation. IEEE Access, 14, 61512–61527, 2026. DOI: https://doi.org/10.1109/ACCESS.2026.3685450.

OWASP Community. Web Application Firewall. s. f. https://owasp.org/www-community/Web_Application_Firewall. Consultado el 30 de julio de 2026.

OWASP Core Rule Set Project. Anomaly Scoring. s. f. https://coreruleset.org/docs/2-how-crs-works/2-1-anomaly_scoring/. Consultado el 26 de agosto de 2026.

OWASP Top 10 Team. OWASP Top 10:2021. 2021. https://owasp.org/Top10/2021/. Consultado el 30 de julio de 2026.

OWASP Web Security Testing Guide. Map Application Architecture. s. f. https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/10-Map_Application_Architecture. Consultado el 26 de agosto de 2026.

Riverol, A., Betarte, G., Martínez, R. y Pardo, Á. Capturing the Security Expert Knowledge in Feature Selection for Web Application Attack Detection. En Proceedings of the 13th Latin-American Symposium on Dependable and Secure Computing (LADC 2024), Recife, Brasil, 26–29 de noviembre de 2024, ACM, pp. 153–158. DOI: https://doi.org/10.1145/3697090.3699797.

Román Medina, A. M. y González Arévalos, J. A. Informe metodológico: Evolución del enfoque binario y justificación de Random Forest. TFG, Facultad Politécnica, Universidad Nacional de Asunción, septiembre de 2026.

Saito, T. y Rehmsmeier, M. The Precision–Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. PLOS ONE, 10(3), e0118432, 2015. DOI: https://doi.org/10.1371/journal.pone.0118432.

Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J. y Williamson, R. C. Estimating the Support of a High-Dimensional Distribution. Neural Computation, 13(7), 1443–1471, 2001. DOI: https://doi.org/10.1162/089976601750264965.

Scikit-learn developers. average_precision_score. Documentación de scikit-learn 1.8.0, 2026a. https://scikit-learn.org/1.8/modules/generated/sklearn.metrics.average_precision_score.html. Consultado el 26 de agosto de 2026.

Scikit-learn developers. compute_class_weight. Documentación de scikit-learn 1.8.0, 2026b. https://scikit-learn.org/1.8/modules/generated/sklearn.utils.class_weight.compute_class_weight.html. Consultado el 26 de agosto de 2026.

Scikit-learn developers. HistGradientBoostingClassifier. Documentación de scikit-learn 1.8.0, 2026c. https://scikit-learn.org/1.8/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html. Consultado el 26 de agosto de 2026.

Scikit-learn developers. IsolationForest. Documentación de scikit-learn 1.8.0, 2026g. https://scikit-learn.org/1.8/modules/generated/sklearn.ensemble.IsolationForest.html. Consultado el 11 de septiembre de 2026.

Scikit-learn developers. LogisticRegression. Documentación de scikit-learn 1.8.0, 2026h. https://scikit-learn.org/1.8/modules/generated/sklearn.linear_model.LogisticRegression.html. Consultado el 11 de septiembre de 2026.

Scikit-learn developers. OneClassSVM. Documentación de scikit-learn 1.8.0, 2026e. https://scikit-learn.org/1.8/modules/generated/sklearn.svm.OneClassSVM.html. Consultado el 11 de septiembre de 2026.

Scikit-learn developers. Preprocessing data. Documentación de scikit-learn 1.8.0, 2026f. https://scikit-learn.org/1.8/modules/preprocessing.html. Consultado el 11 de septiembre de 2026.

Scikit-learn developers. RandomForestClassifier. Documentación de scikit-learn 1.8.0, 2026d. https://scikit-learn.org/1.8/modules/generated/sklearn.ensemble.RandomForestClassifier.html. Consultado el 27 de agosto de 2026.

Sokolova, M. y Lapalme, G. A Systematic Analysis of Performance Measures for Classification Tasks. Information Processing & Management, 45(4), 427–437, 2009. DOI: https://doi.org/10.1016/j.ipm.2009.03.002.

Sureda Riera, T., Bermejo Higuera, J.-R., Bermejo Higuera, J., Martínez Herraiz, J.-J. y Sicilia Montalvo, J.-A. A new multi-label dataset for Web attacks CAPEC classification using machine learning techniques. Computers & Security, 120, artículo 102788, 2022. DOI: https://doi.org/10.1016/j.cose.2022.102788.

Sureda Riera, T., Bermejo Higuera, J.-R., Bermejo Higuera, J., Martínez Herraiz, J.-J. y Sicilia Montalvo, J.-A. SR-BH 2020 multi-label dataset. Harvard Dataverse, versión 1.2, 6 de junio de 2022. DOI: https://doi.org/10.7910/DVN/OGOIXX. Licencia CC0 1.0. Consultado el 1 de agosto de 2026.

Tax, D. M. J. y Duin, R. P. W. Support Vector Data Description. Machine Learning, 54(1), 45–66, 2004. DOI: https://doi.org/10.1023/B:MACH.0000008084.60811.49.

The MITRE Corporation. About CAPEC. s. f.-a. https://capec.mitre.org/about/index.html. Consultado el 30 de julio de 2026.

The MITRE Corporation. CAPEC-126: Path Traversal. CAPEC Version 3.9, 2023. https://capec.mitre.org/data/definitions/126.html. Consultado el 26 de agosto de 2026.

The MITRE Corporation. CAPEC-153: Input Data Manipulation. CAPEC Version 3.9, 2023. https://capec.mitre.org/data/definitions/153.html. Consultado el 26 de agosto de 2026.

The MITRE Corporation. CAPEC-248: Command Injection. CAPEC Version 3.9, 2023. https://capec.mitre.org/data/definitions/248.html. Consultado el 26 de agosto de 2026.

The MITRE Corporation. CAPEC-34: HTTP Response Splitting. CAPEC Version 3.9, 2023. https://capec.mitre.org/data/definitions/34.html. Consultado el 26 de agosto de 2026.

The MITRE Corporation. CAPEC-88: OS Command Injection. CAPEC Version 3.9, 2023. https://capec.mitre.org/data/definitions/88.html. Consultado el 26 de agosto de 2026.

The MITRE Corporation. Common Attack Pattern Enumeration and Classification (CAPEC). s. f.-b. https://capec.mitre.org/. Consultado el 30 de julio de 2026.

The MITRE Corporation. New to CAPEC? s. f.-c. https://capec.mitre.org/about/new_to_capec.html. Consultado el 26 de agosto de 2026.

Thomson, M. y Benfield, C. (eds.). RFC 9113: HTTP/2. RFC Editor, junio de 2022. DOI: https://doi.org/10.17487/RFC9113. Consultado el 18 de agosto de 2026.

Torrano-Giménez, C., Pérez-Villegas, A. y Álvarez, G. TORPEDA: Una Especificación Abierta de Conjuntos de Datos para la Evaluación de Cortafuegos de Aplicaciones Web. Actas de la XII Reunión Española sobre Criptología y Seguridad de la Información (RECSI 2012), San Sebastián, España, 2012. https://recsi2012.mondragon.edu/es/programa/recsi2012_submission_73.pdf/at_download/file. Consultado el 26 de agosto de 2026.

Tsoumakas, G. y Katakis, I. Multi-Label Classification: An Overview. International Journal of Data Warehousing and Mining, 3(3), 1–13, 2007. DOI: https://doi.org/10.4018/jdwm.2007070101.

Wang, K. y Stolfo, S. J. Anomalous Payload-Based Network Intrusion Detection. En Proceedings of the 7th International Symposium on Recent Advances in Intrusion Detection (RAID 2004), Sophia Antipolis, Francia, 15–17 de septiembre de 2004, Springer Berlin Heidelberg, LNCS 3224, pp. 203–222. DOI: https://doi.org/10.1007/978-3-540-30143-1_11.

XGBoost developers. XGBoost Parameters. Documentación de XGBoost 3.2, 2026. https://xgboost.readthedocs.io/en/release_3.2.0/parameter.html. Consultado el 26 de agosto de 2026.

Yang, J., Wu, Y., Yuan, Y., Xue, H., Bourouis, S., Abdel-Salam, M., Prajapat, S. y Por, L. Y. LLM-AE-MP: Web Attack Detection Using a Large Language Model with Autoencoder and Multilayer Perceptron. Expert Systems with Applications, 274, 126982, 2025. DOI: https://doi.org/10.1016/j.eswa.2025.126982.

Zhou, Y., Zhu, H., Chai, Y., Liu, Y., Jiang, Y. y Qian, Y. Toward trustworthy web attack detection: An uncertainty-aware ensemble deep kernel learning model. Information & Management, 63(4), artículo 104325, 2026. DOI: https://doi.org/10.1016/j.im.2026.104325.